RT-Thread SDK 开发指南

V1.0.0

2023 年 8 月 3 日

上海富瀚微电子股份有限公司
地址：上海市徐汇区宜山路 717 号 2 号楼 5 楼 -6 楼邮编：200233
网址：http://www.fullhan.com
传真：021-64066786
电话：021-61121558
邮箱：sales@fullhan.com

版权所有 © 上海富瀚微电子股份有限公司。保留一切权利。
非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标申明

Fullhan®、**富瀚均为上海富瀚微电子股份有限公司的商标**

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意
由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

目录

目录 III

1 SDK 概述 1-1
1.1 SDK 简介 1-1
1.2 SDK 组成 1-1

2 SDK 安装与升级 2-1
2.1 交叉编译器安装 2-1
2.2 开发板环境配置 2-1

3 芯片启动过程 3-1
3.1 启动流程 3-1
3.2 U-Boot 功能 3-1
3.3 Flash 空间分配 3-1
3.4 内存空间分配 3-2

4 Demo 编译与运行 4-1
4.1 demo 简介 4-1
4.2 Flash 分区配置 4-1
4.2.1 定义 Flash 布局 4-1
4.2.2 擦写 Flash 分区 4-2
4.3 编译 demo 4-2
4.4 烧写协处理器固件 4-3
4.5 运行 media_demo 4-4

5 U-Boot 的开发和移植 5-1
5.1 编译 5-1
5.2 移植 5-1

6 RT-thread 实时操作系统开发与移植 6-1
6.1 开发与移植 6-1
6.1.1 板级配置 6-1
6.1.2 代码移植 6-1

7 Flash 镜像文件制作 7-1
7.1 简介 7-1
7.2 制作方法 7-1
7.3 配置文件说明 7-1
7.4 非加密镜像制作 7-3
7.5 加密镜像制作 7-5
7.6 EFUSE KEY 烧写 7-7
7.6.1 Efuse_cmd 与写入 efuse 之间数据的转换 7-9
7.6.2 CONTROL BIT 的含义 7-9
7.6.3 Efuse_cmd 的解析 7-10
7.7 efuse_cmd U-boot 编译 7-10
7.8 efuse_cmd U-boot-2016 编译 7-11
7.9 加密镜像升级与启动 7-12
7.10 多 DDR 参数支持的加密/非加密镜像制作 7-12
7.11 常见问题及解决方案 7-15
7.11.1 Efuse 烧写后无法加载进入到 U-boot 7-15
7.11.2 公钥，私钥错误 7-15

8 引脚复用配置指南 8-1
8.1 寄存器说明 8-1
8.2 配置方法 8-1

9 常用外设使用指南 9-1
9.1 WDT 使用指南 9-1
9.1.1 WDT 硬件原理 9-1
9.1.2 软件接口 9-1
9.1.3 注意事项 9-3
9.2 PWM 使用指南 9-3
9.2.1 概述 9-3
9.2.2 特点 9-3
9.2.3 工作方式 9-4
9.2.4 软件接口 9-4
9.2.5 接口调用 9-6

10 WiFi 使用指南 10-1
10.1 wifi 的编译 10-1
10.1.1 sdio wifi 的编译使能 10-1
10.1.2 usb wifi 的编译使能 10-4
10.2 wifi 的运行 10-5
10.2.1 sdio wifi 与开发板连接 10-5

10.2.2 运行示例 ------station 模式 10-6
10.2.3 运行示例 ------AP 模式 10-7
10.2.4 station 与 ap 模式并发运行 10-7
10.2.5 wifi 网口的桥接 10-8
10.2.6 usb wifi 和 sdio wifi API 说明 10-10

11 USB 使用指南 11-1
11.1 USB-4G 驱动使用指南 11-1
11.1.1 4G 驱动说明 11-1
11.1.2 4G 驱动使用 11-1
11.1.3 新增 4G 模块 11-3
11.1.4 其他说明 11-4
11.2 UVC&UAC 使用指南 11-5
11.2.1 目录说明 11-5
11.2.2 config 说明 11-5
11.2.3 驱动说明 11-6
11.2.4 驱动配置及编译 11-6
11.2.5 flash 镜像制作 11-8
11.2.6 应用操作 11-9
11.2.7 其他说明 11-14
11.2.8 修改 flash_layout 11-15
11.3 USB-GMAC 驱动使用指南 11-18
11.3.1 USB-GMAC 驱动说明 11-18
11.3.2 USB-GMAC 使用说明 11-18
11.3.3 USB-GMAC 功能测试 11-19

12 快速启动开发指南 12-1
12.1 快速启动代码的开发 12-1
12.2 快速启动代码的编译与运行 12-1
12.3 快速启动的 xmodem 镜像制作 12-3
12.4 快启模式与非快启模式的切换 12-4
12.5 人形识别功能应用 12-4
12.6 快速启动时间统计 12-5
12.7 快速启动常见问题及故障调试 12-6

13 系统扩展 13-1
13.1 文件系统 13-1
13.2 C++ 支持 13-2
13.3 shell 支持 13-2
13.4 负载统计 13-3

13.5 其它第三方组件 13-4

14 网络支持 14-1
14.1 socket 协议族支持 14-1
14.2 setsockopt 支持 14-1
14.3 getsockopt 支持 14-2
14.4 网络相关 ioctl 支持 14-3

15 Flash 操作指南 15-1
15.1 SPI Nor Flash 器件的移植 15-1
15.1.1 SPI-Nor U-boot 下的移植 15-1
15.1.2 SPI-Nor rtthread 下的移植 15-9
15.2 SPI Nand Flash 器件的移植 15-14
15.2.1 SPI-Nand Uboot 下的移植 15-14
15.2.2 SPI Nand Flash rtthread 下的移植 15-19

16 RTOS 调试指南 16-1
16.1 内存跟踪 16-1
16.2 运行日志 16-2
16.3 栈回溯 16-4
16.4 系统钩子 16-5
16.5 栈溢出检查 16-7
16.6 自定义命令 16-7
16.7 如何 debug 内存破坏问题, 以及各种随机 exception 问题? 16-9

17 附录开发调试工具 17-1
17.1 视频开发调试 CoolView 工具 17-1
17.1.1 连接开发板 17-1
17.1.2 修改 ISP 配置 17-1
17.2 LOGO 图片转换 PIC2ARGB 工具 17-1

18 附录 RTTv3 兼容接口 18-1
18.1 Posix 接口支持 18-1
18.2 不支持的 POSIX 接口 18-7
18.3 Linux 接口支持 18-9

19 附录 RTTv3 组件 License 列表 19-1

1 SDK 概述

1.1 SDK 简介

SDK 发布包是一个压缩包：FH_RT_Vx.x.x_yyyymmdd.rar，其中 Vx.x.x 是版本号，yyyymmdd 是 SDK 打包日期。

1.2 SDK 组成

整个 SDK 包由下列部分组成：

- 编译环境与工具链
- 预编译的镜像文件
- 调试工具与相关文档
- U-Boot 源码
- RT-thread 系统源码
- 音视频示例程序

目录结构如下:

```
SDK 目录结构如下:
    |-- images          # 可供烧写FLASH芯片的镜像文件Flash_xxx.img，预先编译好的RamBoot_xxx.bin、u-boot_xxx.bin
    |-- docs_tools      # 文档和工具
    |   |-- hardware
    |   |   |-- chip    # 存放芯片数据手册
    |   |   •-- board   # 存放开发板相关说明文档
    |   •-- software
    |       |-- board   # 存放应用开发相关文档
    |       •-- pc      # 交叉编译器和调试开发工具
    |-- bsp
    |   |-- uboot       # uboot源码
    |   •-- ramboot     # 针对不同芯片的第一阶段启动程序
    •-- rt-thread       # rt-thread系统和相关组件
        •-- app         # 应用程序示例代码
```

图片 1: SDK 目录结构

 版权所有 @ 上海富瀚微电子股份有限公司

2 SDK 安装与升级

2.1 交叉编译器安装

目前 SDK 仅支持在 Linux 操作系统上安装、运行。如果当前系统运行的是 Windows，建议在虚拟机中运行一个 Linux 发行版，目前已测试过的发行版是 Ubuntu 12.04 16.04 18.04 。
在 Linux 下安装开发环境步骤如下：

1. 解压 SDK 压缩包：

```
$ unrar  x  FH_RT_Vx.x.x_yyyymmdd.rar
```

   如果没有安装 unrar 工具，也可以使用 `$ rar x FH_RT_Vx.x.x_yyyymmdd.rar` 进行解压。

   如果以上工具都没有，可以先在 Windows 上解压后，再拷贝到 Linux 系统中（若使用的是 Linux 虚拟机，建议使用共享文件夹方式进行传输）。

2. 安装 arm 交叉编译器: 首先确保当前 Linux 系统上的用户拥有 sudo 权限或者 root 权限，然后按如下命令进行操作：

```
$ cd FH_RT_Vx.x.x_yyyymmdd/docs_tools/software/pc
$ ./install_toolchain.sh
$ source /etc/profile
```

   执行 `$ ./install_toolchain.sh` 时，根据提示输入数字序号，一般选择 2，然后回车。

   执行 `$ ./install_toolchain.sh` 后，交叉编译器即安装完成，若未生效，可重启系统。

至此，就可以正式开启基于 SDK 包的代码编译了。

2.2 开发板环境配置

开发板默认是从 SPI NOR Flash 启动。
通常，您拿到的开发板中已烧写 u-boot，如果没有的话，建议通过串口烧写带 u-boot 的 **xmodem_ 芯片型号 _ 其他参数.img**。

以下操作均使用网络来更新，且假设您的开发板上已烧写 u-boot。

1. 配置 tftp 服务器

   可以使用任意的 tftp 服务器，将 FH_RT_Vx.x.x_yyyymmdd/images 下的 **Flash_ 芯片型号 _ 其他参数.img** 拷贝到 tftp 服务器目录下；

2. u-boot 参数配置

   开发板上电后，敲任意键进入 u-boot。设置 serverip（即 tftp 服务器的 ip）、ipaddr（开发板 ip）

   ```
   set serverip xx.xx.xx.xx
   set ipaddr xx.xx.xx.xx
   ```

   ping 服务器 IP 地址，确保网络畅通。

3. 烧写 Flash_ 芯片型号 _ 其他参数.img 到 SPI Flash

   FH_RT_Vx.x.x_yyyymmdd/images 目录中包含了默认配置的 **Flash_ 芯片型号 _ 其他参数.img**，可以直接烧写：

   ```
   sf probe 0
   sf erase 0 100000
   tftp a0000000 Flash_芯片型号_其他参数.img
   sf write a0000000 0 100000
   reset
   ```

若想要制作新的 Flash.img 请参考Flash 镜像文件制作 章节

版权所有 @ 上海富瀚微电子股份有限公司

3 芯片启动过程

3.1 启动流程

开发板上电启动后，芯片内部按以下顺序启动：

1. 运行芯片内部 ROMBOOT
2. 加载 Flash Bootloader 分区，运行 Bootloader（包含 Ramboot）
3. 运行 U-Boot
4. 加载 RTT binary，如 media_demo.bin

3.2 U-Boot 功能

U-Boot 支持通过网络、SD 卡、串口、FLASH 加载 RTT binary，便于调试开发。

3.3 Flash 空间分配

分区	start - end Addr	Size(Bytes)	Sector Name
Bootloader 分区	0x00000000 ~ 0x0003FFFF	256K	flash header +ramboot
用户分区	0x00040000 ~ 0x0004FFFF	64K	ubootenv
用户分区	0x00050000 ~ 0x0009FFFF	320K	uboot
用户分区	0x000A0000 ~ 0x000FFFFF	384K	rtthread_fw.img
用户分区	0x00100000 ~ 剩余空间	-	rtthread.img

表格 1: 使用 U-Boot 时 SPI Flash 地址空间说明

分区	start - end Addr	Size(Bytes)	Sector Name
Bootloader 分区	0x00000000 ~ 0x0009FFFF	640K	flash header +ramboot
用户分区	0x000A0000 ~ 0x000FFFFF	384K	rtthread_fw.img
用户分区	0x00100000 ~ 剩余空间	-	rtthread.img

表格 2: 不使用 U-Boot 时 SPI Flash 地址空间说明

典型 Flash 空间的分配如上表所示。
用户可以根据实际使用情况调整 flash 空间分配，在制作 Flash.img 镜像文件时指定每一段的起始地址，具体方法详见Flash 镜像文件制作。

 版权所有 @ 上海富瀚微电子股份有限公司

3.4 内存空间分配

SDK 定义如下内存区域：

1. RTOS 内存块
2. DMA 内存块
3. SDK 内存块
4. 共享内存块
5. 协处理器内存块

图片 2: 内存空间分配

各内存区域功能定义如下：

1. RTOS 内存块为应用代码以及 OS 所共同占用的内存大小，用户需要根据自己的应用需求，评估定义此块内存的大小
2. DMA 内存块为底层 DMA 操作所需要的独享内存区域，默认定义为 128KB 大小，在 RTOS 内存之后
3. 共享内存块定义为 bootloader 和 RTOS 共享的内存区域，如果 bootloader 不需要给 RTOS 传递参数数据，可以定义此内存块大小为 0
4. 协处理器内存块，为运行协处理器固件所需要的内存区域，目前音频/人形检测等功能模块运行在协处理器上；如果不需要，此内存块大小也可以定义为 0. 如果需要，其占用固定的 512KB 大小。
5. SDK 内存块则是运行 SDK 代码所需要的内存区域，主要为视频处理内存区域；其占用大小可以根据应用需求评估计算得到。

在一般应用中可以根据应用需求评估 SDK 内存占用大小，以及确定 DMA/共享内存/协处理器内存大小后，最后将剩余内存全部分配给 RTOS 使用。
有关内存空间分配的定义，在板级目录下的 board_def.h 中。如果有修改需求，一般只需要配置某项功能的开启或关闭，以及调整内存区块的大小；除非您对 SDK 内容非常的了解，否则，请不要修改其余宏定义。

 版权所有 @ 上海富瀚微电子股份有限公司

4 Demo 编译与运行

4.1 demo 简介

应用示例代码位于 SDK/rtthread/app 目录下，下面是 demo 目录下的示例代码功能说明：

- helloworld：基本 app 框架，使用 posix 接口
- bsp_demo：基本外设操作示例代码
- uvc_demo：uvc 示例代码
- media_demo：视频出图应用示例，包含人形检测等功能

另外，在每个 demo 目录下均有该 Demo 的说明，参见各 Demo 的 ReadMe.txt 文档

4.2 Flash 分区配置

4.2.1 定义 Flash 布局

应用开发首先需要定义 flash 布局，即，更新`platform/$CHIP_ID/$BOARD/flash_layout.c`文件。
当前的分区表项中，每个表项有 4 个字段：

- 分区名称，是一个不超过 15 个字节的字符串，不同分区须使用不同的分区名称，加以区别。
- 分区大小，即该分区分配的大小，单位为字节，分区在 flash 中的偏移即为其前面所有分区大小的总和。
- 分区基本单元大小，即分区的擦写最小单元大小，单位为字节，目前仅支持 64K/4K 这 2 种基本单元。
- 分区属性，包含保留分区/应用代码分区/VBUS 固件分区/用户分区，以及用以标识根文件系统的根分区和最后定义分区（定义最后一个表项）。

由于 SDK 采用分段加载的启动方案，即部分代码是由**应用程序**进行加载的，因此，**应用程序**必须知道剩余的未加载代码在 flash 中的存储位置；**应用程序**代码所在分区由属性字段`PART_APPLICATION`标识，用户必须定义该表项，否则，应用代码可能无法正常启动。

对于支持 VBUS 应用的代码来说，用户还必须定义PART_ARCFIRM分区，该分区用于存放 VBUS 固件，在系统启动中，系统软件将会根据这个分区的定义进行 VBUS 固件加载。最后，如果需要系统在启动时自动挂载 JFFS2 文件系统作为根文件系统，可以为该分区定义PART_ROOT属性；系统会自动识别该属性字段，并尝试把该分区挂载为根文件系统。PART_RESERVED 与 PART_USER 无本质区别，一般用以区别系统分区（SDK）与应用分区（APP），用户在擦写 Flash 分区时，对于系统分区，务必慎重，否则，可能导致板子无法正常启动。
SDK 默认值：

- VBUS 固件存放于 flash 偏移 0xA0000 处，大小 0x60000
- 应用代码存放于 flash 偏移 0x100000 处，大小 0x300000

4.2.2 擦写 Flash 分区

Flash 分区可以在 UBoot 中进行擦写，使用 sf 命令：

```
sf probe 0     # 初始化 sf

sf erase flash_addr len    # 擦除 flash 上面 flash_addr 开始的
                           # 长度为 len 的区块内容

sf read ram_addr flash_addr len    # 将从 flash_addr 开始的
                                   # 长度为 len 的内容读入到
                                   # ram_addr 中

sf write ram_addr flash_addr len   # 将内存 ram_addr 中的长度为
                                   # len 的内容写入到 flash_addr
                                   # 开始的地址中，在写入 flash 前，
                                   # 必须执行 `sf erase`先擦除
                                   # 待写入的 flash 空间
```

4.3 编译 demo

在进行代码编译前，必须先进行代码配置，即：运行make ${CHIP_ID}_defconfig，来使用系统缺省的配置，或者是直接运行make menuconfig来进行菜单配置（其中，${CHIP_ID}替换为芯片名称，比如 fh8852v200）。
编译时，进入 $SDK/rt-thread 目录，执行：

```
$ make fhxxxx_defconfig
$ make appconfig          # 可选(SDK中仅media_demo/uvc_demo支持)
$ make
```

SDK 发布包默认编译 helloworld 示例代码，如果需要编译其它的示例代码，可以通过`make menuconfig`来进行配置选择。
编译结果存放在 2 个 out 目录中, 需要注意这 2 个路径的差别：

- BSP 中间文件 (lib) 存放于当前目录下的 out 文件夹
- 应用代码则存放于 Demo 的 out 目录下，如 app/helloworld/out

编译生成的可执行文件则存放在 Demo 目录下的 out/bin 子目录中。
若要清除编译文件，执行：

```
$ make clean
```

如果要修改配置，执行：

```
$ make menuconfig
```

> **注意**：人形检测功能依赖`vbus`，如果要支持人形检测，需要在`components packages`中打开`vbus`选项，否则会有编译报错。

4.4 烧写协处理器固件

运行音视频应用前，需要先烧写协处理器固件，该固件位于 SDK/images，以芯片型号为后缀，例如：

- rtthread_fw.img.FH8626V100
- rtthread_fw.img.FH885xV200

固件需要先烧写在 Flash 里，例如：

```
U-Boot>tftp  a1000000  rtthread_fw.img.FH8626V100
U-Boot>sf probe 0
U-Boot>sf erase A0000 60000
U-Boot>sf write a1000000 A0000 60000
```

其中，擦除命令中的第一个参数 A0000 为 offset，第二个参数 60000 为 size，这两个参数要与 flash 分区配置文件 flash_layout.c 里分区配置的具有 PART_ARCFIRM 属性的字段内容一致（注意：flash_layout.c 中仅有分区的大小信息，偏移值需要计算）。

4.5 运行 media_demo

编译 media_demo 前，应该首先配置 media_demo。主要是需要选择应用所使用的 sensor。
运行音视频应用前，需要先烧写协处理器固件，请按照烧写协处理器固件 的步骤操作。
编译成功的 demo 会在 $(DEMO)/out/bin 下生成 media_demo.bin 文件，需要将该文件数据传输到 DDR 内存的 a0000000 地址处，例如通过 tftp 命令：

```
U-Boot> tftp a0000000 media_demo.bin
```

然后使用 go 命令跳转到该地址执行：

```
U-Boot> go a0000000
```

 版权所有 @ 上海富瀚微电子股份有限公司

5 U-Boot 的开发和移植

5.1 编译

U-Boot 的源代码位于 SDK/bsp/uboot，编译方法如下：

1. 进入 SDK/bsp/uboot
2. 执行 make 命令，会自动解压 uboot.tar.gz，并进行编译

具体的编译命令请参见 SDK/bsp/uboot/芯片型号/Makefile 的内容。编译好的 uboot 二进制文件位于 SDK/bsp/uboot 芯片型号/uboot/_build 目录下，是 u-boot.bin 这个文件，这个文件用于制作 Flash.img。

5.2 移植

U-Boot 的板级配置文件是 include/configs/fhxxxx_devboard.h，其中 fhxxxx 根据使用的芯片名字来定，例如：fh8852v200_devboard.h，可根据实际硬件设计和需要，修改其中的默认配置，U-Boot 的驱动移植和开发请参考官方文档。

 版权所有 @ 上海富瀚微电子股份有限公司

6 RT-thread 实时操作系统开发与移植

6.1 开发与移植

Rt-thread 系统的源代码位于 SDK/rt-thread。

6.1.1 板级配置

板级配置相关文件位于 SDK/rt-thread/platform 目录下，根据芯片名字（或类型）和开发板类型分为不同的文件夹，例如：fh8856v200/app_board，在该目录下是针对该类型开发板的配置，其中：

- xxxxboard_iopad.h: 定义芯片的引脚定义
- board_def.h：定义内存功能布局，定义开启哪些驱动模块
- flash_layout.c：定义代码镜像在 flash 中的布局

另外，把对不同芯片/板级共有的启动代码放到了 platform/startup 目录下：

- board_startup.c：板级初始化代码，
- os_startup.c：定义 RTT 系统的初始启动流程
- sensor_create.c: 处理 sensor 的创建

把对不同芯片/板级共有的公共代码放到了 platform/common 目录下：

- pinctrl.c：板级引脚复用代码，
- timer.c：板级系统时钟定义/注册代码
- trap.c：板级异常处理代码
- board_info.c：板级驱动注册管理代码

把对不同芯片/板级共有的分段加载代码放到了 platform/loader 目录下：

- flash_load.c：应用代码分段加载代码

6.1.2 代码移植

在进行产品开发时，需要定义新的板级，仅需从芯片下面的板级目录拷贝一份代码进行修改即可：

- xxxxboard_iopad.h：根据产品定义，对芯片引脚进行定义，选择合适的芯片引脚

- board_def.h：定义 RTOS 内存大小，是否使用协处理器等
- flash_layout.c：定义 flash 的存储空间/布局

另外，还需要修改`platform/$CHIP_ID/$CHIP_ID_board_def.h`，来引用新定义的板级目录/文件

 版权所有 @ 上海富瀚微电子股份有限公司

7 Flash 镜像文件制作

7.1 简介

本章描述制作 xmodem.img 和 Flash.img 的方法，前者可以通过 RomBoot 的 uart 下载模式烧写，后者可以通过 Flash 烧录器直接烧写到 Flash 芯片上。
Flash.img 一般由 RamBoot、U-Boot、rtthread.bin 几个二进制文件合成。
Linux 系统下使用镜像生成工具 mkflashimgv4 (windows 系统下使用 mkflashimgv4.exe)，通过配置文件 ImageInfov4.ini 生成启动芯片所需的镜像。

7.2 制作方法

1. 准备各个二进制文件：
   - RamBoot：SDK/images 目录下有编译好的二进制文件 RamBoot.bin
   - U-Boot：SDK/images 目录下有编译好的二进制文件 u-boot.bin，也可以通过 U-Boot 源码编译
   - rtthread.bin：通过 SDK/rt-thread/app 目录下的源码编译得到
2. 将上述几个文件放入 SDK/docs_tools/software/pc/flashimg-tool 目录下
3. 准备对应 DDR 参数文件，复制 SDK/docs_tools/software/pc/flashimg-tool/对应芯片名称文件夹下的的 *.CSV 文件到 SDK/docs_tools/software/pc/flashimg-tool 目录下，更换芯片种类时注意需重新进行这一步骤
4. 进入 SDK/docs_tools/software/pc/flashimg-tool，执行：

```
./mkflashimgv4  ImageInfo_FH8xxx_xxx.ini
```

7.3 配置文件说明

前文制作方法所述目录下有芯片的配置文件。

mkflashimgv4 工具会解析.ini 配置文件，该配置文件定义了每段二进制镜像在 Flash 上的烧写位置和内存的加载位置等信息，格式如下：

```
[code1]
FilePath=文件名
ImageName=镜像文件名称
FlashAddress=FLASH 地址，16 进制格式
MemoryAddress=内存加载地址，16 进制格式
Entry=代码运行入口地址，16 进制格式

[param1]
FilePath=文件名
ImageName=镜像文件名称
FlashAddress=FLASH 地址，16 进制格式
```

各部分解释如下：

- [code1] 段描述含有可执行代码的镜像文件的信息，可以是 ramboot 或者是 uboot
- [param1] 段描述有配置参数生成的镜像文件的信息，例如 PLL 配置，DDR 配置等
- FilePath：镜像文件路径和文件名
- ImageName：会被 bootloader 利用打印解析到的镜像文件名
- FlashAddress：该镜像文件要写入到 flash 的地址
- MemoryAddress：该镜像文件加载到内存的地址
- Entry：该镜像文件运行的入口地址

可以添加多个 code 段、param 段，使用序号区别。最大总段数为 16 个

最好按照序号顺序排列各段，code 段和 param 段可以穿插书写。在最终 flash 镜像文件时 mkflashimgv4 工具会按照 FlashAddress 顺序重新排列各段。

由于该镜像工具只适用于 BootLoader 启动阶段，即加载 RamBoot 和 U-Boot，所以不建议用该镜像工具生成完整 Flash 镜像；另外，默认 ini 配置中大部分的 Type 选项不建议修改，可以修改 Type 值如下：

```
[code1]
...
Type = 0x10
...

[code2]
...
Type = 0x11
...
```

- `Type = 0x10` 表示该段内容为 RamBoot，RomBoot 将跳转到这一段代码继续执行。

- `Type = 0x11` 表示该段内容为 U-Boot，RamBoot 将跳转到这一段代码继续执行。

其中 `Type = 0x10` 段必须包含，否则 romboot 无法完成跳转　　生成的镜像将所有代码段合在一个 xmodem.img 文件中（其中头的部分做 128 字节对齐，代码段之间也做 128 字节对齐），通过 xmodem 协议发送。

```
[code1]
Compressed=Yes
...
```

- `Compressed=Yes` 表示该段内容已经压缩。
- 不写该字段 或 `Compressed=No` 表示该段内容未压缩。

压缩只能加入到 ramboot 之后的 code 段上，并且需要前一级对该压缩模式进行支持。

FH865X 之前的项目 ramboot 不支持解压缩，FH865X 仅支持 GZIP 解压缩。

bin 文件请先自行使用`gzip -9c u-boot.bin > u-boot.gz`后填入。

7.4 非加密镜像制作

1. 准备 u-boot.bin

   SDK/images 目录下有编译好的二进制文件 u-boot.bin，也可以通过 U-Boot 源码编译生成。

2. 准备 DDR 参数初始化配置文件

   根据 DDR 芯片选择配置文件，例如：DDR2_E14D5121632A_512M_480M.csv

3. 准备 ini 配置文件

   编辑生成 ImageInfo.ini 文件

```
[productroot]
MagicNumber=0x2a4c4232
Product=0x4442414f
Version=1
WriteFlash=Yes
AESMode=
AESIV=
AESKey=
PrivateKeyPath=
```

```
EnableCrypto=No ····· [1]

[param1]
Xmodem=Yes
FilePath= DDR2_E14D5121632A_512M_480M.csv ····· [2]
ImageName= E14D5121632A
FlashAddress=0x00001000
MemoryAddress=0x10000000
Entry=0xa0000000
Type=0x23 ····· [3]
Encrypt=No
BistSize=0x1b
KGDStrongTest=0

[code1]
Xmodem=Yes
FilePath=u-boot.bin
ImageName=Uboot
FlashAddress=0x00050000
MemoryAddress=0xa0800000
Entry=0xa0800000
Type=0x10
Encrypt=No
```

注释：

- [1] EnableCrytpo: Yes 表示此 Xmodem 是需要加密的；No 表示此 Xmodem 是非加密的，如果该字段没有，默认为非加密默认。
- [2] FilePath=DDR2_E14D5121632A_512M_480M.csv: 填写 *.csv 的路径。
- [3] Type : 表示该段的属性，此处填写 0x23。
  - 此处填写 0x20，表示不需要进行 DDR Training、KGD Test、Bist Test；
  - 此处填写 0x21，表示需要 KGD Test、Bist Test，但不需要 DDR Training；
  - 实际芯片该处应该填写 0x23，表示需要进行 DDR Training
  - 如果不需要自适应 DDR 参数，请勿将 Type 填写为 0x43。

4. 镜像生成

   将上述准备好的几个文件放入之前所述制作方法 中的文件夹，并按说明执行操作。最终，制作好的 xmodem.img 和 Flash.img 位于当前目录下。

 版权所有 @ 上海富瀚微电子股份有限公司

7.5 加密镜像制作

加密镜像支持加密启动方式，通过 EFUSE 内部存储的 CTRLBITS 确认是否使用加密模式，加载的代码是经过加密的密文，需要通过 AES 进行解密, 并通过摘要算法计算并校验此代码的数字签名。

加密启动准备和升级过程：

- 制作非加密和加密镜像
- 使用非加密镜像启动
- 在 uboot 中对芯片进行加密（每个芯片只能运行一次）
- 升级加密镜像
- 加载并运行加密程序

以下对内容和相关配置进行展开描述。

1. 准备 u-boot.bin

   SDK/images 目录下有编译好的二进制文件 u-boot.bin，也可以通过 U-Boot 源码编译生成。

2. 生成公钥私钥（已有秘钥，可忽略此步）

   - 私钥：rsa_private_key.pem，可以使用 linux 下 openssl 命令生成

   ```
   openssl genrsa -out rsa_private_key.pem 2048
   ```

   - 公钥：rsa_public_key.pem，可以使用 openssl 命令生成

   ```
   openssl rsa -in rsa_private_key.pem -pubout
               -out rsa_public_key.pem
   ```

   将生成的公钥私钥根据 SDK 类型放入以下其中一个目录中：

   - Linux SDK: SDK/board_support/tools/flash_upgrade
   - RTOS SDK: SDK/docs_tools/software/pc/flashimg-tool

3. 准备 DDR 参数初始化配置文件

   根据 DDR 芯片选择配置文件，例如：DDR3_M15F1G1664A_1G_600M.csv（以实际芯片为准）。

4. 准备 ini 配置文件

   根据 SDK 类型，在以下其中一个目录中创建 ImageInfo.ini 文件:

   - Linux SDK: SDK/board_support/tools/flash_upgrade

 版权所有 @ 上海富瀚微电子股份有限公司

- RTOS SDK: SDK/docs_tools/software/pc/flashimg-tool

以 CTR 加密模式为例，下面展示了一个典型的 ImageInfo 加密配置文件，具体如下：

```
[productroot]
MagicNumber=0x2a4c4232
Product=0x4442414f
Version=1
WriteFlash=Yes
AESMode=CTR ····· [1]
AESIV=112233445566778899aabbccddeeff11 ····· [2]
AESKey=603deb1015ca71be2b73aef0857d7781 ····· [3]
PrivateKeyPath=rsa_private_key.pem
EnableCrypto=Yes ····· [4]
SHALEN=256 ····· [6]

[param1]
Xmodem=Yes
FilePath= DDR3_M15F1G1664A_1G_600M.csv
ImageName= M15F1G1664A
FlashAddress=0x00001000
MemoryAddress=0x10000000 ····· [8]
Entry=0xa0000000 ····· [8]
Type=0x23
Encrypt=Yes ····· [5]
BistSize=0x1b
KGDStrongTest=0

[code1]
Xmodem=Yes
FilePath=rsa_public_key.pem
ImageName=PublicKey
FlashAddress=0x0000f000
MemoryAddress=0x10000000 ····· [8]
Entry=0x10000000 ····· [8]
Type=0x18
Encrypt=Yes
OutputHash=Yes ····· [7]

[code2]
Xmodem=Yes
FilePath=u-boot.bin
ImageName=Uboot
FlashAddress=0x00050000
MemoryAddress=0xa0800000 ····· [8]
Entry=0xa0800000 ····· [8]
Type=0x10
Encrypt=Yes
```

 版权所有 @ 上海富瀚微电子股份有限公司

注释：

- [1] AES 加密的模式，需要与烧写到 Efuse 中的 mode 方式一致。
- [2] AES 矢量，需要和烧写到 Efuse 中的矢量一致，具体描述在芯片 EFUSE 中烧写 KEY。
- [3] AES 密钥：需要和烧写到 Efuse 中的密钥一致，具体描述在芯片 EFUSE 中烧写 KEY。
- [4] EnableCrypto：
  - Yes 表示支持 rsa 非对称加密，会将所有的 code 段做成 rsa 加密
  - No 表示不支持 rsa 非对称加密。
- [5] Encrypt ：Yes 表示该镜像会被做成 aes 对称加密；No 表示该镜像不会被做成 aes 对称加密。
- [6] 设定制作签名用的源数据的 hash 方式 sha 的长度，FH8632、FH885x、FH8626v100 请设置为 160（SHA-1），其他芯片如 FH8862、FH885XV200 和 FH889X 请设置为 256（SHA-256），不设置此行默认为 160（SHA-1）。
- [7] 可选项，默认不写为 no，用于输出 public key 的 sha256 值，用于存储到 efuse 上做公钥的一致性检查。
- [8] MemoryAddress 和 Entry 部分地址禁止改动。

注意：制作加密镜像时，请同时将 Encrypt 和 EnableCrypto 选择为 Yes。

5. 加密镜像生成

   将上述准备好的几个文件放入之前所述制作方法中的文件夹，并按说明执行操作。最终，制作好的 xmodem.img 和 Flash.img 位于当前目录下。

7.6 EFUSE KEY 烧写

以 CTR 加密模式为例，在芯片未被加密前，需要先进入非加密模式进行 KEY 的烧录。可使用前文生成的非加密镜像，进入 uboot 下 (确认此 uboot 配置并支持 efuse_cmd, 详见 uboot 开发文档)，烧写 EFUSE KEY 和 CONTROL BITS，命令如下：

```
efuse_cmd 2 0 16 0\n\p
efuse_cmd 2 1 235 0\n\p
efuse_cmd 2 2 61 0\n\p
efuse_cmd 2 3 96 0\n\p
efuse_cmd 2 4 190 0\n\p
efuse_cmd 2 5 113 0\n\p
efuse_cmd 2 6 202 0\n\p
```

```
efuse_cmd 2 7 21 0\n\p
efuse_cmd 2 8 240 0\n\p
efuse_cmd 2 9 174 0\n\p
efuse_cmd 2 10 115 0\n\p
efuse_cmd 2 11 43 0\n\p
efuse_cmd 2 12 129 0\n\p
efuse_cmd 2 13 119 0\n\p
efuse_cmd 2 14 125 0\n\p
efuse_cmd 2 15 133 0\n\p ·******[1]

efuse_cmd 2 16 68 0\n\p
efuse_cmd 2 17 51 0\n\p
efuse_cmd 2 18 34 0\n\p
efuse_cmd 2 19 17 0\n\p
efuse_cmd 2 20 136 0\n\p
efuse_cmd 2 21 119 0\n\p
efuse_cmd 2 22 102 0\n\p
efuse_cmd 2 23 85 0\n\p
efuse_cmd 2 24 204 0\n\p
efuse_cmd 2 25 187 0\n\p
efuse_cmd 2 26 170 0\n\p
efuse_cmd 2 27 153 0\n\p
efuse_cmd 2 28 17 0\n\p ·******[5]
efuse_cmd 2 29 255 0\n\p
efuse_cmd 2 30 238 0\n\p
efuse_cmd 2 31 221 0\n\p ·******[2]

efuse_cmd 2 48 5 0\n\p ·********[3]

efuse_cmd 2 63 49 0\n ·********[4]
```

以上命令烧写完，对应的 EFUSE 内容为：

```
603deb1015ca71be2b73aef0857d7781
112233445566778899aabbccddeeff11
00000000000000000000000000000000
00000005000000000000000031000000
```

注释：

- [1] 烧写 AESKey：实例中的 0 - 15 行，代表写入 Efuse entry 0-15 中 AES 算法需要的 128-bit KEY。AES 中对应的实际数据为：603deb1015ca71be2b73aef0857d7781，对应 ImageInfov4.ini 文件中 AESKey 对应。
- [2] 烧写 AESIV：实例中的 16 -- 31 行，代表写入 Efuse entry 16-31 中 AES 算法需要的 128-bit IV。AES 中对应的实际数据为：112233445566778899aabbccddeeff11，对应 ImageInfov4.ini 文件中 AESIV，16-31 可以是 AESIV 也可以烧写 public key hash 值（entry16-47）。
- [3] 00000000000000000000000000000000，表示写入 Efuse entry 32-47 为空。
- [4] 烧写 Efuse Lock 信息：05- Efuse entry 48 为 CONTROL BITS，31-Efuse entry63 为 efuse key 锁定。00000005000000000000000031000000，
- [5] 向 Efuse 写入 0，硬件无法生效，但是可以被正确读取。

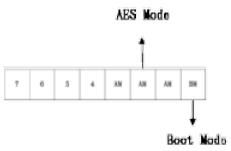
7.6.1 Efuse_cmd 与写入 efuse 之间数据的转换

Efuse_cmd 写入数据 4 个为一小组 16 个为一大组，小端写入十进制数据，每一个大组数据分别与 ini 文件中 AESKey/AESIV 中的 16 进制数据相对应。

Efuse cmd 与实际写入 efuse 中的数据的对应关系，以实例中的第 0 -- 4 行为例：

- 向 Efuse 的 Entry 0 中写入十进制 16，与 AESKey 值中的:603deb10 中的十六进制 0x10 相对应；
- 向 Efuse 中的 Entry 1 中写入十进制 235，与 AESKey 值中的：603deb10 的十六进制 0xeb 对应；
- 向 Efuse 中的 Entry 2 中写入十进制 61，与 AESKey 值中的：603deb10 的十六进制 0x3d 对应；
- 向 Efuse 中的 Entry 3 中写入十进制 96，与 AESKey 值中的：603deb10 的十六进制 0x60 对应。

7.6.2 CONTROL BIT 的含义



图片 3: 启动模式

 版权所有 @ 上海富瀚微电子股份有限公司

Operation mode select:
3'b000: ECB mode
3'b001: CBC mode
3'b010: CTR mode
3'b100: CFB mode
3'b101: OFB mode

图片 4: 控制比特位

如上图所示，共支持 5 种 AES 加密模式，Boot Mode 位为 1，除了第一种 ECB 外，其余模式都需要输入初始向量，初始向量固定存放在 EFUSE 中。

无论是否需要输入初始向量，在配置文件中的 AESIV= 该行都不应该被删除。

在制作加密镜像时，须引用这些参数。

注意：

- 解析 KEY 的位置为 Efuse entry 0-15，解析 IV 的位置为 Efuse entry 16-31。
- 当 AESIV from flash 位为 1 时，获取 IV 从 FLASH 中而不是 efuse 中。
- 当 ECB 模式或者其他模式 AESIV from flash 位为 1，PublicKeyCheck 为 1 时，会从 Efuse entry 16-47 读出公钥的 hash 值，进行比较。
- 在使用工具时，Publickey 生成是会有打印，获取后按 AESkey 类似的方法烧入 otp(请勿转换 hash 的大小端)。

7.6.3 Efuse_cmd 的解析

Efuse cmd 有四个参数：

1. 第一个参数：写入 Efuse 的指令
   - 2：向对应的 Entry 写入数据
   - 3：从对应的 Entry 读出数据
2. 第二个参数：指定将要进行操作的 Entry 号
3. 第三个参数：写入 Entry 的数据，以十进制表示
4. 第四个参数：指定要操作的 AESKEYID，默认写 0

7.7 efuse_cmd U-boot 编译

编译带有 efuse_cmd 的 uboot 需要进行以下的操作：

1. 解压发布包中的 U-boot 源码
2. 进入 $(U-boot)/include/configs/ 目录下的**芯片型号 _devboard.h** 文件

3. 在该文件下启用宏定义 (Line 323)

```
#define CONFIG_FH_AES
#define CONFIG_FH_ALGORITHM_CORE
#define CONFIG_HIK_AES_API
#define CONFIG_CMD_EFUSE_SIMPLE_TEST
#define CONFIG_CMD_HIK_AES_TEST
```

4. 保存设置
5. 回到 $(U-boot) 目录，使用芯片默认配置

```
$ make fhxxxx_devboard_config
```

6. 编译 U-boot，可在当前文件夹下得到 U-boot

```
$ make FHxxxx
```

7.8 efuse_cmd U-boot-2016 编译

编译带有 efuse_cmd 的 uboot 需要进行以下的操作：

1. 解压发布包中的 U-boot-2016 源码
2. 根目录下运行 make menuconfig
3. 进入 Command line interface 菜单
4. 打开 Enable efuse debug cmd 选项,用于使能 CONFIG_CMD_EFUSE_SIMPLE_TEST 宏
5. 保存设置
6. 编译 U-boot，可在当前文件夹下得到 U-boot

```
$ make FHxxxx
```

 版权所有 @ 上海富瀚微电子股份有限公司

7.9 加密镜像升级与启动

重启后立刻双击回车，出现

```
U-Boot> ROM:    Use nor flash.
ROM:    Init cipher.
ROM:    Download start, enter mode
ROM:    xmodem start
ROM:    Please upload firmware using xmodem
ROM:    Waiting...
ERR:    203
```

图片 5: 通过 xmodem 升级镜像

将升级镜像文件拖入，选择 Send Xmodem… 开始传输。 待传输完成后，重启，自动进入 uboot。

正确烧录 Efuse Key 与 Efuse IV 后，可以正常启动至 U-boot。看到 U-boot 打印后，表示加密启动正常。

Efuse 正常启动打印如下图：

```
U-Boot> ROM:    Use nor flash.
ROM:    Init cipher.
ROM:    Download start, enter mode
ROM:    Ok

U-Boot 2010.06-00014-g5512668-dirty (Jun 20 2018 - 15:39:57)

DRAM:  64 MiB
[ALGO]: 'Crypto' register name is aes-ecb
[ALGO]: 'Crypto' register name is aes-cbc
[ALGO]: 'Crypto' register name is aes-ctr
[ALGO]: 'Crypto' register name is aes-ofb
[ALGO]: 'Crypto' register name is aes-cfb
[ALGO]: 'Crypto' register name is des-ecb
[ALGO]: 'Crypto' register name is des-cbc
[ALGO]: 'Crypto' register name is des-ofb
[ALGO]: 'Crypto' register name is des-cfb
[ALGO]: 'Crypto' register name is des3-ecb
[ALGO]: 'Crypto' register name is des3-cbc
[ALGO]: 'Crypto' register name is des3-ofb
[ALGO]: 'Crypto' register name is des3-cfb
[ALGO]: 'Crypto' find name is aes-ecb
find driver name is aes-ecb
encry ok
SF: Got idcode ef 40 15 00 00
use default flash ops...
spic use FH_SPIC_QUAD_READ && 3bytes mode
In:    serial
Out:   serial
Err:   serial
MMC:   FH_MMC: 0
Card did not respond to voltage select!
No EMMC device found!!!
Net:   set to RMII
FH EMAC
Hit any key to stop autoboot:  0
U-Boot>
```

图片 6: Efuse 启动打印

7.10 多 DDR 参数支持的加密/非加密镜像制作

FH8862、FH889X、FH885xv200 和 FH885x 系列支持多 DDR 参数支持的加密/非加密镜像制作，该镜像支持自动识别并且适配 DDR 的功能，此功能可以避免使用错误 DDR 参数导致系统无法启动以及由于不同 DDR 参数的差异导致产生多份 Xmodem 等问题。

该镜像制作步骤可参考本章非加密镜像制作、加密镜像制作的内容，唯一需要注意的是 **ImageInfo.ini** 文件的配置，此处以非加密启动的配置文件 ini 为例，进行展示。

创建 ImageInfo.ini 文件，以下是一个典型的 ImageInfo 适配 DDR 参数配置文件，具体如下：

```
[productroot]
MagicNumber=0x2a4c4232
Product=0x4442414f
Version=1
WriteFlash=Yes
AESMode=
AESIV=
AESKey=
PrivateKeyPath=
EnableCrypto=No

[param1]
Xmodem=Yes
PackageID=0x0007840d ••••••••••••••••••••••••••••••••••••• [1]
FilePath= DDR3_M15F1G1664A_1G_792M.csv
ImageName=M15F1G1664A
FlashAddress=0x00001000
MemoryAddress=0x10005000
Entry=0x10005000
Type=0x43 ••••••••••••••••••••••••••••••••••••••••••••••••• [2]
Encrypt=Yes
BistSize=0x1b
KGDStrongTest=0

[param2]
Xmodem=Yes
PackageID=0x0007840c
FilePath=DDR2_E14D5121632A_512M_528M.csv
ImageName=E14D5121632A
FlashAddress=0x00002000
MemoryAddress=0x10005000
Entry=0x10005000
Type=0x43
Encrypt=Yes
BistSize=0x1a
KGDStrongTest=0

[param3]
Xmodem=Yes
PackageID=0x0007840e
FilePath=DDR3_W632G16NG_2G_792M.csv
ImageName=W632G16NG
FlashAddress=0x00003000
MemoryAddress=0x10005000
Entry=0x10005000
Type=0x43
Encrypt=Yes
BistSize=0x1c
```

```
KGDStrongTest=0

[param4]
Xmodem=Yes
PackageID=0x0007840f
FilePath=DDR3_W632G16NG_2G_792M.csv
ImageName=W632G16NG
FlashAddress=0x00004000
MemoryAddress=0x10005000
Entry=0x10005000
Type=0x43
Encrypt=Yes
BistSize=0x1c
KGDStrongTest=0

[code1]
Xmodem=Yes
FilePath=RamBoot.bin ••••••••••••••••••••••••••••••••••• [3]
ImageName=ramboot
FlashAddress=0x00010000
MemoryAddress=0x10000000 •••••••••••••••••••••••••••••• [4]
Entry=0x10000000 •••••••••••••••••••••••••••••••••••••• [4]
Type=0x10
Encrypt=Yes

[code2]
Xmodem=Yes
FilePath=u-boot.bin
ImageName=Uboot
FlashAddress=0x00050000
MemoryAddress=0xa0800000
Entry=0xa0800000
Type=0x11
Encrypt=Yes
```

注释：

- [1] 富瀚平台使用自适应 DDR 的特殊标识码，在任何情况下都不要改动。
- [2] 富瀚平台识别自适应 DDR 标识 Type，请勿随意改动。
- [3] RamBoot.bin 是支持自适应 DDR 功能中的必要组件，必须写在 ini 文件中。
- [4] MemoryAddress 和 Entry 部分地址禁止改动。

将上述准备好的几个文件放入之前所述制作方法中的文件夹，并按说明执行操作。最终，制作好的 xmodem.img 和 Flash.img 位于当前目录下。

 版权所有 @ 上海富瀚微电子股份有限公司

7.11 常见问题及解决方案

7.11.1 Efuse 烧写后无法加载进入到 U-boot

```
ROM:    Use nor flash.
ROM:    Init cipher.
ROM:    Download start, enter mode
ROM:    216
ROM:    Error
```

图片 7: Efuse 无法启动

当出现上述打印时，可能是由于以下原因造成无法启动：

- Efuse 烧录的加密模式和配置文件 ini 中的加密模式不符
- Efuse 烧录的 AESIV 和配置文件 ini 中的 AESIV 不符
- Efuse 烧录的 AESKey 和配置文件 ini 中的 AESKey 不符

由于是加密启动，如果配置文件和烧写进 Efuse 中的密钥和 IV 不匹配，则无法进入 U-boot，此处未作错误打印是为了避免被破解。

7.11.2 公钥，私钥错误

编译 ini 会报 RSA.importKey 错误。
强制修改

```
ImageName=PublicKey
FlashAddress=0x0000f000
```

内容时会出现

```
U-Boot> reset
resetting ...
ROM:    Use nor flash.
ROM:    Init cipher.
ROM:    Init DDR..Training done.
ROM:    216
```

图片 8: key 错误无法启动

 版权所有 @ 上海富瀚微电子股份有限公司

8 引脚复用配置指南

通过配置 PinMux 相关的寄存器，可以修改引脚复用、上下拉、驱动能力等。

8.1 寄存器说明

与 PinMux 操作有关的寄存器 bit 定义如下：

Bits	Name	Default	Description
6:4	DS	0x0	驱动能力
16	PD	0x0	上拉
20	PU	0x0	下拉
27:24	MFS	0x0	复用功能选择

对于驱动能力的选择范围为 0~7，一般情况下，引脚的输出频率越高 DS 的值要求越高，参考值如下：

IO 频率	GPIO	18OD33(1.8V)	18OD33(3.3V)	1.8V	3.3V
1M	0	0	0	0	0
10M	0	0	0	1	1
25M	0	0	0	10	10
50M	0	1	10	100	11
100M	1	10	100	111	101
150M	1	100	101	111	111
200M	10	101	110	X	111

表示在指定电压域下，最大 IO 翻转频率对应的 DS 建议取值（三位二进制）。比如 SD_CLK 引脚，其输出电压为 1.8V，输出频率 50M，则 DS 可以选取 0b100。

8.2 配置方法

PinMux 模块，有三种方法可以配置复用关系。

直接配置寄存器

- uboot 下

  通过 uboot 的 mw 命令直接修改 PMU 寄存器，需要对照 PMU 文档修改

```
mw f00000c0 101000
mw f00000c4 1101000
```

 版权所有 @ 上海富瀚微电子股份有限公司

- rt-thread 下

  通过 msh 命令在 rt-thread 下直接访问 PMU 寄存器

```
memrw -w 0xf00000c0 0x101000
memrw -w 0xf00000c4 0x1101000
```

通过命令配置

- UBoot 下

  在 UBoot 命令行下支持选择 device 和 mux

```
pinctrl sdev 设备名称
pinctrl smux 设备名称 引脚名称 编号
pinctrl printall
```

  编号指从哪个 mux 出来，具体要看代码，举例：

```
pinctrl sdev SD0
pinctrl smux SD1 SD1_CLK 0
pinctrl printall
```

- rt-thread 下

  在 msh 命令行下通过 pinctrl 命令同样可以修改 pinmux

```
pinctrl -d 设备名称
pinctrl -m 设备名称 引脚名称 编号
pinctrl -p 引脚名称 up|down|none
pinctrl -f 引脚名称 val[0~7]
pinctrl -l
```

  编号指从哪个 mux 出来，具体要看代码，举例：

```
pinctrl -d GPIO61
pinctrl -m GPIO61 GPIO61 0
pinctrl -p GPIO61 up
pinctrl -f GPIO61 3
```

 版权所有 @ 上海富瀚微电子股份有限公司

修改 iopad.h 文件

在 uboot 和 rt-thread 各自板级目录下都能够找到 iopad.h 或者 fhxxxx_iopad.h 文件，他们分别表示不同的芯片封装。根据前面讲的代码结构，当我们想要切换某个设备的时候，一般只需要修改 iopad.h 文件中的 fh_pinctrl_selected_devices 数组。

其中每一个字符串对应一个 PINCTRL_DEVICE，当然我们有时候需要去选择设备的每一个功能从芯片的哪一个引脚接出来，这就需要去修改 DEVICE 里的具体配置了，以 SPI 为例：

```
PINCTRL_MUX(SSI1_CLK, 0, &PAD37_SSI1_CLK, &PAD42_SSI1_CLK);
PINCTRL_MUX(SSI1_RXD, 0, &PAD36_SSI1_RXD, &PAD41_SSI1_RXD);
PINCTRL_MUX(SSI1_TXD, 0, &PAD35_SSI1_TXD, &PAD40_SSI1_TXD);
PINCTRL_MUX(SSI1_CSN_0, 0, &PAD38_SSI1_CSN_0,
            &PAD43_SSI1_CSN_0);
PINCTRL_MUX(SSI1_CSN_1, 0, &PAD39_SSI1_CSN_1,
            &PAD44_SSI1_CSN_1);
PINCTRL_DEVICE(SSI1, 4, &MUX_SSI1_CLK, &MUX_SSI1_RXD,
                &MUX_SSI1_TXD, &MUX_SSI1_CSN_0);
```

这里 SPI1 总共有定了有 4 个脚，CSN1 因为用不到暂时删除了，从代码定义可以看到 SPI1_CLK 既可以从 PAD37 出来也可以从 PAD42 出来，现在默认选中 0，那么到底该怎么选择功能引脚呢？我们先来看看 PAD37 和 PAD42 上具体有什么功能：

```
PINCTRL_FUNC(ARM_JTAG_TCK, 37, FUNC0, PUPD_UP, 0);
PINCTRL_FUNC(SSI1_CLK, 37, FUNC1, PUPD_UP, 0);
PINCTRL_FUNC(SSI2_CLK, 37, FUNC2, PUPD_UP, 0);
PINCTRL_FUNC(PWM2, 37, FUNC3, PUPD_UP, 0);
PINCTRL_FUNC(GPIO2, 37, FUNC4, PUPD_UP, 0);
PINCTRL_FUNC(ARC_JTAG_TDI, 42, FUNC0, PUPD_UP, 0);
PINCTRL_FUNC(SSI1_CLK, 42, FUNC1, PUPD_UP, 0);
PINCTRL_FUNC(PAE_JTAG_TDI, 42, FUNC2, PUPD_UP, 0);
PINCTRL_FUNC(GPIO7, 42, FUNC3, PUPD_UP, 0);
```

PINCTRL_FUNC 宏的参数中，分别代表引脚名称、PAD 号、当前 PAD 的第几功能、上下拉、驱动能力。如果需要修改上下拉和驱动能力，可以直接修改。

我们知道，一个芯片引脚在同一时刻只能复用成一种功能。PAD37 如果被复用成 SPI1_CLK 的话，ARM_JTAG、PWM2，GPIO2 和 SPI2 就不能用了（当然这些功能还有其他引脚可以出，这里只是为了举例暂时不考虑），很多主要的功能都被占用；而 PAD42 相对来说就好一些，它和 ARC_JTAG，PAE_JTAG 还有 GPIO7 复用，JTAG 一般只是在初期调试的时候会用到，做成产品之后是必须关闭的，这样看来 SPI1_CLK 从 PAD42 出

 版权所有 @ 上海富瀚微电子股份有限公司

来会比较好，代价只是 GPIO7 不能使用。

同理，SPI1 的其他引脚也是和 ARC_JTAG，PAE_JTAG 复用的，所以分别切换到 PAD40 ~ PAD43 上，代价是 GPIO5 ~ GPIO8 不能使用。

示例代码如下：

```
PINCTRL_MUX(SSI1_CLK, 1, &PAD37_SSI1_CLK, &PAD42_SSI1_CLK);
PINCTRL_MUX(SSI1_RXD, 1, &PAD36_SSI1_RXD, &PAD41_SSI1_RXD);
PINCTRL_MUX(SSI1_TXD, 1, &PAD35_SSI1_TXD, &PAD40_SSI1_TXD);
PINCTRL_MUX(SSI1_CSN_0, 1, &PAD38_SSI1_CSN_0,
            &PAD43_SSI1_CSN_0);
PINCTRL_MUX(SSI1_CSN_1, 1, &PAD39_SSI1_CSN_1,
            &PAD44_SSI1_CSN_1);
```

修改完毕之后，最好再检查一下 fh_pinctrl_selected_devices 数组，看下有没有 GPIO5 ~ GPIO8 也被选了进来，如果有的话就会造成冲突。

我们甚至可以按自己需求设计不同的设备，比如：

```
PINCTRL_DEVICE(SD0, 8, &MUX_SD0_CLK, &MUX_SD0_CD,
                &MUX_SD0_CMD_RSP, &MUX_SD0_WP,
                &MUX_SD0_DATA_0, &MUX_SD0_DATA_1,
                &MUX_SD0_DATA_2, &MUX_SD0_DATA_3);
PINCTRL_DEVICE(SD0_1BIT, 5, &MUX_SD0_CLK, &MUX_SD0_CD,
                &MUX_SD0_CMD_RSP, &MUX_SD0_WP,
                &MUX_SD0_DATA_0);
PINCTRL_DEVICE(SD0_1BIT_NOCD, 5, &MUX_SD0_CLK, &MUX_GPIO10,
                &MUX_SD0_CMD_RSP, &MUX_SD0_WP,
                &MUX_SD0_DATA_0);
```

同样是 SD0，在不同的应用下，可能会只要用到一线模式，有时候可能不用原来的 CD 脚而是选取了一个 GPIO 来做 CD 检测，这时候就可以多加一个设备，随后在 fh_pinctrl_selected_devices 中选中这个设备。

注：RTOS SDK v3 版本及以上才支持 pupd、ds 命令。

 版权所有 @ 上海富瀚微电子股份有限公司

9 常用外设使用指南

9.1 WDT 使用指南

9.1.1 WDT 硬件原理

WDT 超时范围

由于 IP 配置的原因，WDT 超时 counter 范围有 16 个固定的档位。

以 wdt 输入时钟为 1800000Hz 为例，计算公式为：

$$time = \frac{2^{15+top}}{wdt_{clock}}$$

Top(固定档位)	counter	实际时间 (秒)
0	65536	0.036408889
1	131072	0.072817778
2	262144	0.145635556
3	524288	0.291271111
4	1048576	0.582542222
5	2097152	1.165084444
6	4194304	2.330168889
7	8388608	4.660337778
8	16777216	9.320675556
9	33554432	18.64135111
10	67108864	37.28270222
11	134217728	74.56540444
12	268435456	149.1308089
13	536870912	298.2616178
14	1073741824	596.5232356
15	2147483648	1193.046471

表格 3: WDT 输入时钟 1800000 Hz

如上表所示，WDT 的硬件 counter 只有这 16 种选择，这个 counter 从 WDT 启动开始会按照输入时钟的频率逐渐递减到 0。如果在 counter 为 0 之前有了踢狗动作，则 counter 重新开始计数，如果 counter 递减到 0 了，那么 WDT 就会触发芯片重启。

9.1.2 软件接口

接口介绍

RTT 的软件接口上主要通过 RTT 自带的设备通用层实现，wdt 驱动本身的接口也都是通过 rt_device_control 实现的，有如下几个 ioctl 命令：

```
case RT_DEVICE_CTRL_WDT_START:
case RT_DEVICE_CTRL_WDT_KEEPALIVE:
case RT_DEVICE_CTRL_WDT_SET_TIMEOUT:
case RT_DEVICE_CTRL_WDT_GET_TIMEOUT:
case RT_DEVICE_CTRL_WDT_GET_TIMELEFT:
```

- RT_DEVICE_CTRL_WDT_START: 用于启动 WDT
- RT_DEVICE_CTRL_WDT_KEEPALIVE: 用于踢狗
- RT_DEVICE_CTRL_WDT_SET_TIMEOUT: 用于设置超时时间，参数单位为秒
- RT_DEVICE_CTRL_WDT_GET_TIMEOUT: 获取当前超时时间
- RT_DEVICE_CTRL_WDT_GET_TIMELEFT: 离 WDT 重启还有多久，单位为秒

接口调用

RTT 下首先打开 WDT 设备：

```
int wdt_fd;

wdt_fd = open("/dev/fh_wdt0", O_RDWR);
```

然后设置超时时间：

```
ioctl(wdt_fd, RT_DEVICE_CTRL_WDT_SET_TIMEOUT, &timeout);
```

最后起一个线程定时踢狗：

```
int i = 0;
for (; i < kick_times; i++)
{
    sleep(timeout / 2);
    ioctl(wdt_fd, RT_DEVICE_CTRL_WDT_KEEPALIVE, &timeout);
}
```

 版权所有 @ 上海富瀚微电子股份有限公司

9.1.3 注意事项

由于硬件只设定了 16 个档位，所以软件上会将配入的超时秒数根据当前 WDT 的输入时钟转换成硬件 counter，比如用户配置了 10s，软件上会先算出 counter = 10 * 18000000 = 18000000，然后查表寻找比这个 counter 更大的那一档，根据上面提供的表格来看，只能选择第 9 档，counter 为 33554432，那么实际 WDT 超时时间为 18.64s。

9.2 PWM 使用指南

9.2.1 概述

对于 FH8626V100/FH885XV200 提供 1 组 12 路独立的脉宽调制信号输出。

9.2.2 特点

输入时钟 50M，对于每组 PWM 输出新增功能如下：

- 占空比 duty 可配置
- 初始相位可控
  - 零时刻是对应通道的 enable 信号上升沿时刻
- 通道输出延时可控
- 两种输出模式:
  A) 输出有限个脉冲
  B) 输出无限个脉冲
- 通道输出关闭时结束时刻可控
  - 两种结束时刻：
    A) 立即结束
    B) 等待当前脉冲输出结束后停止
- 通道输出关闭时三种通道电平结束模式
  - 当前通道的电平状态有 3 种结束模式：
    A) 0 电平状态；
    B) 1 电平状态；
    C) 保持当前电平状态
- shadow 机制
  - 在通道输出有限个脉冲情况下，当 shadow 使能时，新的配置在该有限个脉冲输出结束后才生效

 版权所有 @ 上海富瀚微电子股份有限公司

- APB Slave
  - APB 时钟 pclk=120MHz
- 输入频率：pwm_clk=50MHz
- 输出频率：2Hz~500KHz 可配置

9.2.3 工作方式

PWM（以 PWM0 为例）输出配置流程如下：

1. 选择合适的时钟源，通过计算得到需要的周期数和高电平拍数。
2. 将对应数据写入 PWM0_CTRL、PWM0_CFG0~4 寄存器。
3. 对 GLOBAL_CTRL0 和 GLOBAL_CTRL2 bit[0] 写入 1，使能 PWM 输出。

例如：需要输出 1 个频率为 50kHz，高电平占 72.5%（即占空比），脉冲个数为 10，停止时电平为低且延迟一个周期输出的波形。
选择 50MHz 时钟为时钟源，周期数配置为 50MHz /50kHz=1000，四舍五入后为 1000，十六进制为 0x00003E8。高电平数配置为 1000（周期数）x 72.5%（占空比）=725，四舍五入后为 725，十六进制为 0x00002D5。
按如下步骤进行寄存器操作，即可输出所需要的波形：

1. 读取 PWM0_STATE2 bit[4]，等待 bit[10] 为 0（表示 PWM 空闲，可以输出方波）
2. 向 PWM0_CFG0 写入 0x0000_03E8。
3. 向 PWM0_CFG1 写入 0x0000_02D5。
4. 向 PWM0_CFG2 写入 0x0000_0000。
5. 向 PWM0_CFG3 写入 0x0000_03E8。
6. 向 PWM0_CFG4 写入 0x0000_000A。
7. 向 GLOBAL_CTRL0 和 GLOBAL_CTRL2 bit[0] 写入 1。

（以下步骤可以省略，只是为了验证正在输出的方波是否按配置输出）

8. 读取 PWM0_STATE2 bit[4]，等待 bit10 位为 1（表示 PWM 正在输出方波）
9. 读取 PWM0_STATE0 bit[31:0] 和 0x0A 进行校验 (当 PWM0_STATE2 bit[4] 为 1 时表示 PWM 正在输出方波，当该位为 0 时表示已经输出完设定的方波数目)。

9.2.4 软件接口

软件接口上主要通过 OS 自带的设备通用层实现，软件上通过 ioctl 函数，填充 fh_pwm_config 结构体实现对 PWM 的配置：

 版权所有 @ 上海富瀚微电子股份有限公司

```
struct fh_pwm_config
{
    unsigned int period_ns;
    unsigned int duty_ns;
#define FH_PWM_PULSE_LIMIT         (0x0)
#define FH_PWM_PULSE_NOLIMIT       (0x1)
    unsigned int pulses;
    unsigned int pulse_num;
#define FH_PWM_STOPLVL_LOW          (0x0)
#define FH_PWM_STOPLVL_HIGH         (0x3)
#define FH_PWM_STOPLVL_KEEP         (0x1)
    unsigned int stop_module;
#define FH_PWM_STOPCTRL_INS        (0x1)
#define FH_PWM_STOPCTRL_DELAY      (0x0)
    unsigned int stop_ctrl;
    unsigned int delay_ns;
    unsigned int phase_ns;
    unsigned int percent;
    unsigned int shadow_enable;
    unsigned int finish_once;
    unsigned int finish_all;
};
```

- period_ns: pwm 周期数
- duty_ns: 高电平周期数
- pulse: 输出有限波还是无线波
- pulse_num: 输出方波数
- stop_module : 停止后电平高低，0 表示低，3 表示高
- delay_ns: pwm 使能后延迟多久后再输出
- phase_ns: 初始相位拍数
- percent: 占空百分比
- finish_once: 完成一个周期后的是否产生中断，1 表示产生中断，0 表示不产生中断
- finish_all: 完成全部方波输出后的是否产生中断，1 表示产生中断，0 表示不产生中断

软件接口上主要通过设备通用层实现，PWM 驱动本身也提供 ioctl 接口，ioctl 命令如下：

- ENABLE_PWM
  - 传入的是 fh_pwm_config 结构体，内部包含了所有的设置，配置完之后再使能 pwm
- DISABLE_PWM

  - 关闭 PWM，传入 fh_pwm_config 结构体
- SET_PWM_DUTY_CYCLE
  - 按 Counter 设置 PWM 占空比，传入 fh_pwm_config 结构体
- GET_PWM_DUTY_CYCLE
  - 按 Counter 获取当前占空比，传入 fh_pwm_config 结构体
- SET_PWM_DUTY_CYCLE_PERCENT
  - 按百分比设置当前占空比，传入 fh_pwm_config 结构体
- ENABLE_MUL_PWM
  - 使能多个 PWM，传入 pwm id 的 mask 地址

9.2.5 接口调用

RTT 下首先找到并打开 PWM 设备：

```
pwm_fd = open("/dev/pwm", O_RDWR | O_SYNC);
```

然后设置超时时间：

```
ioctl(pwm_fd, SET_PWM_DUTY_CYCLE, &pwm_config);
int mask = 0x1;
ioctl(pwm_fd, ENABLE_MUL_PWM, &mask);
```

完成输出后关闭 PWM：

```
ioctl(pwm_fd, DISABLE_PWM, &pwm_config);

close(pwm_fd);
```

 版权所有 @ 上海富瀚微电子股份有限公司

10 WiFi 使用指南

10.1 wifi 的编译

10.1.1 sdio wifi 的编译使能

sdio wifi 公共配置

1. 使能 SDIO Wifi

```
Drivers --->
WiFi Configuration--->
   [*] Enable WiFi
        Choose USBWIFI or SDIOWIFI(Enable SDIO Wifi)--->
```

2. sdio 控制器 WIFI_SDIO (the SDIO used by wifi) 和 USING 1BIT MODE(参考 sdio 控制器 bit width) 选择使能

```
Drivers --->
WiFi Configuration--->
   [*] Enable WiFi
         Choose USBWIFI or SDIOWIFI(Enable SDIO Wifi)--->
   (0)the SDIO used by wifi
   [ ]   USING 1BIT MODE
```

SDIO WiFi 型号选择

以sdio wifi 公共配置为前提，以下 WiFi 型号，基于sdio wifi 公共配置 menuconfig 页面进行选择

- Marvell 8801

  WiFi Configuration 中选择 Enable MARVEL

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable SDIO Wifi)--->
(0)   the SDIO used by wifi
```

```
[ ]   USING 1BIT MODE
      Choose SDIOWifi(Enable MARVEL)--->
```

- Cypress 43438

  WiFi Configuration 中使能 Enable CYPRESS 并选择 Enable CY43438

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI (Enable SDIO Wifi)--->
(0)   the SDIO used by wifi
[ ]   USING 1BIT MODE
      Choose SDIOWifi(Enable CYPRESS)--->
      Choose Cypress Wifi(Enable CY43438)--->
```

> 注：Cypress 43438 支持`CY43438_LOWPOWER`和`CY43438_NORMAL`两种模式，请根据需求在`Choose CY43438 mode`中进一步选择

- Cypress 43455

  WiFi Configuration 中使能`Enable CYPRESS`并选择`Enable CY43455`

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI (Enable SDIO Wifi)--->
(0)   the SDIO used by wifi
[ ]   USING 1BIT MODE
      Choose SDIOWifi(Enable CYPRESS)--->
      Choose Cypress Wifi(Enable CY43455)--->
```

- Realtek 8192FS

  WiFi Configuration 中使能`Enable RTL8192FS`

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable SDIO Wifi)--->
(0)   the SDIO used by wifi
[ ]   USING 1BIT MODE
      Choose SDIOWifi(Enable RTL8192FS)--->
```

- HiSilicon Hi3861L

版权所有 @ 上海富瀚微电子股份有限公司

WiFi Configuration 中使能Enable HI3861L和USING 1BIT MODE，Hi3861L 只支持 1BIT 模式。另外，Hi3861L 模块的 SDIO 中断使用 GPIO 中断，请务必检查 GPIO 中断使用的 GPIO 引脚，并正确配置 pinshare。

Hi3861L menuconfig 配置：

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable SDIO Wifi)--->
(0)   the SDIO used by wifi
[*]   USING 1BIT MODE
      Choose SDIOWifi(Enable HI3861L)--->
```

中断及 SDIO 的 pinshare 配置：

```
diff --git a/platform/fh8652/app_board/appboard_iopad.h b/platform/fh8652/app_board/appboard_iopad.h
index 69358db..386f261 100644
--- a/platform/fh8652/app_board/appboard_iopad.h
+++ b/platform/fh8652/app_board/appboard_iopad.h
@@ -30,7 +30,7 @@ char *fh_pinctrl_selected_devices[] = {
 #if (WIFI_SDIO == 0)
 "SD0_WIFI", "GPIO17"/* HW_WIFI_POWER_GPIO */, "GPIO28"/* WIFI_ENABLE_GPIO */,"UART1", "PWM2", "PWM3", "PWM4", "PWM5", "GPIO47"
 #elif (WIFI_SDIO == 1)
-"SD0_NO_WP", "SD1_WIFI", "GPIO40"/* HW_WIFI_POWER_GPIO */, "GPIO28"/* WIFI_ENABLE_GPIO */
+"SD0_NO_WP", "SD1_WIFI_1BIT", "GPIO40"/* HW_WIFI_POWER_GPIO */, "GPIO28"/* WIFI_ENABLE_GPIO */
 #endif
                                 这里GPIO40已经引出，且没用和其他GPIO复用冲突，所以FH8652平台可以直接使用GPIO40作为HI3861L的GPIO中断接口。其他平台类似。
 #else
 "SD0_NO_WP", "ETH","UART1", "PWM2", "PWM3", "PWM4", "PWM5", "GPIO47"
diff --git a/platform/fh8652/fh8652_iopad.h b/platform/fh8652/fh8652_iopad.h
index 9487793..f0b08b3 100644
--- a/platform/fh8652/fh8652_iopad.h
+++ b/platform/fh8652/fh8652_iopad.h
@@ -558,6 +558,7 @@ PINCTRL_DEVICE(SD1_NO_WP, 7, &MUX_SD1_CD, &MUX_SD1_CLK, &MUX_SD1_CMD_RSP,
         &MUX_SD1_DATA_0, &MUX_SD1_DATA_1, &MUX_SD1_DATA_2, &MUX_SD1_DATA_3);
 PINCTRL_DEVICE(SD1_WIFI, 6, &MUX_SD1_CLK, &MUX_SD1_CMD_RSP, &MUX_SD1_DATA_0,
         &MUX_SD1_DATA_1, &MUX_SD1_DATA_2, &MUX_SD1_DATA_3);
+PINCTRL_DEVICE(SD1_WIFI_1BIT, 3, &MUX_SD1_CLK, &MUX_SD1_CMD_RSP, &MUX_SD1_DATA_0);
 PINCTRL_DEVICE(SENSOR_CLK, 1, &MUX_SENSOR_CLK);
 PINCTRL_DEVICE(SSI0, 4, &MUX_GPIO6, &MUX_SSI0_CLK, &MUX_SSI0_RXD,
         &MUX_SSI0_TXD);
@@ -683,6 +684,7 @@ void fh_pinctrl_init_devicelist(OS_LIST *list)
     PINCTRL_ADD_DEVICE(SD1_1BIT_NO_WP);
     PINCTRL_ADD_DEVICE(SD1_NO_WP);
     PINCTRL_ADD_DEVICE(SD1_WIFI);
+    PINCTRL_ADD_DEVICE(SD1_WIFI_1BIT);
     PINCTRL_ADD_DEVICE(SENSOR_CLK);
     PINCTRL_ADD_DEVICE(SSI0);
     PINCTRL_ADD_DEVICE(SSI0_4BIT);
```

图片 9: Hi3861L SDIO 和 GPIO 配置

Hi3861L 需要在应用中对 hichannel 进行初始化，初始化流程：

```
extern int w_sdio_config(int sdio_id, int bit_width, int clock,
    ↪ int gpio_num_irq);
extern int hichannel_heartbeat_init(int tx_interval_s, int
    ↪ timeout_s);
(void)w_sdio_config(x, 1, 50000000, y);
hichannel_heartbeat_init(5, 10);
```

上述代码中，w_sdio_config 函数第一个参数为 sdio_id，请根据实际使用的 sdio id 填写，第四个参数 gpio_num_irq 为 wifi sdio_int 引脚需要连接的 GPIO 接口，根

 版权所有 @ 上海富瀚微电子股份有限公司

据自己的配置进行填写。

为保证 sdio 的正确枚举，hi3861L 平台必须比 fullhan 平台提前上电，一般由 hi3861L 平台控制 fullhan 平台 power。RTos 系统 hi3861L 驱动和 Hi3861L 测代码需要配合使用，如需自定义功能请找 fullhan 提供源码。

10.1.2 usb wifi 的编译使能

usb wifi 公共配置

1. usb host mode 使能

```
Drivers --->
Usb Configuration--->
   [*] Enable usb driver
         Choose usb mode(Enable usb host mode core)--->
   -*-  usb host controller enable
```

2. usb wifi 使能

```
Drivers --->
WiFi Configuration--->
   [*] Enable WiFi
         Choose USBWIFI or SDIOWIFI(Enable USB Wifi)--->
```

USB WiFi 型号选择

以usb wifi 公共配置为前提，以下 WiFi 型号，基于usb wifi 使能 menuconfig 页面进行选择

- Realtek 8188FTV

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable USB Wifi)--->
      Choose USBWifi(Enable RTL8188FTV)--->
```

- Realtek 8192FC

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable USB Wifi)--->
```

```
      Choose USBWifi(Enable RTL8192FC)--->
```

- Realtek 8192EU

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable USB Wifi)--->
      Choose USBWifi(Enable RTL8192EU)--->
```

- Realtek 8731BU

```
[*] Enable WiFi
      Choose USBWIFI or SDIOWIFI(Enable USB Wifi)--->
      Choose USBWifi(Enable RTL8731BU)--->
```

10.2 wifi 的运行

10.2.1 sdio wifi 与开发板连接

若平台 SDIO 与其他外设有引脚复用，需先确保硬件不冲突（如 SDIO 4bit 与以太网可能存在复用）

FH8626V100

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 GPIO3 引脚

FH8852V200/FH8856V200

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 GPIO2 引脚

FH8858V200

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 GPIO15 引脚

 版权所有 @ 上海富瀚微电子股份有限公司

FH8652/FH8656

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 GPIO28 引脚

FH8658

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 GPIO15 引脚

其他平台

将 Marvell 8801 或 Cypress 43438/43455 或 Realtek 8192FS WIFI 板子的 WL_REG_ON PIN 连接至平台板的 3.3V VCC 引脚，上述所有 SDIO 模块 wifi 将 WL_REG_ON PIN 连接至平台板的 GPIO 引脚的目的都是为了 wifi 模块的复位操作提供便利，如果没有 fullhan 平台板或单纯为了测试 wifi 功能，则直接将接至 VCC 引脚使 wifi 模块使能即可。此操作对上述各型号平台同样适用。

特殊模块

Hi3861L 与开发板连接

1. 首先将 Hi3861L 的 sdio wifi 小板的 wifi_vcc 引脚连接到开发板上的 VCC3.3 接口，将 GND 引脚连接至开发板上的 GND 接口。
2. 将 SDIO_INT 引脚连接至开发板上的 GPIO 中断引脚，这里根据 pinshare 配置进行连接。

10.2.2 运行示例 ——station 模式

1. 加载 wifi 以启用 station 模式连接特定 AP:

```
wifi --conn_ap --ssid x (--passwd x)
```

2. 运行 ifconfig 查看是否生成无线网口
3. 设置 IP 地址 (下述两种方式选择其一)：

   1）执行 dhcpc start w0 从 DHCP server 获取 IP（需确保环境中存在 DHCP server）

   2）运行 ifconfig w0 xx 以设置无线网口的 IP（确保与所连路由器在同一网段）

 版权所有 @ 上海富瀚微电子股份有限公司

> Hi3861L 模块比较特殊，一般使用'dhcp_3861 0 0' 命令从 3861L 端获取 ip 地址。特殊情况下，如将 dhcp 端口 67、68 端口转发到 FH 平台，则可以使用 dhcpc start w0 从 DHCP server 获取 IP, 但是由于低功耗状态下需要保护，我们不建议这么做。

4. 运行 ifconfig 确认无线网口的网络参数
5. ping 无线路由器的 IP 地址以检查网络连通性

10.2.3 运行示例 ——AP 模式

1. 启用 AP 模式生成指定信道和 ssid 的无线热点:

```
wifi --work_ap --ssid x --chan_id x (--passwd x)
```

2. 设置无线热点的 IP 地址:

   1）运行 ifconfig ap xx 以设置无线网口的 IP 地址

   2）运行 ifconfig 确认无线网口的网络参数

> Hi3861L 模块比较特殊，一般使用'dhcp_3861 1 0' 命令从 3861L 端获取 ip 地址。特殊情况下，如将 dhcp 端口 67、68 端口转发到 FH 平台，则可以使用 dhcpc start w0 从 DHCP server 获取 IP, 但是由于低功耗状态下需要保活，我们不建议这么做。

3. 启动 DHCP server 向连接该无线热点的无线终端分配 IP 地址：

```
dhcpd start ap
```

> 注：若该命令不存在，则需要编译前 make menuconfig 使能`Enable DHCP server`并设置`DHCPD_SERVER_IP`

4. 将无线终端连接到新创建的无线热点

10.2.4 station 与 ap 模式并发运行

该功能目前 Realtek 8188FTV、8192FC、8192EU、8731BU 支持。

1. 启用 station 模式连接特定 ap（命令见前述章节）
2. 启用 ap 模式生成指定信道和 ssid 的无线热点（命令见前述章节）

10.2.5 wifi 网口的桥接

wifi 网口桥接时不会操作 wifi，wifi start/stop 与 bridge 是独立操作的。

wifi AP 和 station 桥接做无线中继

1. 使能 AP 和 station 桥接

   1）测试命令：

   添加 bridge 接口

```
br_wifi --addbr <bridge>
```

   例如：

```
br_wifi --addbr br
```

   将 wifi ap 和 sta 网口加入 bridge

```
br_wifi --br <bridge> --addif <netif>
```

   例如：

```
br_wifi --br br --addif ap
br_wifi --br br --addif w0
```

   将 wifi ap 和 sta 网口从 bridge 中删除

```
br_wifi --br <bridge> --delif <netif>
```

   例如：

```
br_wifi --br br --delif ap
br_wifi --br br --delif w0
```

   2）函数实现：

   添加 bridge 接口

```
br_netif_init(char *br_name);
```

 版权所有 @ 上海富瀚微电子股份有限公司

将相关网口加入 bridge

```
br_add_if(char *br_name, char *name)
```

将相关网口从 bridge 中删除

```
br_del_if(char *br_name, char *name)
```

注：进行 ap_sta 桥接时须保证已经生成 ap 和 w0 接口。bridge 删除接口只是将某个网口从 bridge 中删除，如果不想使用桥接，请使用撤销桥接命令。命令中对应的 ``bridge" 为 bridge 网口名，``netif" 为需要添加或删除的网口名。

2. 撤销桥接

   1）测试命令：

```
br_wifi --delbr <bridge>
```

   例如：

```
br_wifi --delbr br
```

   2）函数实现：

```
br_netif_deinit(char *br_name)
```

注：deinit 时不会 stop wifi，如若想操作 wifi，需要使用 wifi 命令

wifi AP 和以太网桥接

1. 使能 AP 和以太网桥接

   ap 和以太网桥接与 ap 和 sta 桥接类似，只需将对应的网口名替换成 ap 和 e0 即可。

wifi 桥接其它注意事项

1. 如若以太网和 wifi sta 使用的网口名称不是 e0，w0，需要注意，在 addif 之后需要调用接口设置 br 的 mac 地址。

   1）测试命令

 版权所有 @ 上海富瀚微电子股份有限公司

```
br_wifi --hwaddr <xx:xx:xx:xx:xx:xx>
```

例如：

```
br_wifi --hwaddr 10:20:30:40:50:60
```

2）函数实现：

```
br_set_hwaddr(char *hwaddr)
```

注：桥接设置 mac 地址时需根据桥接方式将以太网或 wifi sta 对应网口 mac 地址设置至 bridge 网口中。

10.2.6 usb wifi 和 sdio wifi API 说明

w_sdio_config(int sdio_id, int bit_width, int clock, int gpio_num_irq)

wifi sdio 预先初始化，其中各入参说明如下:

- sdio_id: sdio 控制器 index.(0:sdio0, 1:sdio1, 2:sdio2, ...)
- bit_width: sdio bit width (1:using 1bit mode, 4:using 4bit mode)
- clock: sdio clock frequency, 可设置为 25000000 或 50000000
- gpio_num_irq: wifi 接收报文的中断机制 (-1: to use SDIO interrupt, >=0: the GPIO pin index)

注：

- 各 sdio wifi 均需首先调用该接口
- 目前 gpio_num_irq 使能 gpio 中断接收报文仅 Cypress 43438/43455 支持

w_start(unsigned int mode, unsigned int sleep_flag)

以特定的状态和工作模式加载 wifi

- mode:
  - 0 表示以 station 模式加载 wifi
  - 1 表示以 AP 模式加载 wifi
- sleep_flag：
  - 0 表示以普通状态工作
  - 1 表示以低功耗状态工作 (当前仅 Cypress 43438 station 可选低功耗状态)

w_stop(unsigned int mode)

卸载特定工作模式下的 wifi

- mode：
  - 0 表示卸载 station 模式的 wifi
  - 1 表示卸载 AP 模式的 wifi

w_ap_on(char *ssid, char* passwd, unsigned int chan_id)

启用 wifi 以作为 AP

- ssid：将要创建 AP 的 ssid
- passwd: 将要创建 AP 的密钥
- chan_id: 将要创建 AP 的工作信道 (目前仅 43455 支持 5G 信道)

w_sta_scan_ext(FH_UINT32 max_ap_num, FH_CHAR *ssid, FH_UINT32 scan_channel_down, FH_UINT32 scan_channel_up)

扫描周边热点, 并且返回扫描到的 AP 列表

- max_ap_num: 期望返回的扫描到的 AP 的最大数目, 0 表示没有限制
- ssid: 带 ssid 的扫描, 一般情况下填 NULL 即可
- scan_channel_down: 指定扫描信道的起始信道
- scan_channel_up: 指定扫描信道的结束信道

比如: scan_channel_down=4, scan_channel_up=6, 表示在 [4,5,6] 三个信道扫描
比如: scan_channel_down=0, scan_channel_up=0, 表示全信道扫描

w_sta_free_scaninfo(FH_WiFi_AccessPoint_List *aplist)

w_sta_free_scaninfo() 需在后 w_sta_scan_ext 调用以释放 w_sta_scan_ext 中动态分配的内存

注:

- 再次执行 w_sta_scan_ext 前必须调用 w_sta_free_scaninfo 来释放此前分配的内存
- 设置 scan_channel_down 与 scan_channel_up 指定信道范围扫描功能尚未实现

 版权所有 @ 上海富瀚微电子股份有限公司

- w_sta_scan 是 w_sta_scan_ext 的调试版本, 该函数触发 station 扫描周围的 AP 并打印扫描结果

w_sta_status(void)

station 返回与 AP 的连接状态信息，返回值为 0 表示已建立连接

w_list_cfg_ap(void)

显示 station 模式时已配置的 AP 列表；若已建立连接，则带"[CURRENT]" 标识的是已连接的 AP

w_sta_cfg_ap(unsigned int id, unsigned int add)

station 模式时增加或删除特定编号的 AP 配置

- id: 要操作的 AP 配置编号，取值范围为 [0, 2]
- add: 0 表示删除该组 AP 配置，1 表示增加该组 AP 配置

w_sta_cfg_ssid(unsigned int id, char *ssid)

station 模式时配置特定编号 AP 的 ssid

- id: 要操作的 AP 编号，取值范围为 [0, 2]
- ssid: 要配置 AP 的 ssid

w_sta_cfg_psk(unsigned int id, char *passwd)

station 模式时配置特定编号 AP 的密钥

- id: 要操作的 AP 编号，取值范围为 [0, 2]
- passwd: 要配置 AP 的密钥

w_sta_sel_ap(unsigned int id)

station 模式时连接指定编号的 AP，失败时会周期性重连，直至连接成功

- id：要连接的 AP 编号

w_sta_rm_ap(unsigned int id)

station 模式时断开与指定编号 AP 的连接并删除该组 AP 配置

- id：要断开连接的 AP 编号

w_sta_disconn(unsigned int id)

station 模式时断开与指定编号 AP 的连接

- id：要断开连接的 AP 编号

> 注：
> **station 和 AP 模式启用的 API 调用示例请分别参考 wifi_conn_ap() 和 wifi_work_ap()**

 版权所有 @ 上海富瀚微电子股份有限公司

11 USB 使用指南

11.1 USB-4G 驱动使用指南

11.1.1 4G 驱动说明

4G 驱动目前支持的拨号方式包括 PPPos 驱动、ECM 和 RNDIS 三种驱动，PPPos 驱动通过 USB 串口与模块进行网络通讯，ECM/RNDIS 通过建立 USB 网卡使用专用数据接口与 4G 模块进行网络通讯。

由于不同厂家的 4G 模块封装的 AT 命令和 AT 接口都稍有差异。因此，每个使用 PPPos 拨号上网的 4G 模块在首次接入 RTos 系统前都应进行测试，并对 PPPos 的拨号的 dial_commands 数组进行修改，以保障 PPPos 拨号成功。目前 PPPos 已经适配的模块有：移远 EC20/EC200、诺控 NL668、龙尚 U9300、合宙 Air720、有人 WH-G405tf、创建 CJ907M、有方 N720V5。PPPos 拨号 demo 在 **SDK/rt-thread/utils/mobile_tele/tele_ppp_demo.c** 文件中。

ECM 和 RNDIS 驱动对于使用 ECM/RNDIS 协议的 USB 设备均可识别并生成 USB 网卡，因此所有 ECM/RNDIS 协议的 4G 模块都可以直接使用 RTos 系统的 ECM/RNDIS 驱动，并进行拨号上网。值得注意的是不同厂家 4G 模块的 ECM/RNDIS 拨号命令存在差异，需要针对具体厂家/模块进行修改。ECM/RNDIS 只需要了解并修改拨号命令即可，目前 demo 里面列举了移远 EC200、合宙 Air720、有人 WH-G405tf、创景 CJ907M、有方 N720V5 这几种模块的拨号方法。如果模块支持自动拨号则无需发送拨号命令 (如 Air720)，则拨号命令可以用`AT\r`代替。RNDIS 拨号 demo 在 **rt-thread/utils/mobile_tele/tele_rndis_demo.c** 文件中。

11.1.2 4G 驱动使用

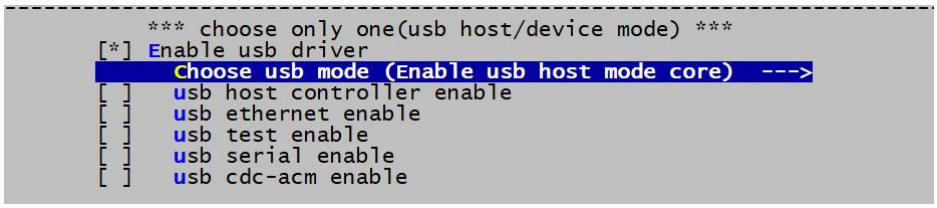
使用 4G 驱动前，首先需要确定使用 PPPos 方式拨号上网还是 RNDIS 方式拨号上网，然后确定模块型号。假设模块型号在 demo 中已经进行了适配。操作方法如下：

1. 进入 sdk rt-thread 目录，执行 make menuconfig；
2. 进入 make menuconfig，选择 Enable usb host mode core，位置如下：

   Location:

```
->Drivers
   -> Usb configuration
    -> Enable usb driver (RT_USING_USB [=y])
```

```
-> Choose usb mode (<choice> [=y])
```



图片 10: 开启 usb host mode

3. 打开 4G 模块驱动，并选择拨号方式：

   Location:

```
-> components packages
-> Enable MOBILE_TELE (RT_USING_MOBILE_TELE [=y])
 -> Choose dail method (<choice> [=y])
```

图片 11: 选择拨号方式

4. 对于 PPPos 拨号：Choose dail method 选择 MOBILE_TELE via pppos;
5. 对于 ECM/RNDIS 拨号：Choose dail method 选择 MOBILE_TELE via ecm/rndis；

   对于 ecm/rndis 方式拨号的模块，无论是否之前已经适配，都需要修改 sdk/rt-thread/ utils/mobile_tele/tele_rndis_ecm_demo.c 中的 dail_cmd1 拨号指令为对应模块所需要的值，文件注释中已经提供了适配模块的拨号指令。

6. menuconfig 中 select app demo 可选择任意 demo
7. 退出 menuconfig，使用 make app 进行编译；
8. 系统运行后使用 tele_dial 指令拨号，如果拨号失败可运行 tele_exit 退出拨号后再次执行 tele_dial 重新拨号，也可以使用 tele_send_PT 指令手动拨号；
9. 拨号成功后使用 dhcpc start u0 指令获取 ip 地址。

11.1.3 新增 4G 模块

对于 demo 中尚未适配的 4G 模块，如果需要接入 fullhan 的 RTos 平台中，可按照如下方法进行操作。

新增 PPPos 拨号对 4G 模块的支持

一般来说，4G 模块都支持 PPPos 拨号。

1. 查询模块的 PID 和 VID 码（可通过向厂家咨询，查看产品手册、USB 抓包工具或 PC 平台获取），将 PID 和 VID 码添加到 **SDK/rt-thread/driver/usb/host/serial/usb_serial.c** 文件的`usb_serial_usb_id`结构体中:

```
static struct usb_device_id usb_serial_usb_ids[] = {
    {USB_DEVICE(0x1286, 0x4e3d), .driver_info = SERIAL_NUM_INFO(2, 3),},/* LUAT AIR720H */
    {USB_DEVICE(0x1508, 0x1001), .driver_info = SERIAL_NUM_INFO(3, 1),},/* FIBOCOM NL668 */
    {USB_DEVICE(0x05c6, 0x9003), .driver_info = SERIAL_NUM_INFO(2, 3),},/* Quectel UC20 */
    {USB_DEVICE(0x05c6, 0x9215), .driver_info = SERIAL_NUM_INFO(2, 3),},/* Quectel EC20 */
    {USB_DEVICE(0x2c7c, 0x0125), .driver_info = SERIAL_NUM_INFO(2, 3),},/* Quectel EC25 */
    {USB_DEVICE(0x2c7c, 0x0121), .driver_info = SERIAL_NUM_INFO(2, 3),},/* Quectel EC21 */
    {USB_DEVICE(0x2c7c, 0x6026), .driver_info = SERIAL_NUM_INFO(4, 3),},/* Quectel EC200 */
    {USB_DEVICE(0x1c9e, 0x9b3c), .driver_info = SERIAL_NUM_INFO(2, 1),},/* Longsung U9300C */
    {USB_DEVICE(0x05c6, 0x90b6), .driver_info = SERIAL_NUM_INFO(SERIAL_NUM_NONE, 2),},/* chuangjing CJ907 */
    {USB_DEVICE_AND_INTERFACE_INFO(0x19d2, 0x0579, 0xff, 0xff, 0xff), SERIAL_NUM_INFO(2, 3),},
    { }
};
```

图片 12: 添加 PID 和 VID

2. 上图中设备的 `driver_info` 为模块支持 AT 命令接口的 usb interface 接口号，可逐个测试或向厂家咨询，`SERIAL_NUM_INFO`中第一个参数用于常规 AT 命令查询，第二个参数用于 PPPos 拨号和 PPPos 数据通道。一般模块最少有一个 AT 命令接口，只有一个 AT 接口的应参考 CJ907 模块，写在第二个参数中；

> **NOTE：如果可以确定模块的 AT 命令接口，填写到 SERIAL_NUM_INFO 中，则以下（3~4）步骤可忽略。**

3. 用4G 驱动使用中的方法，配置 menuconfig，demo 中适当位置调用 fh_tele_at_debug（1）函数打开 at debug，然后编译运行

> 注：确认模块使用 serial 驱动还是 cdc-acm 驱动，如果为 cdc-acm 驱动，则 (1~2) 修改应在 **SDK/rt-thread/driver/usb/host/serial/cdc-acm.c** 文件中）

4. 编译并运行程序后，可以使用 fh_tele_at_send 和 fh_tele_at_send_oob 函数发送`AT\r`分别确认两个 AT 接口是否可用（收到 ok 回复表示可用），如果不可用则继续修改串口接口号直到两个都可以或至少有一个可用。其中 fh_tele_at_send 使用 PT 接口，fh_tele_at_send_oob 使用 AT 接口（均为 ATcommand 接口，这里 AT/PT 仅为区分用途，PT 用于 PPPos 拨号和数据通路）

 版权所有 @ 上海富瀚微电子股份有限公司

5. 根据 4. 中确定的接口号重新修改 SERIAL_NUM_INFO 参数，然后再修改 **SDK/rt-thread/utils/mobile_tele/tele_ppp_demo.c** 中 dial_commands 数组，使拨号命令适配新的模块，修改思路：先尝试原数组不变是否可以拨号正常使用，正常使用则无需修改，如果不能完成拨号，查看报错的 AT 命令并进行修改。也可根据需求自定义增加/删减；

6. 编译运行即可，使用方法与4G 驱动使用中步骤相同。

新增 ECM/RNDIS 拨号对 4G 模块的支持

1. 首先应确定 4G 模块是否支持 ECM/RNDIS 驱动，可向厂家咨询
2. 如果该模块没有适配 PPPos 拨号，应按照本节新增 PPPos 拨号对 4G 模块的支持中的方法在usb_serial.c/cdc-acm.c中添加产品 PID、VID 和串口接口号（因为 RNDIS 发送拨号命令需要串口的支持）
3. 确定 4G 模块 ECM/RNDIS 拨号命令，在下面文件中修改 ecm/rndis 拨号命令 SDK/rt-thread/utils/mobile_tele/tele_rndis_ecm_demo.c
4. menuconfig 中配置为 rndis 拨号，编译运行即可，配置方法与4G 驱动使用相同

11.1.4 其他说明

1. 用户可根据自己需要进行修改，也可参考这里的使用方法自行构建拨号 demo。
   - ppp 拨号 demo: SDK/rt-thread/utils/mobile_tele/tele_rndis_ecm_demo.c
   - rndis 拨号 demo: SDK/rt-thread/utils/mobile_tele/tele_ppp_demo.c
2. demo 中提供有模块复位函数，但并未实现复位方法，这里建议模块 reset 用 gpio 实现，确保复位成功。
3. RTos 系统实现了两种协议的串口，usb-serial 串口的 usb 协议码为（0xff，0xff，0xff）/（0xff，0x00，0x00）。Cdc-acm 串口是 cdc 类协议，协议码为 0x02，0x02，0x06，已经适配的模块中 Neoway N720 模块使用的 cdc-acm 串口驱动，其他模块用的都是 usb-serial 串口驱动。
4. fh_tele_at_send 和 fh_tele_at_send_oob 两个 API 可用于发送单个 AT 命令到 4G 模块，两个函 API 仅所用接口不同。fh_tele_at_send 用 PT 接口，fh_tele_at_send_oob 用 AT 接口，如果使用 PPPos 拨号，在 PPPos 建立链接后，PT 口被占用，只能使用 fh_tele_at_send_oob 函数发送 AT。如果使用 ECM/RNDIS 拨号，则两个函数都可以用。用户可根据需求对两个 API 自行封装调用，代码中封装了 tele_send_cmd 函数作为参考。

 版权所有 @ 上海富瀚微电子股份有限公司

11.2 UVC&UAC 使用指南

11.2.1 目录说明

- common: dsp、isp 公共函数，如初始化 ISP、创建编码通道等
- usb_audio: uac mic&speaker 应用 demo 例程
- usb_video: uvc 应用 demo 例程
- usb_vcom: usb cdc-vcom 应用 demo 例程
- usb_hid: usb hid keyboard 应用 demo 例程

11.2.2 config 说明

kernal config

1. FH_RT_USB_DEVICE [=y] && FH_RT_USB_DEVICE_COMPOSITE [=y]；
2. FH_RT_USB_DEVICE_UVC [=y] 使能 uvc 驱动；
3. FH_RT_USB_DEVICE_UAC [=y] 使能 uac 驱动；
4. FH_RT_USB_DEVICE_CDC [=y] 使能 cdc-vcom 驱动；
5. vendor_label 和 device name 可以配置设备名称；
6. UVC_DOUBLE_STREAM[=y] or [=n], 是否使能双码流；
7. Choose UVC EP type, 选择 ISOC or BULK 方式作为 UVC 端点传输方式；
8. ISOC 模式下:UVC_PACKET_SIZE [=1024]isoc 端点大小,ISOC_HIGH_BANDWIDTH_EP [=y] 启用高带宽模式；
9. BULK 模式下:UVC_PACKET_SIZE 最大为 512,UVC_BULK_PAYLOAD_PACKET_NUM bulk 模式每个 payloay 传输 usb 包个数，即多少个 UVC_PACKET_SIZE;
10. ENABLE_SPEAKER_DESC [=y] or [=n], 是否使能 usb speaker；
11. FH_RT_USB_DEVICE_HID 和 FH_RT_USB_DEVICE_MSTORAGE 为 hid 和 u 盘功能，视需求自行配置；

app config

1. 配置 ISP 和 DSP 参数, 如果使能 UVC 双码流则 DSP 必须使用双通道；
2. FH_APP_USING_COOLVIEW [=y] or [=n], 是否使能 coolview；

   如果使能 COOLVIEW, 则选择 select coolview type: tcp or vcom, 可用使用 tcp 和 cdc 两种方式调试 coolview；
3. FH_APP_USING_HEX_PARA [=y] or [=n], 是否使用 flash 相关功能，包括：从 flash 获取 sensor 参数、获取 UVC 参数配置、获取 UVC 产品信息、在线升级功能、hex 参数导入导出功能和 image 导出功能等所有与 flash 有关的功能。

 版权所有 @ 上海富瀚微电子股份有限公司

FH_APP_USING_HEX_PARA [=y] 仅在 img 烧录进 flash 下可用, tftp 方式加载时不可用；

4. APP_USB_DEVICE_UVC [=y] 使能 UVC 应用；
5. NV12TOYUY2[=y] or [=n], y: 将 yuv 输出转换为 yuy2 格式，n: 为 NV12 格式；
6. UVC_DOUBLE_STREAM 双码流功能
7. APP_USB_DEVICE_UAC[=y] 使能 UAC 应用;
8. ENABLE_SPEAKER_DESC UAC speaker 功
9. APP_USB_DEVICE_CDC [=y] 使能 usb cdc 应用；
10. APP_USB_DEVICE_HID [=y] or [-n] 是否使能 usb hid 应用；

注意：该版本 RTOS 应用层与内核隔离，因此 APP 中 UVC 功能配置应和 kernel 中的配置保持一致。

11.2.3 驱动说明

Uvc 驱动支持的功能包括 uvc 单双码流，uvc isoc 端点模式和 bulk 端点模式，其中 isoc 模式下支持 isoc 高带宽模式，带宽能力是非高带宽能力的 3 倍，最高传输速率达到 24MB/S。视频格式方面支持 YUY2、NV12、MJPEG、H264、H265 四种格式，格式、分辨率和帧率可以在 demo 中进行选配。

Uac 驱动支持 uac 录音和扬声器两种功能，采样率分别都支持 8k、16k、22.05k、32k、44.1k 和 48k，uac 功能可以不依赖 uvc、vcom 单独编译使用。

Usb-cdc-vcom 驱动为 usb 虚拟串口，实现了用 usb 调节 coolview 功能和 shell 命令行功能，二者之间可通过 shell 窗口按 ESC 键实现切换。

另外，uvc 还实现了 img 固件的在线升级、sensor 效果参数文件 hex 文件的在线更新、img 固件导出、uvc 参数保存、uvc 产品信息修改及保存几个功能，需要注意：这几个功能都只能在 img 固件程序烧录到 flash 时使用，且必须借助 windows 端 exe 工具使用。Uvc 及 uvc 的扩展功能不依赖 uac 和 usb-cdc-vcom。此版本 UVC 升级可以通过模拟 U 盘、usb 虚拟串口、dfu 和 hid 的方式实现。

11.2.4 驱动配置及编译

配置

按需求，根据`config 说明`进行配置，但需要注意一下几点

1. Tftp 加载 uvc_demo.bin 代码时 FH_APP_USING_HEX_PARA [=n]，如果制作成 Flash.img，烧录到 flash 的方法加载程序 FH_APP_USING_HEX_PARA [=y]；
2. Tftp 加载 uvc_demo.bin 代码时，如果使用音频功能，需要根据 flash_layout 信息烧写 rtthread_fw.img arc 固件到对应的 flash 地址；
3. Uvc 在 isoc 端点模式下端点大小默认为 1024，bulk 端点模式下端点大小建议 512，改变端点模式时要注意端点大小是否一起改变，没有则需要手动更改；

uvc 配置

1. uvc 视频格式及分辨率配置：
   - 在 demo/usb_video/format.h 文件中配置了 uvc 视频格式的结构体, format&format2 结构体实现支持的视频格式配置，uvc_frames_xxx 实现对于 xxx 格式的分辨率和帧率配置，mjpeg 格式的分辨率配置需要 8 字节对齐。可增加或删除某些配置。
   - 默认格式的配置：format 结构体第一行格式为默认格式；
   - 默认幅面配置：format 结构体的 defaultindex 参数为各种格式默认幅面 index 号，uvc_frames_xxx 结构体幅面编号从上到下，defaultindex=1 为第一个幅面；
2. flash_layout 配置：
   - 使用 tftp 加载 uvc_demo.bin 程序时可使用默认 flash_layout 配置，如果使用烧录 Flash.img 固件的方式加载程序，则需要修改 flash_layout 配置，此时应修改 flash_layout 为（自定义 flash_layout 需要修改内容较多，见本文后续内容）：

```
{"reservd1",  0x020000, BLOCK_64K, PART_RESERVED},
{"reservd2",  0x050000, BLOCK_64K, PART_RESERVED},
{"cp-firmware", 0x040000, BLOCK_64K, PART_ARCFIRM},
{"reservd3",  0x020000, BLOCK_64K, PART_RESERVED},
{"application", 0x200000, BLOCK_64K,
                    PART_LAST| PART_APPLICATION},
```

uac 配置

使用 tftp 加载 uvc_demo.bin 程序时，需要根据 flash_layout 分布烧录 arc 固件。

版权所有 @ 上海富瀚微电子股份有限公司

编译

make clean;make all, 将 uvc_demo.bin 通过 tftp 加载到平台。或通过 mkflashimg 工具生成 img 文件，烧录到 flash 中，制作 img 方法见Flash 镜像文件制作。

使用

程序运行后通过 usb 接口接入 windows 系统 usb 或其他 usb host 平台中。

11.2.5 flash 镜像制作

1. demo 编译后输出文件在 app/uvc_demo/out/bin/目录,将 app/uvc_demo/out/bin/目录下的 uvc_demo.img 文件拷贝到 flashimg-tool/目录，并重命名 app.img；
2. 拷贝 docs_tools/softwate/pc/uvc/image/RamBoot_Uvc_FH8***.bin 到 flashimg-tool/目录；
3. 拷贝 docs_tools/softwate/pc/uvc/image/update_uvc_fh8***.img 到 flashimg-tool/目录；
4. 拷贝 images/rtthread_fw.img.FH8*** 到 flashimg-tool/目录；
5. 拷贝 docs_tools/softwate/pc/uvc/image/updatecfg 到 flashimg-tool/目录；
6. 拷贝 docs_tools/softwate/pc/uvc/image/rtt_update_uvc_fh8***.ini 到 flashimg-tool/目录；
7. 拷贝 docs_tools/softwate/pc/uvc/image/ isp_param_pack.py 到 flashimg-tool/目录；
8. 拷贝 sensor 效果参数的 hex 文件到 flashimg-tool/目录；
9. 修改 isp_param_pack.py 脚本中 hex 文件名称，然后运行 isp_param_pack.py 脚本生成 param_packed.bin 文件，如果 param_packed.bin 不在 flashimg-tool/目录，则拷贝到 flashimg-tool/目录；
10. 如果不具备上述 7~9 生成 param_packed.bin 文件的条件，也可以直接创建一个 param_packed.bin 的空白文件拷贝到 flashimg-tool/目录，如果这样做，则 isp 在初始化时不再从 flash 中获取 sensor 参数，而是从 rt-thread/lib/fh8xxx/lib/sensor_hex 中获取效果参数，直到下一次使用工具升级 hex 参数之后。
11. 在 linux 系统下使用./mkflashimgv4 rtt_update_uvc_fh8***.ini 制作 flash.img。
12. 在 windows 系统下使用 mkflashimgv4.exe rtt_update_uvc_fh8.ini 制作 flash.img，可在 cmd 中运行，或创建一个.bat 文件，并将 mkflashimgv4.exe rtt_update_fh8.ini 复制到 bat 中，运行 bat 即可。

13. 如果需要在 flash.img 中加入时间版本功能，则需要将下面的文件拷贝到 flashimg-tool/目录，并执行

```
docs_tools/softwate/pc/uvc/image/uvc_version.py(linux)
uvc_version.exe(windows)
```

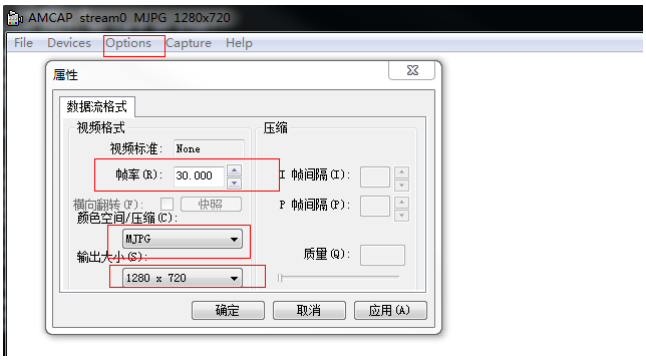
14. 将制作好的 Flash.img 烧录到 flash。

11.2.6 应用操作

UVC

1. 视频播放：

   - Windows 端可使用 pc 自带相机进行视频播放，也可以使用视频工具如微信、QQ 等，但在测试时多使用 AMcap 软件进行播放，AMcap 软件可以进行视频格式、分辨率和帧率的切换，也支持亮度、对比度等视频效果参数的调节。
   - 使用 AMcap 修改视频格式：Options--->Capture Foramt（如果在 windows 端播放 h264 格式视频，请先安装 h264 解码器，否则视频播放会卡顿）



图片 13: 修改视频格式

   - 使用 AMcap 修改视频参数：Options--->Capture Property

图片 14: 修改视频参数

- Linux 系统可使用 guvcview 和 mplayer 软件播放 uvc 视频，理论上所有支持 v4l2 的开源软件都可以直接支持 uvc 设备，这里我们使用 guvcview 开源软件。

```
$./guvcview -b -x 1280x720 -f NV12
$./guvcview -b -x 1280x720 -f MJPG
$./guvcview -b -x 1280x720 -f H264
```

2. Flash 操作

- Uvc 在线升级:在 windows 端使用 docs_tools/softwate/pc/uvc/tool/UsbTool.exe 工具，选择需要升级的 Flash.img 文件，然后点击升级即可。升级过程首先需要 exe 工具识别 uvc 设备，点击升级后 uvc 设备会重启进入升级镜像。
- 根据 UsbTool.exe 的配置文件 UsbToolCfg.ini 中 UpdateType 的配置，升级镜像会将设备模拟成 U 盘设备或虚拟串口设备，用文本编辑器打开 UsbToolCfg.ini，如果 UpdateType=UDisk，升级镜像将会模拟为 u 盘，需要电脑能够正确识别 u 盘才能保证升级完成。

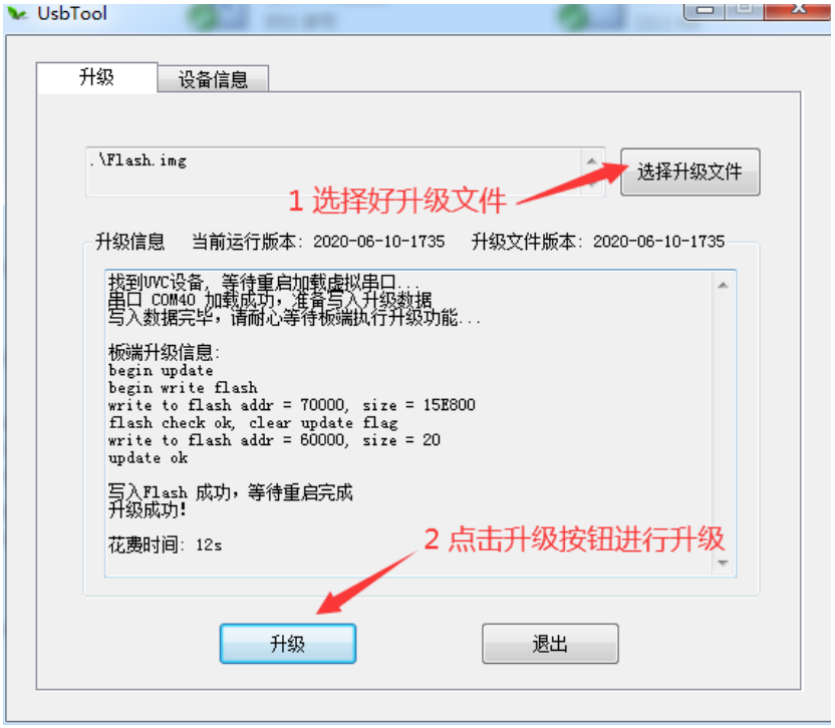
```
[Update]
UpdateType=UDisk
```

如果 UpdateType=Com，升级镜像将会模拟为串口设备，电脑正确识别串口端口才能保证升级完成。

```
[Update]
UpdateType=Com
```

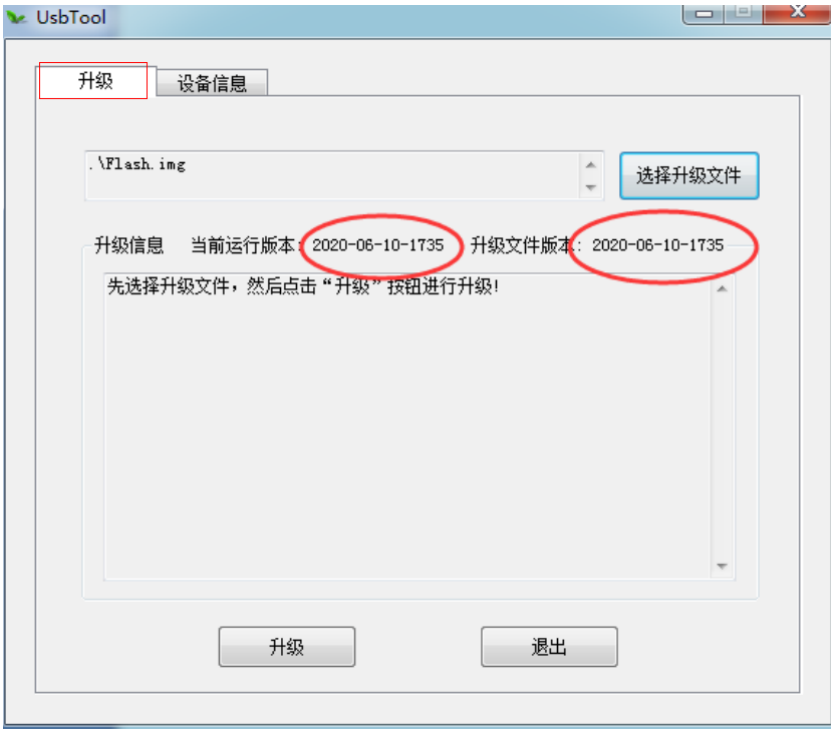
- UsbTool_20211123_1703 版本支持 U 盘、虚拟串口、DFU、HID 四种升级方式，DFU 和 HID 方式升级需要配合新版升级镜像 update_uvc_fh8*** 使用；新版更多支持内容参见软件说明；
- 升级完成后设备会再次重启为 uvc 设备，此时升级完毕。这种方法为部分升级，只修改 updatecfg 标志位内容、参数文件和 app.img 内容。
- 如果需要全部升级，即 ramboot、升级镜像或 flash_layout 布局有修改采用全部升级的方法，在 UsbTool.exe 目录点击***升级完整固件.bat*** 即可启动 UsbTool.exe 的升级完整固件功能。注意：升级完整固件过程中请勿断电, 升级完整固件失败有变砖风险, 请谨慎使用。

 版权所有 @ 上海富瀚微电子股份有限公司



图片 15: 固件升级

- 版本读取显示：启动 UsbTool，切换到``升级" 选项卡页面程序会自动去读取固件版本号一次，如果有正连接着摄像机，会把版本号读取出来，显示为当前运行版本：XXX ，当选择了升级文件之后，会把升级文件版本号也显示出来，显示为升级文件版本：XXX 。



图片 16: 版本信息

版权所有 @ 上海富瀚微电子股份有限公司

- 读取/设置设备信息：启动 UsbTool，切换到``设备信息" 选项卡界面，点``读取" 按钮可以从摄像机读取设备信息，点``设置" 按钮可以把信息更新到摄像机。

图片 17: 设备信息

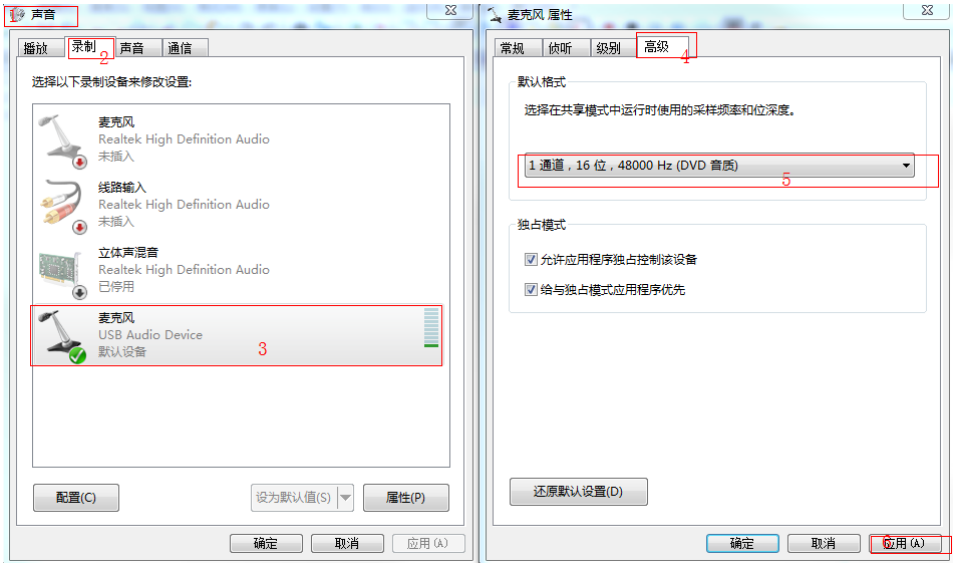
- Hex 文件升级：sensor 效果参数可以保存为 hex 文件，通过 RTT 数据关联工具.exe 进行升级，打开 docs_tools/softwate/pc/uvc/tools/RTT 数据关联工具.exe，选择导入效果参数，选择调试好的 hex 文件，点击确定即可，升级完成后重启设备。
- Flah.img 固件导出：同样使用 RTT 数据关联工具.exe，选择导出 Flash 固件按钮即可导出 flash 中的固件到 windows。使用记事本打开 UsbExportTool-Cfg.ini 文件，修改 ExportSize 可配置导出固件大小，根据 Flash 实际大小选择。

UAC

1. mic 麦克风

- 打开 windows 录音软件可进行测试，或在状态栏右键单击小喇叭图标选择 -录音设备，可看到并管理麦克风。选择对应 usb 麦克风，右键单击选择 -属性 -高级，在此界面可配置麦克风采样率。

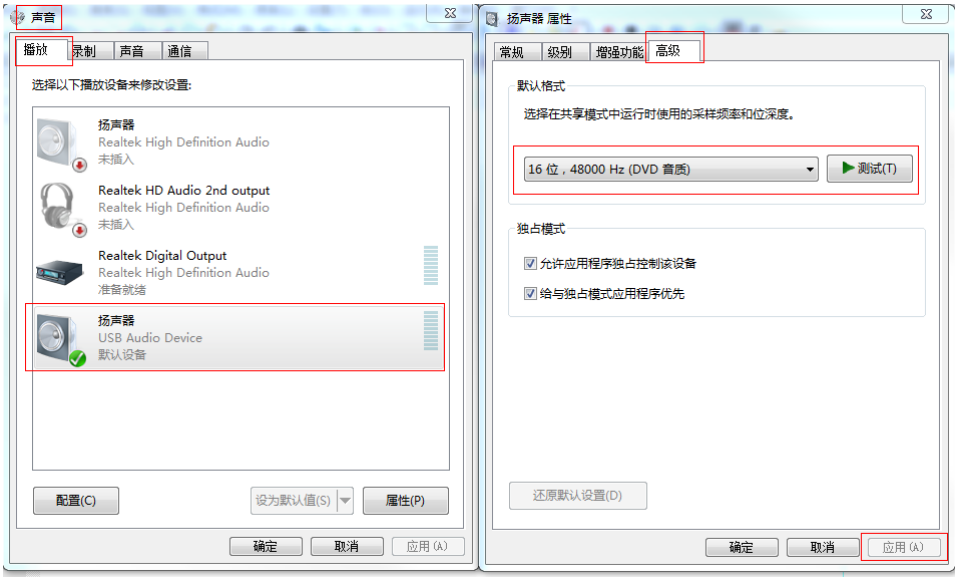
 版权所有 @ 上海富瀚微电子股份有限公司



图片 18: 麦克风设置

2. speaker 扬声器

- 设备接小喇叭或耳机后，即可听到 pc 端的所有操作声音，可播放音乐、视频声音等。在状态栏右键单击小喇叭图标选择 -播放设备，可看到并管理扬声器。选择对应 usb 扬声器，右键单击选择 -属性 -高级，在此界面可配置扬声器采样率。



图片 19: 扬声器设置

usb-vcom

Windows 7 系统需要安装 cdc 串口驱动，串口驱动在 tools 目录中，名称为 usb_cdc_driver.inf。win10 系统在识别到 usb-vcom 设备插入 usb 后会自动安装串口驱动。

 版权所有 @ 上海富瀚微电子股份有限公司

1. coolview

   - usb-cdc-vcom 虚拟串口实现了通用串口、console 和 coolview 三种模式，正常情况下默认处于通用串口模式，打开 usb 串口对应的 shell 窗口，按 shift+7 输入 & 符号可切换到 coolview 模式，切换到 coolview 模式后关闭 usb 串口对应的 shell 窗口，然后直接打开 coolview 软件，选择串口连接方式正确选择串口即可连接 coolview。注意：在串口中输入 & 符号切换到 coolview 模式是一次性的，每次打开 coolview 都需要到串口输入终端进行一次切换，关闭 coolview 或断开 coolview 的串口连接即切换回通用串口模式，需要重复上述步骤重新输入 & 符号切换到 coolview 方可再次连接 coolview，具体模式切换可参考串口打印。

2. cdc-shell 命令行

   - usb-cdc-vcom 虚拟串口还实现了用 usb 模拟串口进行命令行操作的功能，但是默认使用是 coolview，如果要使用 shell 命令行功能，需要关闭 coolview，然后打开 usb 串口对应的 shell 窗口，按 ESC 键即可切换到 shell 模式，关闭 shell 窗口则自动切换到 coolview 模式，注意：打开命令行时请务必关闭 coolview 软件，否则 coolview 软件占用串口端口会导致命令行连接失败。

3. 通用串口模式

   - usb-cdc-vcom 虚拟串口还可用于通用串口用于传输数据，demo 中对通用串口的使用没有实现对应的 example，但符合通用串口的使用方法，直接用 posix 接口操作即可，但是需要注意，通用串口收发数据很容易有 ESC 和 & 符对应的 ASCII 的数据输入导致串口异常切换到其他模式进而造成其他不可预知的问题，因此如果要使用通用串口请务必自定义更复杂的 cdc-shell 和 coolvie 模式的切换指令以防止异常切换。指令修改在 cdc_vcom.c 文件的 _ep_out_handler 函数中。

11.2.7 其他说明

1. uvc 视频参数调节功能具体实现在 demo 中通过回调注册到内核，demo 中只实现了部分参数的调节实现，仅作为参考，如需更多参数请自行设计实现。参数调节实现在 uvc_demo/usb_video/uvc_callback.c 中。

2. sensor sleep 功能目前仅实现了几个 sensor 的配置，作为参考。用户新增 sensor 后需要在 uvc_demo/common/isp/sample_common_isp.c 中 Sensor_Sleep() 和 Sensor_Wakeup() 函数增加 sensor 寄存器配置。

3. uvc 帧率控制函数 get_fmt_by_fps() 目前提供了 720p30、25 帧，960p30、25 帧，1080p30、25、20、15 帧，400wp30、25、20、15 帧的控制选择，其他帧率暂不支持，如需支持可参考已有代码自行增加即可。

 版权所有 @ 上海富瀚微电子股份有限公司

4. uvc 扩展功能在 demo 中提供接口，如需增加自定义功能，可在下面的文件中的中 uvc_extern_intr_proc() 和 uvc_extern_intr_ctrl() 函数里实现：

```
uvc_demo/usb_video/uvc_extern.c
```

5. kernel 中 uvc 驱动提供了 uvc_init.h 头文件，该头文件中提供了 uvc_demo 中用到的结构体、宏定义和 API。其中控件参数的宏定义数值均可根据需求自定义修改。

6. Uvc 的 ramboot 支持直接进入升级系统，当 uvc 应用系统启动崩溃等原因导致无法正常升级时可以从 ramboot 直接进入升级系统进行升级，操作方法：1、关闭芯片 dowmload 模式，2、在串口窗口按住 enter 键，然后重启平台，3、进入升级系统后使用 uvc 升级软件升级即可

7. 在升级系统中使用 set_update() 函数可以更改升级方式

11.2.8 修改 flash_layout

1. 根据需求修改 flash_layout.c 中的 flash 布局，注意：在修改 uvc demo 的 flash_layout 布局时，请勿修改 0x00000 ~ 0x60000 地址之间的 flash 布局，0x20000 ~ 0x60000 为升级镜像 flash 区域，而升级镜像 update_uvc_fh8***.img 中的 flash_layout 已经固定，无法修改。如有需求需要修改此布局，请联系 Fullhan 修改。

2. update_uvc_fh8***.img 升级镜像 flash_layout 如下

```
    {"reservd1",  0x020000, BLOCK_64K, PART_RESERVED},
    {"application", 0x040000, BLOCK_64K, PART_LAST| PART_APPLICATION},
```

3. 修改 Ramboot 文件夹中的 user.c 文件，包括了 UPDATE_CHECK_ADDR 地址、RTT_UPDATE_ADDR 地址和 RTT_FLASH_ADDR 地址，根据 flash_layout 修改。修改之后重新编译 RamBoot，并将编译后的 Ramboot.bin 二进制文件重命名为 RamBoot_Uvc_FH8*.bin，并拷贝到 sdk/docs_tools/software/pc/flashimg-tool/目录。UVC 的 RamBoot 删除了 uboot，具体修改代码见附件 1。

4. 修改 app/uvc_demo/usb_video/uvc_extern.h 文件中的 ISP_PARAM_ADDR 地址和 UPDATE_CFG_ADDR 地址，根据 flash_layout 修改。重新编译后将 app.img 拷贝到 sdk/docs_tools/software/pc/flashimg-tool/目录。

5. 修改 mkflashimg 工具的 ini 配置文件，根据 flash_layout 修改。注意：请勿修改 update_uvc_fh8***.img 地址。

 版权所有 @ 上海富瀚微电子股份有限公司

6. 修改 UsbTool.exe 的配置文件 UsbToolcfg.ini，使用记事本打开该文件修改 UpdateFlagAddr，请务必结合 flash_layout 布局进行修改，否则不能完成正常升级。

7. 修改 RTT 数据关联工具.exe 的配置文件 UsbExportToolCfg.ini，修改 ExportSize 可配置导出固件大小，根据 Flash 实际大小选择。

- 附件 1：Ramboot 修改代码

```
#include "user.h"

#define CONFIG_PHY_RESET_GPIO   (2)
#define CONFIG_CIS_RESET_GPIO   (14)

#define UBOOT_FLASH_ADDR        (0x50000)
#define UBOOT_SIZE              (0x38000)
#define UBOOT_ENTRY         (0xa0800000)

#define RTT_FLASH_ADDR      (0xd0000)
#define RTT_HEADER_SIZE     (RTT_FLASH_ADDR + 4)
#define RTT_DATA_SIZE       (RTT_FLASH_ADDR + 8)
#define RTT_ENTRY           (0xa0000000)

#define BOOT_FLAG_OFFSET    (0x48000)

#define UBOOT_DETECT_GPIO   (0)

#define RTT_UPDATE_ADDR     (0x20000)
#define UPDATE_CHECK_ADDR   0x60000

static int goto_uboot(void)
{
    return 0;
}

int fh_strlen(const char *s)
{
    const char *sc;

    for (sc = s; *sc != '\0'; ++sc) /* nothing */
        ;

    return sc - s;
}

int fh_memcmp(void *pBuf1, void *pBuf2, int size)
{
```

```
    int i = 0;
    char *pData1 = (char *)pBuf1;
    char *pData2 = (char *)pBuf2;
    for (i=0; i<size; i++)
    {
        if (pData1[i] != pData2[i])
        {
            return (pData1[i] - pData2[i]);
        }
    }

    return 0;
}

int check_update()
{
    int update = 1;

    int i;
    int err = 0;
    char data[32] = {0};

    FH_Flash_Read(UPDATE_CHECK_ADDR, 28, (unsigned char *)data);

    char *strUpdate = "Hello World, Run!";
    if (fh_memcmp(data, strUpdate, fh_strlen(strUpdate)) == 0)
    {
        update = 0;
    }
    return update;
}

void user_setttings(void)
{
    /* TODO: pull up before sensor init */

    /* wifi power/ Reset */

    if (goto_uboot())
    {
        /* Set Image Address, Size, etc.*/
        FH_Boot_SetAddr(UBOOT_FLASH_ADDR, UBOOT_SIZE, UBOOT_ENTRY);
    }
    else
    {
        unsigned int flash_addr;
        unsigned int image_size;

        unsigned int rtt_img_addr;
```

 版权所有 @ 上海富瀚微电子股份有限公司

```
        rtt_img_addr = check_update() ? RTT_UPDATE_ADDR : RTT_FLASH_ADDR;

        FH_Flash_Read(rtt_img_addr+4, 4, (unsigned char *)&flash_addr);
        FH_Flash_Read(rtt_img_addr+8, 4, (unsigned char *)&image_size);

        FH_Boot_SetAddr(rtt_img_addr + flash_addr, image_size, RTT_ENTRY);
    }
}

void user_init()
{
    FH_SetAutoInitPLL(1);
}

void user_pll0()
{
    FH_SetPLL(0,0x64,0,0,1,0x8,0x8);
}

void user_pll1()
{
    //FH_SetPLL(1,0x3c,0,1,0,0x8,0x8);
    FH_SetPLL(1,0x58,0,1,1,0x8,0x8);
}

void user_pll2()
{
    FH_SetPLL(2,0x55,0,0,1,0x8,0x8);
}
```

11.3 USB-GMAC 驱动使用指南

11.3.1 USB-GMAC 驱动说明

USB-GMAC 模块型号当前仅支持 AX88772，默认关闭，如需使用，需要在 menuconfig 中进行配置。

11.3.2 USB-GMAC 使用说明

1. 进入 sdk rt-thread 目录，执行 make menuconfig；
2. 进入 make menuconfig，选择 Enable usb host mode core，位置如下：

   Location:

```
->Drivers
   -> Usb configuration
    [*] Enable usb driver (RT_USING_USB [=y])
     -> Choose usb mode (<choice> [=y])
        (X) Enable usb host mode core
```

3. 选择 usb host controller enable、usb ethernet enable 和 usb ethernet select asix device，位置如下：

   Location:

```
->Drivers
   -> Usb configuration
   [*] usb host controller enable
   [*] usb ethernet enable
   [*]   usb ethernet select asix device
```

4. 添加完对应选项，make 即可。

11.3.3 USB-GMAC 功能测试

1. 系统起来之后插上 usb 转 gmac 模块，正常识别 usb-gmac 后会生成 u0 接口；
2. 正常使用网络命令操作即可。dhcp 获取 IP 地址或者 ifconfig u0 xx.xx.xx.xx 配置网口 IP 地址进行使用。

 版权所有 @ 上海富瀚微电子股份有限公司

12 快速启动开发指南

快速启动，即快速启动应用代码，保证期望的功能模块尽早运行。

为实现快速启动的目标，我们采用了边运行边加载的方式，从而保证核心代码可以最快获得运行。

因此，整个应用代码不是一次性加载到内存进行运行的，而是先只加载必须的代码，对不影响初始启动的代码由应用程序后续自行加载。

12.1 快速启动代码的开发

快速启动的开发，对代码的配置有严格的要求，否则，即使编译生成的代码能够通过 TFTP 方式加载运行，也可能最终在烧录到 flash 中后，仍然不能正常运行。

快速启动代码的开发必须注意以下几点：

1. 确定应用工程的 flash 布局，即：最终产品的 flash 分区划分，其中，最重要的是，应用层代码在 flash 中的存放位置（可参考Demo 编译与运行中Flash 分区配置章节）；
2. 用户可以根据项目/产品需要增加初始化代码，此处代码仅指应用启动前的代码，不包含应用方案代码；
3. 修改链接文件 (baseos/media.ld)，保证用户添加的初始化代码在运行时被正确加载（链接文件的修改，请参考 ``GNU 工具 LD 的说明文档"）；
4. 为保证快速启动，用户添加的初始化代码不应很大，也不宜耗时太长；
5. 如果在快速启动中支持了 VBUS，则无论是 TFTP 方式还是烧录到 Flash 并从 Flash 启动，都需要预先烧录 VBUS 固件，烧录方式见Demo 编译与运行中烧写协处理器固件章节。

   快速启动应用在其它开发上与非快速启动应用没有太大差别，仅需在配置时，选择 fastboot_board，同时需要开启 spi flash 的 4 线模式。

12.2 快速启动代码的编译与运行

快速启动代码在编译时，会得到 2 个镜像文件：app.img 和 rtthread.bin，其中：

- rtthread.bin 用于 tftp 方式运行/调试
- app.img 用于烧录到 flash 中执行

在编译时，最终会输出 load code size: 0xAB000 字样，说明初始运行代码的大小，即，运行代码所需加载的代码大小，这个数值在后面将会用到，此处记录该值为 LOAD_SIZE。

快速启动代码的调试运行一般分 2 步进行：

1. 第一步是通过 tftp 方式运行 rtthread.bin ，确认 tftp 方式加载的代码可以正常运行；
2. 第二步是将 app.img 烧录到 flash 中，从 flash 启动运行。

仅当 tftp 方式运行通过时，才能进行 flash 烧写，从而验证 flash 启动运行是可以正常工作的，否则，烧录到 flash 的代码很可能不能正常运行。

tftp 方式运行同**非快速启动**，此处不再赘述。

下面仅就烧录 app.img 到 flash 中的代码启动执行作说明。

app.img 在 flash 中的烧录地址，需要与 flash_layout.c 中的配置一致，其起始地址需要通过计算得到：

- 查找 flash_layout.c 中具有 PART_APPLICATION 属性的字段
- 表项第二个字段表示分区的大小
- 计算 PART_APPLICATION 属性段前所有段大小的总和，该值即为 app.img 应该烧录到的 flash 地址偏移，记为 FLASH_OFFSET。

在 UBoot 中，通过 tftp 的方式把 app.img 下载并烧录到 FLASH_OFFSET 地址处，烧录方法参考Demo 编译与运行中烧写协处理器固件章节的方法。

在正式运行代码前，最好先给板子断电，让板子以冷启动的方式运行。

进入 U-Boot，执行命令：

```
U-Boot>sf probe 0
U-Boot>sf read a0000000 FLASH_OFFSET + 0x10 LOAD_SIZE
U-Boot>go a0000000
```

举例如下：

1. 编译输出为：

```
 load code size: 0x58000
 user code size: 0x287693
total code size: 0x2e0693 (3,016,339)
```

根据输出，可知, LOAD_SIZE 为 0x58000

2. app 镜像在 flash 中的偏移为 0x100000(FLASH_OFFSET)

 版权所有 @ 上海富瀚微电子股份有限公司

3. UBoot 下的加载测试命令应为：

```
U-Boot> sf read a0000000 0x100010 0x58000
```

其中：0x100010 = FLASH_OFFSET + 0x10 = 0x100000 + 0x10
上述步骤执行后，系统即可正常运行，同时说明代码正确，否则，还需要进一步调整代码。

12.3 快速启动的 xmodem 镜像制作

快速启动在实际应用时，需要跳过 U-Boot 的加载，在 Ramboot 中直接加载应用代码，以达到快速启动的目的。为支持快速启动，通常需要在 Ramboot 中做一些特别的初始化操作，以保证应用代码可以正常启动运行。

在 SDK 的快速启动应用中，需要在 Ramboot 中完成：

1. pinmux 切换，切换 CIS_CLOCK.
2. enable CIS_CLOCK
3. reset sensor

对于以上修改，用户可以通过修改发布包中的 user.c，并编译链接生成 RamBoot.bin 来实现。

对于 FH865x/FH885xV200 芯片，user.c 修改如下：

```
void fast_rtos_settings(void)
{
    unsigned int val;

    /* un-gate cis-clock */
    val = *(volatile unsigned int *)0xf000001c;
    val &= 0xff7fffff;
    *(volatile unsigned int *)0xf000001c = val;

    /* select cis-clock pinmux */
    val = *(volatile unsigned int *)0xf000009c;
    val &= 0xf0ffffff;
    *(volatile unsigned int *)0xf000009c = val;

    /* reset sensor */
    FH_Gpio_SetDirection(CONFIG_CIS_RESET_GPIO, 1);
```

```
    FH_Gpio_SetValue(CONFIG_CIS_RESET_GPIO, 0);
}
```

修改完成 user.c 后，在当前目录下执行 make(bsp/ramboot/FH865X_SPI_4_WIRE/)，可以生成 RamBoot.bin 文件。

生成 RamBoot.bin 文件后，还要使用该文件结合 u-boot.bin 生成 xmodem.img, 方法如下：(xxxx 指代芯片型号)

1. 进入芯片目录，(如: docs-tools/software/pc/flashimg-tool/865X/)
2. 拷贝编译得到的 RamBoot.bin 到当前目录
3. 拷贝 $SDK/images/u-boot_xxxx.bin 到当前目录，并重命名为 u-boot.bin
4. 执行../mkflashimgv4 xxxx_fast.ini，生成 xmodem.img

注意：如果用户在产品设计中，使用了不同的 GPIO 来控制 sensor 的复位，则需要修改 CONFIG_CIS_RESET_GPIO 的定义。

12.4 快启模式与非快启模式的切换

在 UBoot 中可以正常加载 app 镜像，且完成支持快速启动的 xmodem 烧录后，可以通过 bootmode 命令进行启动模式的切换。
切换到快启模式：

```
msh \> bootmode -f
```

切换到非快启模式：

```
msh \> bootmode -u
```

切换到 UBoot 加载模式下，可以对 app 调试镜像进行更新
需要注意的一点是，默认的启动标记地址为 0x48000，在制作 xmodem.img 的 ini 配置文件中定义，如果该默认地址有改动，需要同步修改到 bootmode 命令中。bootmode 命令源码位于 platform/common/bootswitch.c 中。

12.5 人形识别功能应用

在 FH865x 平台上，快速启动默认开启人形识别功能，因此，需要加载人形识别的模型文件，模型文件必须烧录到带有 PART_NN_MODEL 属性的分区上，分区定义在 flash_layout.c 中，在烧录到 flash 上之前，需要根据应用进行压缩制作，支持 GZIP/LZOP 这 2 种压缩方式，制作可以通过 build/tools/gznbg.sh 脚本完成，在 Linux 控制台上：

 版权所有 @ 上海富瀚微电子股份有限公司

```
$ build/tools/gznbg.sh [1|2] lib/fh865x/lib/nna_modules/pedestrain/
  ↪ persondet.nbg
```

上述命令生成 nbg.(zip|lzop).img，该文件可以直接烧录到具有 PART_NN_MODEL 属性的分区上。

注意，当前版本中，NBG 文件为特别定制版本，支持 1：1 幅面，而非快速启动中使用的为 16：9 幅面。

另外，在进行快启人形识别时，因为 sensor 曝光时间的不同，需要根据应用的 sensor 进行适当的调整：

1. 选择人形识别时间：

SDK 从第 4 帧开始识别，即识别小图阶段的 4/5/6 这 3 帧图像，如果应用中第四帧图像不适合人形识别，需要调整小图帧数以及识别帧数 2. 选择确认人形识别需要的计算帧数：
由 platform/xxxx/fast_board/fastvideo_config.h 中 NN_DETECT_FRAMES 定义，SDK 默认识别小图阶段的最后几帧

12.6 快速启动时间统计

支持快速启动的耗时统计，需要在 menuconfig 中打开配置：evaluate startup timecost。

打开该配置后，启动完成后会打印如下信息：

```
************* system startup timecost statistics *************
                                                    (unit: us)
            OS startup time:    4262781

      system init timecost:
             startup time:    4264958
        pre-init code cost:      22022
        load drv code cost:      28987
         run drv code cost:       3727
        load usr code cost:     260833
         run usr code cost:     183019
       sys init total cost:     498596

     fastboot startup cost:
          isp startup time:    4265066
     sensor init(pre) cost:      18499  @4268085
       small pic time cost:      50150  (AE/AWB)
      resize pic time cost:      40916
     sensor post init cost:      28348  @4347878
        isp init done time:    4286979
      first small pic time: [   4293928    4301823]
        first big pic time: [   4475065    4539424]

          dsp startup time:    4315968
         module init cost:       2345
      wait isp resize cost:     267851

      Flash Load time cost:
              DSP&ISP cost:      10549,  speed: 24073(KB/s)
            User Code cost:     133942,  speed: 23516(KB/s)
           decompress cost:     102169
             VBUS Code cost:      11092,  speed: 22156(KB/s)
           decompress cost:      10641

******************** end of statistics ********************
```

图片 20: 启动耗时统计

其中：

- 所有大数据单位为微秒

- 统计时间自 OS 启动后进入 rtthread_startup 开始
- Cost 显示的耗时时间，time 是时间节点（可以与 OS startup time 对比）

12.7 快速启动常见问题及故障调试

1. App.img 烧录到 flash 中后上电启动，在打印`## Starting application at 0xA0000000 ...`后无任何其它输出

出现这个问题的通常原因是加入的启动代码引用了未加载代码，需要确保加入的初始代码必须整体添加到``启动代码" 段中

另外一种情况是在通过 U-Boot 加载代码时，没有偏移 16 个字节，见快速启动代码的编译与运行

2. 启动中打印`No Code found for SEG2`

通常是代码的烧录地址和 flash 分区方案不一致所致，请检查确认代码的烧录地址和 flash_layout.c 中定义的分区方案是否一致。

需要注意的是，flash_layout.c 中仅定义了分区的大小，分区的偏移需要通过计算得到。

3. 启动后系统崩溃

第二段代码未正确加载或是引用代码未加载所致，请检查所添加的初始代码已被链接到初始启动代码，以及代码被烧录到正确的 flash 地址

4. 快速启动后不能正确识别 sensor

针对快速启动，ramboot 需要定制，需要在 ramboot 中完成对 sensor 的复位等操作，请更新 ramboot。

5. FH865x 平台快速启动后不能正确进行人形识别

存在 2 种情况：

i. 未正确烧录人形识别的模型文件，或未烧录到正确的分区上
ii. 烧录的模型文件支持幅面不正确

6. NN 模型初始化失败：

请确认模型文件的大小是否超过最大支持大小，最大支持大小在 platform/fast-boot/fh_nn_init.c 中定义，默认为 3MB。

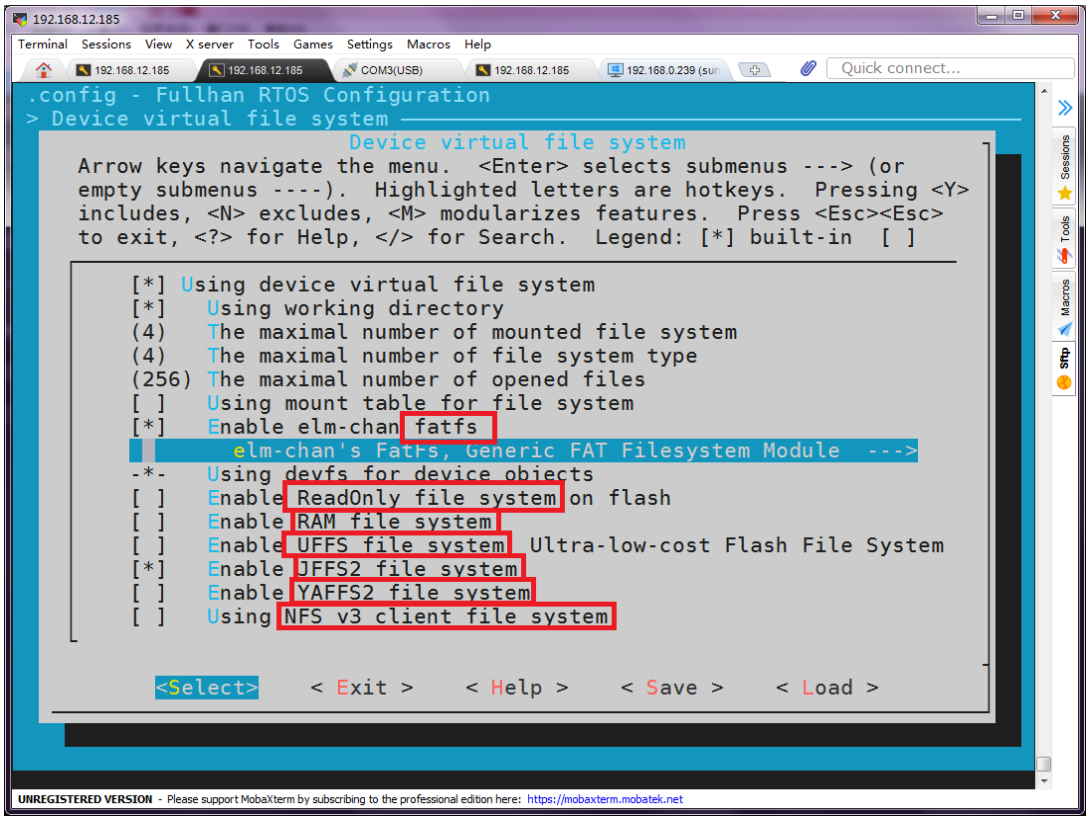
如果模型文件超过 3MB，请修改最大支持大小的数值；修改后如果出现系统内存不足问题，则需要扩大 OS 内存大小。

13 系统扩展

RTT SDK 除了提供基本的内核之外，还提供了很多的系统扩展功能及第三方组件，如文件系统，C++ 支持，命令行支持，LwIP 网络协议栈等。

13.1 文件系统

RTT SDK 支持基于设备的文件系统,支持 RAMFS/ROMFS/JFFS2/FAT/UFFS/YAFFS/NFS 等文件系统，系统默认开启 JFFS2/FAT 文件系统支持，需要使用其它文件系统功能，需要在菜单配置中进行配置打开。



图片 21: 文件系统配置选择

对于系统支持的文件系统，在系统启动时，可以实现文件系统的自动挂载。

- JFFS2 文件系统，在 flash 布局配置中配置 ROOTFS 属性，该 Flash 分区即可被自动挂载为根文件系统（默认 JFFS2 挂载根文件系统）
- FAT 文件系统，板子上插入了 SD 卡，且开启了 SDIO 支持，会自动挂载 FAT 文件系统到/mnt 文件夹下（或是/根文件系统）

- UFFS 文件系统，默认挂载到/nand0
- YAFFS 文件系统，同 JFFS2，默认挂载到/根文件系统
- RAMFS 文件系统，默认挂载到根文件系统
- ROMFS 文件系统，默认挂载到/rom
- NFS 文件系统，默认挂载到/nfs

需要注意的是，对于 FAT 文件系统，由于 SD 卡是可插拔设备，仅在系统启动中，才会自动挂载该文件系统；在系统运行中插入 SD 卡时不会自动挂载，需要用户手工挂载；
文件系统的挂载使用 mount 命令，具体使用方法可以通过`mount -h`命令进行查看。
文件系统的操作，支持常用命令，如：ls/cd/touch/rm/cat/mkdir/df/cp/mv/等命令。
说明：FAT 文件系统包括 FAT32 和 exFAT 文件系统。

13.2 C++ 支持

RTT SDK 可以扩展支持 C++，在需要使用 C++ 功能时，仅需要在系统配置中开启 C++ 支持功能即可；SDK 自动编译/链接 C++ 支持库。
RTT SDK 目前仅支持 C++ 03（或称 98）标准，不支持 C++ 11 及以后标准。
注意事项：
由于 RTT SDK 默认使用分段加载方法进行系统的加载/启动，因此，在初始化阶段不建议使用 C++ 代码；在软件架构上，C++ 初始化过程也是在后加载代码加载完成后进行的。

13.3 shell 支持

RTT SDK 支持 shell 扩展，支持用户命令行，用户可以自行定义命令，进行命令行命令扩展。
命令行默认支持常用的文件系统操作，以及网络操作（ping, ifconfig 等）
查看当前命令行支持哪些命令，可以使用 help 命令来实现；对于大部分支持的命令，都支持 -h 选项，用以说明命令的具体用法。
如果需要在代码中扩展支持新的命令，需要：

1. C 代码中包含头文件 rttshell.h:

```
#include <rttshell.h>
```

2. 定义用户命令，函数的参数形式为：

```
int user_cmd(int argc, char *argv[])
```

3. 导出用户命令：

```
SHELL_CMD_EXPORT(user_cmd, user cmd description)
```

注意：
如果用户定义的命令在编译时生成为库文件，则可能因为链接原因，导致最终该命令不可用，此时，用户需要修改链接文件 extern.ld，并在其中添加如下代码：`EXTERN(user_cmd)`
这样，可以确保 user_cmd 命令被最终链接到可执行文件中，从而可以被正常使用。
对于旧版本用户，需要注意：
旧版本 SDK 系统默认使用 finsh 命令行，新版本 SDK 默认使用 msh 命令行；finsh 命令不支持命令参数检查，需要用户自行处理参数，因此，使用不当，很容易导致系统崩溃；而 msh 中，则需要处理命令行参数问题，并最终调用相应的命令实现，可以在开发阶段避免这种参数错误导致的问题，从而可以大大提高系统的健壮性，避免误操作引起系统崩溃。
对于开发人员，需要注意，finsh 命令与 msh 命令的函数形式差异。

13.4 负载统计

RTT SDK 支持 top 命令，可以实时查看系统的负载情况，缺省每 2 秒刷新一次，可以通过 -d 参数进行调整刷新间隔。
top 支持依赖于系统的 HOOK 支持，使用中，必须打开内核的 Hook 支持选项，才可以使用 top 功能。
top 显示系统线程的优先级，调度次数，总计耗时，以及 CPU 占用百分比。

```
thread           pri     enter   tick(ms)   pct.
------           ------- ------- ------- ----
tcpip            100     169     0          0
etx              126     0       0          0
erx              126     284     0          0
tshell           20      1       37         1
mmcsd_detect     64      0       0          0
init_drv         80      0       0          0
tidle            255     162     1949       97
timer            8       0       0          0

Press 'q' to quit.
```

图片 22: top 命令显示

在正常的应用中，不应该出现 tidle 线程 (即 IDLE 线程) 占用为 0 的情况，如果长时间出现这样的情况，会导致系统资源无法回收，进而导致系统资源不足，程序运行异常等不可预知情况；如果长时间出现这种情况，需要对应用程序进行优化、功能裁剪。
如果软件代码确有这种需求，可以使用系统的 IDLE 钩子功能，使相关代码运行在 IDLE 优先级，且能同时保证系统的 IDLE 线程可以正常执行；系统默认支持 4 个 IDLE 钩子 (需要开启 HOOK 支持)。

13.5 其它第三方组件

RTT SDK 移植了部分第三方的组件库，包括:

- bonjour
- mbedtls
- libcurl
- udt
- srt
- miniupnpc
- unlzma
- unlzo
- iperf
- zlib
- cJSON
- mxml
- zbar
- tar

大部分组件都使用 POSIX 接口移植实现，组件支持的功能由第三方组件库提供。

14 网络支持

RTOS 网络接口使用 posix 标准，但是 RTOS 协议栈只支持部分功能。本文针对 socket 协议族类型、setsockopt、getsockopt 和 ioctl 支持的功能进行列举。

14.1 socket 协议族支持

序号	family	功能	备注
1	AF_INET	IPv4	支持
2	AF_INET6	IPv6	支持
3	AF_PACKET	RAW	有限支持

其它未列举的协议族暂不支持。
SDK 中默认 AF_INET 和 AF_INET6 已开启，AF_PACKET 未开启，如若需要使用此功能，需 make menuconfig 打开 Enable RAWLINK 选项。位置如下：
Location:

```
->light weight TCP/IP stack
    [*] Enable RAWLINK
```

14.2 setsockopt 支持

序号	level	optname	备注
1	SOL_SOCKET	SO_BROADCAST	支持
2	SOL_SOCKET	SO_KEEPALIVE	支持
3	SOL_SOCKET	SO_REUSEADDR	支持
4	SOL_SOCKET	SO_SNDTIMEO	支持
5	SOL_SOCKET	SO_RCVTIMEO	支持
6	SOL_SOCKET	SO_RCVBUF	支持
7	SOL_SOCKET	SO_SNDBUF	只有 TCP 支持
8	SOL_SOCKET	SO_LINGER	支持
9	SOL_SOCKET	SO_NO_CHECK	只有 udp 支持，udp lite 不支持
10	SOL_SOCKET	SO_BINDTODEVICE	支持
11	IPPROTO_IP	IP_TTL	支持
12	IPPROTO_IP	IP_TOS	支持
13	IPPROTO_IP	IP_PKTINFO	有限支持
14	IPPROTO_IP	IP_MULTICAST_TTL	支持
15	IPPROTO_IP	IP_MULTICAST_IF	支持
16	IPPROTO_IP	IP_MULTICAST_LOOP	支持
17	IPPROTO_IP	IP_ADD_MEMBERSHIP	支持

 版权所有 @ 上海富瀚微电子股份有限公司

序号	level	optname	备注
18	IPPROTO_IP	IP_DROP_MEMBERSHIP	支持
19	IPPROTO_TCP	TCP_NODELAY	支持
20	IPPROTO_TCP	TCP_KEEPIDLE	支持
21	IPPROTO_TCP	TCP_KEEPINTVL	支持
22	IPPROTO_TCP	TCP_KEEPCNT	支持
23	IPPROTO_TCP	TCP_MAXSEG	支持
24	IPPROTO_IPV6	IPV6_V6ONLY	支持
25	IPPROTO_IPV6	IPV6_JOIN_GROUP	支持
26	IPPROTO_IPV6	IPV6_LEAVE_GROUP	支持
27	IPPROTO_UDPLITE	UDPLITE_SEND_CSCOV	支持
28	IPPROTO_UDPLITE	UDPLITE_RECV_CSCOV	支持
29	IPPROTO_RAW	IPV6_CHECKSUM	支持

SDK 中默认 SO_LINGER、TCP_KEEPIDLE、TCP_KEEPINTVL、TCP_KEEPCNT、UDPLITE_SEND_CSCOV、UDPLITE_RECV_CSCOV、IP_PKTINFO 等功能未开启。其它未列举出的功能暂不支持。

1. IP_PKTINFO 功能支持不完全，暂不建议使用。
2. 如若需要使用 TCP_KEEPIDLE、TCP_KEEPINTVL、TCP_KEEPCNT 功能，需要在 rt-thread/net/lwip-2.1.2/src/lwipopts.h 中将 LWIP_TCP_KEEPALIVE 定义为 1，默认为 0。
3. 如若需要使用 UDPLITE_SEND_CSCOV、UDPLITE_RECV_CSCOV 功能，需要在 rt-thread/net/lwip-2.1.2/src/lwipopts.h 中将 LWIP_UDPLITE 定义为 1，默认为 0。
4. 如若需要使用 SO_LINGER 功能，需 make menuconfig 打开 Enable SO_LINGER 选项。位置如下：

   Location:

```
->light weight TCP/IP stack
   [*] Enable SO_LINGER
```

14.3 getsockopt 支持

序号	level	optname	备注
1	SOL_SOCKET	SO_ACCEPTCONN	支持
2	SOL_SOCKET	SO_BROADCAST	支持
3	SOL_SOCKET	SO_KEEPALIVE	支持
4	SOL_SOCKET	SO_REUSEADDR	支持
5	SOL_SOCKET	SO_TYPE	支持
6	SOL_SOCKET	SO_ERROR	支持
7	SOL_SOCKET	SO_SNDTIMEO	支持
8	SOL_SOCKET	SO_RCVTIMEO	支持
9	SOL_SOCKET	SO_RCVBUF	支持
10	SOL_SOCKET	SO_SNDBUF	只有 TCP 支持
11	SOL_SOCKET	SO_LINGER	支持

序号	level	optname	备注
12	SOL_SOCKET	SO_NO_CHECK	只有 udp 支持，udp lite 不支持
13	IPPROTO_IP	IP_TTL	支持
14	IPPROTO_IP	IP_TOS	支持
15	IPPROTO_IP	IP_MULTICAST_TTL	支持
16	IPPROTO_IP	IP_MULTICAST_IF	支持
17	IPPROTO_IP	IP_MULTICAST_LOOP	支持
18	IPPROTO_TCP	TCP_NODELAY	支持
19	IPPROTO_TCP	TCP_KEEPIDLE	支持
20	IPPROTO_TCP	TCP_KEEPINTVL	支持
21	IPPROTO_TCP	TCP_KEEPCNT	支持
22	IPPROTO_TCP	TCP_MAXSEG	支持
23	IPPROTO_TCP	TCP_INFO	支持
24	IPPROTO_IPV6	IPV6_V6ONLY	支持
25	IPPROTO_UDPLITE	UDPLITE_SEND_CSCOV	支持
26	IPPROTO_UDPLITE	UDPLITE_RECV_CSCOV	支持
27	IPPROTO_RAW	IPV6_CHECKSUM	支持

getsockopt 的功能支持与 setsockopt 类似，其它未列举出的功能暂不支持。

14.4 网络相关 ioctl 支持

序号	CMD	功能	备注
1	FIONREAD	获取接收缓冲区中的字节数	支持
2	FIONBIO	清除或设置非阻塞 I/O 标志	支持
3	SIOCETHTOOL	检测网卡与网线连接状态	支持
4	SIOCGIFADDR	获取接口 IP 地址	支持
5	SIOCSIFADDR	设置接口 IP 地址	支持
6	SIOCGIFNETMASK	获取子网掩码	支持
7	SIOCSIFNETMASK	设置子网掩码	支持
8	SIOCGIFMTU	获取接口 mtu	支持
9	SIOCSIFMTU	设置接口 mtu	支持
10	SIOCSIFHWADDR	设置接口 MAC 地址	支持
11	SIOCGIFHWADDR	获取接口 MAC 地址	支持
12	TIOCOUTQ	检测当前发送缓冲区数据大小（unacked + unsent）	支持
13	SIOCGIFCONF	获取接口信息	部分支持
14	SIOCGIFFLAGS	获取接口标志	支持
15	SIOCSIFFLAGS	设置接口标志	支持
16	SIOCGIFINDEX	获取接口索引	支持
17	SIOCADDRT	添加路由表项	不支持（未实现）
18	SIOCDELRT	删除路由表项	不支持（未实现）

SDK 中列举出的功能默认都已支持，其它未列举出的功能暂不支持。

 版权所有 @ 上海富瀚微电子股份有限公司

15 Flash 操作指南

本文基于 FH_SPIC 控制器，提供 Flash 器件的移植方法及相关操作。

- 当选用的 SPI Nor Flash 器件的型号与 SPI Nor Flash 器件兼容性列表中型号不同时，需要在 SPI Nor Flash 驱动的 ID 列表中新增器件 ID 节点以及器件相关的功能函数，主要包括 id 值、时钟、实现并指定器件相关的功能函数接口（4byte 寻址、四线读使能、复位函数等）。
- 当选用的 SPI Nand Flash 器件的型号与 SPI Nand Flash 器件兼容性列表中型号不同时，需要在 SPI Nand Flash 驱动的 ID 列表中新增器件 ID 节点以及器件相关的功

  能函数，主要包括 id 值、qe 开启 bit、ecc 开启比特等。

15.1 SPI Nor Flash 器件的移植

15.1.1 SPI-Nor U-boot 下的移植

相关驱动路径

uboot 下的路径：
bsp\uboot\drivers\mtd\spi\
路径下与移植相关的文件

```
spi_flash.c :flash 探测及注册的入口，区分不同厂家的 flash 并进行注册与初始化
winbone.c\gd.c\...\zb.c : 部分特殊器件的功能函数
default_flash.c: 通用的功能函数代码
Makefile: 新加特殊器件的功能函数文件，需要在此添加文件名以便能进行编译
```

如何新增一颗 SPI Nor Flash

以 GD25Q128 为例，讲述如何新增一颗 flash

- 步骤 1：查阅器件手册，新增 id 编号。

 版权所有 @ 上海富瀚微电子股份有限公司

通过 9FH 命令找到 Flash id 信息

TABLE OF ID DEFINITIONS

GD25Q128E

Operation Code	MID7-MID0	ID15-ID8	ID7-ID0
9FH	C8	40	18

图片 23: 获取 flash id

获取器件的 chip size、block size 信息

GD25Q128E

Each device has	Each block has	Each sector has	Each page has	
16M	64/32K	4K	256	Bytes
64K	256/128	16	-	pages
4K	16/8	-	-	sectors
256/512	-	-	-	blocks

图片 24: 获取 chip size

在相应的文件 gd.c 中添加此款 flash 信息，如无该厂家对应的 c 文件，可由 default_flash.c 复制一份，修改名字为相应厂家，然后按步骤修改配置，首先在宏定义部分添加此款 flash 的 chip id 即 0x4018，如下图所示：

```
#define GIGADEVICE_ID_GDX16      0x3015
#define GIGADEVICE_ID_GDX32      0x3016
#define GIGADEVICE_ID_GDX64      0x3017
#define GIGADEVICE_ID_GDQ64      0x4017
#define GIGADEVICE_ID_GDQ128FV   0x4018
#define GIGADEVICE_ID_GDQ256FV   0x4019
#define GIGADEVICE_ID_GDQ16FV    0x4015
#define GIGADEVICE_ID_GDQ16BV    0x6015
```

图片 25: Flash id 宏定义

接着找到 gd_spi_flash_table[] 这个表格，添加详细信息，如下图所示：

 版权所有 @ 上海富瀚微电子股份有限公司

```
    {
        .id          = GIGADEVICE_ID_GDQ128FV,
        .l2_page_size    = 8,
        .pages_per_sector  = 16,
        .sectors_per_block = 16,
        .nr_blocks     = 256,
        .name          = "GDQ128",
#ifdef FLASH_MULTI_WIRE_SUPPORT
        .multi_wire_open = ONE_WIRE_SUPPORT | DUAL_WIRE_SUPPORT | QUAD_WIRE_SUPPORT | MULTI_WIRE_SUPPORT,
#endif
        .spd_info = {
/*          {.speed = 75000000,.delay = 2,},
            {.speed = 60000000,.delay = 2,}, */
            {.speed = 50000000,.delay = 1,},
        },
```

图片 26: Flash 参数

其中 id、pages_per_sector、sectors_per_block、nr_block 按 datasheet 所述填写即可，l2_page_size 为 log2 为底 page_size 的对数，即 2 的 8 次方为 256，即填 8，此变量之后在读写时用来计算地址偏移量。

FLASH_MULTI_WIRE_SUPPORT 的宏开启时，能使用多线模式，multi_wire_open 需要添加必要的 flag 如开启四线需要添加 **QUAD_WIRE_SUPPORT | MULTI_WIRE_SUPPORT**；开启两线需要添加 **DUAL_WIRE_SUPPORT | MULTI_WIRE_SUPPORT** 等

spd_info 需要填写 flash 支持的频率，默认填入 50000000 即可，同时需要将 delay 设置为 1；其中 delay 表示的是延后几个时钟周期进行采样，取决于不同的 flash 发送数据延迟时间 tclqv（datasheet 中可查看到）

t_{CLQV}	Clock Low To Output Valid			7	ns

图片 27: Tclqv

以及所焊接板子的走线过长过短影响数据传输及采样。本地测试 50M 频率输入 delay 填 1 最为稳定，如果有发读数据发生偏移问题，可以更改此处 sample_delay 来进行调试，不同的 flash 可能会有所区别，但是大部分在 1 的情况下都能正确读取数据。

- 步骤 2：确认读写操作命令与代码文件是否一致

通过 datasheet 查看 flash 命令，如图所示：

 版权所有 @ 上海富瀚微电子股份有限公司

Table 10. Commands

Command Name	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8	Byte 9
Write Enable	06H								
Write Disable	04H								
Read Status Register-1	05H	(S7-S0)	(cont.)						
Read Status Register-2	35H	(S15-S8)	(cont.)						
Read Status Register-3	15H	(S23-S16)	(cont.)						
Write Status Register-1	01H	S7-S0							
Write Status Register-2	31H	S15-S8							
Write Status Register-3	11H	S23-S16							
Volatile SR write Enable	50H								
Read Data	03H	A23-A16	A15-A8	A7-A0	(D7-D0)	(cont.)			
Fast Read	0BH	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)	(cont.)		
Dual Output Fast Read	3BH	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)(1)	(cont.)		
Quad Output Fast Read	6BH	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)(2)	(cont.)		
Dual I/O Fast Read	BBH	A23-A16(3)	A15-A8(3)	A7-A0(3)	M7-M0(4)	(D7-D0)(1)	(cont.)		
Quad I/O Fast Read	EBH	A23-A16(5)	A15-A8(5)	A7-A0(5)	M7-M0(6)	dummy	dummy	(D7-D0)(2)	(cont.)
Set Burst with Wrap	77H	dummy(7)	dummy(7)	dummy(7)	W7-W0(7)				
Page Program	02H	A23-A16	A15-A8	A7-A0	D7-D0	Next Byte			

图片 28: flash cmd

与相应的器件代码中宏定义部分的命令做相应修改，如下图所示：

```
/*commands*/
#define CMD_GD_WREN      0x06    /* Write Enable */
#define CMD_GD_WRDI      0x04    /* Write Disable */
#define CMD_GD_RDSR      0x05    /* Read Status Register */
#define CMD_GD_WRSR      0x01    /* Write Status Register */
#define CMD_GD_READ      0x03    /* Read Data Bytes */
#define CMD_GD_FAST_READ    0x0b    /* Read Data Bytes at Higher Speed */
#define CMD_GD_PP        0x02    /* Page Program */
#define CMD_GD_SE        0x20    /* Sector (4K) Erase */
#define CMD_GD_BE        0xd8    /* Block (64K) Erase */
#define CMD_GD_CE        0xc7    /* Chip Erase */
#define CMD_GD_DP        0xb9    /* Deep Power-down */
#define CMD_GD_RES       0xab    /* Release from DP, and Read Signature */

#define CMD_GD_RDSR2        0x35    /* Read Status Register 2 */
#define CMD_GD_WRENV        0x50    /* Volatile SR write enable */
#define CMD_GD_WRSR2        0x31    /* Write Status2 Register */
#define CMD_GD_DUAL_READ_3_ADR    0x3b
#define CMD_GD_QUAD_READ_3_ADR    0x6b
#define CMD_GD_DUAL_READ_4_ADR    0x3c
#define CMD_GD_QUAD_READ_4_ADR    0x6c
#define CMD_GD_QUAD_WRITE         0x32
```

图片 29: flash cmd 宏定义

- 步骤 3：确认 QE 使能设置

使能 QE 需要设置寄存器的特定比特位。这只能通过实际的器件手册来分析。
我们以 GD25Q128 为例讲解一下接口实现。在 gd.c 中，找到 flash_set_qe_bit 接口。

 版权所有 @ 上海富瀚微电子股份有限公司

```
static void flash_set_qe_bit(struct spi_flash *flash, u8 bit_value){
    struct spi_slave *spi = flash->spi;
    u8 status2;
    int ret;
    u8 cmd[4] = { 0xff, 0xff, 0xff, 0xff };
    ret = spi_claim_bus(flash->spi); 配置spi
    if (ret) {
        printf("SF: Unable to claim SPI bus\n");
        return ;
    }
    status2 = 0;
    /* read status 2 */
    ret = spi_flash_cmd(flash->spi, CMD_GD_RDSR2, &status2, 1); 发送读取qe寄
    if (ret < 0) {                                              存器命令
        debug("SF: Enabling Write failed\n");   tag1
        goto out;
    }
    printf("status register2: 0x%x\n",status2);

    status2 &= ~(1 << QE_POS); tag2
    status2 |= bit_value << QE_POS;
    ret = spi_flash_cmd(flash->spi, CMD_GD_WREN, NULL, 0); 发送写使能命令
    if (ret < 0) {
        printf("SF: Enabling Write failed\n");
        goto out;
    }
    cmd[0] = CMD_GD_WRSR2;
    cmd[1] = status2; tag3
    ret = spi_xfer(spi, 16, &cmd[0], NULL, SPI_XFER_BEGIN); 发送写qe寄存器命
    if (ret) {                                               令并写入寄存器值
        goto out;
    }
    spi_xfer(spi, 0, NULL, NULL, SPI_XFER_END);
    ret = gd_wait_ready(flash, SPI_FLASH_QE_TIMEOUT); 等待命令结束
    if (ret < 0) {
        debug("SF: Winbond page programming timed out\n");
        goto out;
    }
```

图片 30: qe 代码

需要注意如下几个 tag 位配置
tag1: 读 qe 所在寄存器命令
tag2:QE_POS 为寄存器实际需要设置 QE 为 1 的位置
tag3: 写 qe 所在寄存器命令

- 步骤 4：大于 16MB Flash 需要开启 4byte 模式

开启 4byte 模式与开启 qe 类似，需要从相应的 datasheet 中查询方法
我们以 GD25Q256 为例，需要发送命令开启 4byte 模式。
需要按以上步骤添加 GD25Q256 的 id，之后关注切换 4byte 模式的接口

```
static void spi_flash_reset_4byte_gd(struct spi_slave *spi, u8 four_byte)
{
    u8 idcode[5];
    if (four_byte) {
        spi_flash_cmd(spi, 0x35, idcode, 1);
        if (idcode[0] & 0x01)
            debug("flash already in 4bytes mode\n");
        else {
            spi_flash_cmd(spi, 0xb7, NULL, 0);
            debug("flash now in the 4bytes mode\n");
        }
    } else {
        spi_flash_cmd(spi, 0x35, idcode, 1);
        if (idcode[0] & 0x01) {
            debug("change flash to 3bit mode\n");
            spi_flash_cmd(spi, 0xe9, NULL, 0);
        } else
            debug("flash now in the 3bit mode\n");
    }
}
```

读取寄存器查看是否已经处于4byte模式

传参决定是否开启4byte模式

开启4byte模式命令

关闭4byte模式、开启3byet模式命令

图片 31: 4byte 模式讲解

其中查看寄存器命令、开启 4byte 模式命令、关闭 4byte 模式命令需要针对不同型号 flash 的 datasheet 进行适配。

- 步骤 5 : 最后确认各操作函数接口是否准确

在 gd.c 中找到 spi_flash_probe_gd 接口，接口内查看以下内容：

```
    stm->flash.write = gd_write;
    stm->flash.erase = gd_erase;
    stm->flash.read = gd_read_fast;
    stm->flash.protect = gd_protect;
    stm->flash.size = (u32)page_size * (u32)params->pages_per_sector
            * (u32)params->sectors_per_block
            * (u32)params->nr_blocks;
    stm->flash.sector_size = (u32)page_size * (u32)params->pages_per_sector;
    debug("SF: Detected %s with page size %u, total %u bytes\n",
          params->name, page_size, stm->flash.size);
```

图片 32: flash 函数接口

确认各操作与 flash 手册描述一致即可。

 版权所有 @ 上海富瀚微电子股份有限公司

- 步骤 6 : flash 型号开启宏定义

以 FH8856V200 为例，文件打开路径为：
uboot\include\configs\fh8856v200_devboard.h
找到 CONFIG_SPI_FLASH，在后面添加 Flash 开启宏定义，
例如 GDflash 则可命名为 CONFIG_SPI_FLASH_GD

```
/* SPIFlash */
#define CONFIG_SPI_FLASH
#ifdef CONFIG_SPI_FLASH
# define CONFIG_SPI_FLASH_ALL
# define CONFIG_SPI_FLASH_MACRONIX
# define CONFIG_SPI_FLASH_WINBOND
# define CONFIG_SPI_FLASH_BOYA
# define CONFIG_SPI_FLASH_DEFAULT
# define CONFIG_SPI_FLASH_GD
# define CONFIG_SPI_FLASH_XMC
# define CONFIG_SPI_FLASH_EON
# define CONFIG_SPI_FLASH_NM
# define CONFIG_SPI_FLASH_FM
# define CONFIG_SPI_FLASH_XTX
# define CONFIG_SPI_FLASH_ZB
```

图片 33: flash 宏定义

- 步骤 7 spi_flash.c 中添加 flash 注册入口

打开文件找到 spi_flash_probe 接口，按照如下格式添加 flash 注册入口

```
#ifdef CONFIG_SPI_FLASH_GD
    case 0xc8:
        flash = spi_flash_probe_gd(spi, idcode);
        break;
#endif

#ifdef CONFIG_SPI_FLASH_XMC
    case 0x20:
        flash = spi_flash_probe_xmc(spi, idcode);
        break;
#endif

#ifdef CONFIG_SPI_FLASH_EON
    case 0x1C:
        flash = spi_flash_probe_eon(spi, idcode);
        break;
#endif
```

图片 34: flash 注册函数

 版权所有 @ 上海富瀚微电子股份有限公司

所判断宏定义即为步骤 6 板级文件所添加的宏定义

- 步骤 8 : Makefile 添加新文件名

如果创建了新的器件文件，需要 Makefile 中添加文件名确保编译成功：

```
COBJS-$(CONFIG_SPI_FLASH)	+= spi_flash.o
#add for sf mtd
COBJS-$(CONFIG_SPI_FLASH_MTD) += sf_mtd.o
COBJS-$(CONFIG_SPI_FLASH_ATMEL)	+= atmel.o
COBJS-$(CONFIG_SPI_FLASH_MACRONIX)	+= macronix.o
COBJS-$(CONFIG_SPI_FLASH_SPANSION)	+= spansion.o
COBJS-$(CONFIG_SPI_FLASH_SST)	+= sst.o
COBJS-$(CONFIG_SPI_FLASH_STMICRO)	+= stmicro.o
COBJS-$(CONFIG_SPI_FLASH_WINBOND)	+= winbond.o
COBJS-$(CONFIG_SPI_M95XXX) += eeprom_m95xxx.o
COBJS-$(CONFIG_SPI_FLASH_BOYA)	+= boya.o
COBJS-$(CONFIG_SPI_FLASH_DEFAULT)	+= default_flash.o
COBJS-$(CONFIG_SPI_FLASH_GD)	+= gd.o
COBJS-$(CONFIG_SPI_FLASH_XMC)	+= xmc.o
COBJS-$(CONFIG_SPI_FLASH_EON)	+= eon.o
COBJS-$(CONFIG_SPI_FLASH_NM)	+= nm.o
COBJS-$(CONFIG_SPI_FLASH_FM)	+= fm.o
COBJS-$(CONFIG_SPI_FLASH_XTX)	+= xtx.o
COBJS-$(CONFIG_SPI_FLASH_ZB)	+= zb.o
ifdef CONFIG_SPL_BUILD
COBJS-$(CONFIG_SPL_SPI_SUPPORT)	+= spi_spl_load.o
endif
```

图片 35: Makefile 添加文件

之后重新编译 uboot，至此，一款 flash 在 uboot 下的添加就算完成。

15.1.2 SPI-Nor rtthread 下的移植

相关驱动路径

目前内核下驱动路径：
`rt-thread\drivers\flash`
路径下相关文件

```
spi_flash_sfud.c:flash 注册入口
sfud.c: 区分不同厂家的 flash 并进行初始化
sfud_flash_def.h: 存放自添加的 flashID 参数表格
```

如何新增一颗 SPI Nor Flash

在 sfud_flash_def.h 中找到 SFUD_FLASH_CHIP_TABLE 与 SFUD_FLASH_EXT_INFO_TABLE，所有支持的 flash 型号 id 与参数在此处进行添加。
以 GD25Q128 为例，讲解一下 SFUD_FLASH_CHIP_TABLE 中 id 添加与参数配置，找到 id 表格中 GD 所划分的地方：

```
{"GD25Q128B", SFUD_MF_ID_GIGADEVICE, 0x40, 0x18, 16L*1024L*1024L, SFUD_WM_PAGE_256B, 64L*1024L, 0xD8, 100000000},
```

图片 36: flash id 表

其中 INFO 的参数讲解为：

```
* | name | mf_id | type_id | capacity_id | capacity | write_mode | erase_gran | erase_gran_cmd | default_frequency |
```

图片 37: flash INFO

而在 SFUD_FLASH_EXT_INFO_TABLE 中的配置为：

```
/* GD25Q128B */                                                                    \
{SFUD_MF_ID_GIGADEVICE, 0x40, 0x18, NORMAL_SPI_READ|DUAL_OUTPUT|QUAD_OUTPUT},      \
```

图片 38: qe 参数表格

除了填写 id，后面的 NORMAL_SPI_READ|DUAL_OUTPUT|QUAD_OUTPUT 表示支持单线、双线、四线的读取模式，按照格式与 Flash datasheet 描述如实填写即可。
步骤 1：通过 9F 命令查询 ID
通过 flash 器件的 datasheet 找到对应 ID 与相关参数，按照格式填入表格。
步骤 2：确认函数接口
由于 sfud.c 下的读写擦函数接口均遵循标准协议，如无特殊器件，无需再核对读写擦接口，要确认的函数接口主要是 QE 使能接口与 RESET 接口。
1. 首先确认 QE 使能接口，可以找到 sfud.c 下 spi_nor_set_qe(const sfud_flash *flash, uint8_t flash_mf_id)。

 版权所有 @ 上海富瀚微电子股份有限公司

```c
sfud_err spi_nor_set_qe(const sfud_flash *flash, uint8_t flash_mf_id)
{
    int status = -1;

    switch (flash_mf_id)
    {
    case SFUD_MF_ID_MICRONIX:
        status = spi_nor_mx_set_qe(flash);
        if (status)
        {
            SFUD_INFO("Macronix quad-read not enabled");
            return status;
        }
        return status;
    case SFUD_MF_ID_WINBOND:
        status = spi_nor_wb_set_qe(flash);
        if (status)
        {
            SFUD_INFO("winbond quad-read not enabled");
            return status;
        }
        return status;
    case SFUD_MF_ID_GIGADEVICE:
        status = spi_nor_gd_set_qe(flash);
        if (status)
        {
            SFUD_INFO("GD quad-read not enabled");
            return status;
        }
        return status;
    case SFUD_MF_ID_XMC:
        if (flash->chip.type_id == 0x60)
        {
            status = spi_nor_xmc_set_qe(flash);
            if (status)
            {
                SFUD_INFO("XMC quad-read not enabled");
                return status;
            }
        }
        return status;
    default:
        return status;
        }
}
```

图片 39: rtt QE 使能接口

能看见针对不同的型号，QE 使能接口并不一致，其中 case 区分的为各厂家的 manufacture ID, 此宏定义在`rt-thread\drivers\flash\sfud\inc\sfud_flash_def.h`中定

义，如需添加新厂 Flash 器件，可以在此文件内添加厂家 ID，如下所示。

```
#define SFUD_MF_TABLE                                                    \
{                                                                        \
    {"Cypress",    SFUD_MF_ID_CYPRESS},                                  \
    {"Fujitsu",    SFUD_MF_ID_FUJITSU},                                  \
    {"EON",        SFUD_MF_ID_EON},                                      \
    {"Atmel",      SFUD_MF_ID_ATMEL},                                    \
    {"XMC",        SFUD_MF_ID_XMC},                                      \
    {"Micron",     SFUD_MF_ID_MICRON},                                   \
    {"AMIC",       SFUD_MF_ID_AMIC},                                     \
    {"Sanyo",      SFUD_MF_ID_SANYO},                                    \
    {"Intel",      SFUD_MF_ID_INTEL},                                    \
    {"ESMT",       SFUD_MF_ID_ESMT},                                     \
    {"Fudan",      SFUD_MF_ID_FUDAN},                                    \
    {"Hyundai",    SFUD_MF_ID_HYUNDAI},                                  \
    {"SST",        SFUD_MF_ID_SST},                                      \
    {"GigaDevice", SFUD_MF_ID_GIGADEVICE},                               \
    {"ISSI",       SFUD_MF_ID_ISSI},                                     \
    {"Winbond",    SFUD_MF_ID_WINBOND},                                  \
    {"Micronix",   SFUD_MF_ID_MICRONIX},                                 \
}
```

图片 40: Flash MFID 定义

对于不同的 flash QE 接口，需要进行确认与更新，主要区别在于 QE bit 的位置以及读写寄存器命令，在之前的 u-boot 下适配 QE 接口已有说明，便不再赘述。

2. 其次确认软复位接口，找到 reset_chip(struct spi_nor *nor, struct flash_info* info) 接口

 版权所有 @ 上海富瀚微电子股份有限公司

```c
sfud_err sfud_reset(const sfud_flash *flash)
{
    sfud_err result = SFUD_SUCCESS;
    const sfud_spi *spi = &flash->spi;
    uint8_t cmd_data[2];

    result = set_4_byte_address_mode((sfud_flash *)flash, false);
    if (result != SFUD_SUCCESS)
    {
        return result;
    }
    SFUD_ASSERT(flash);

    /* lock SPI */
    if (spi->lock)
        spi->lock(spi);

    result = wait_busy(flash, SFUD_STATUS_READ);

    if (result != SFUD_SUCCESS)
    {
        /* unlock SPI */
        if (spi->unlock)
            spi->unlock(spi);
        return result;
    }

    switch (flash->chip.mf_id)
    {
    case 0x9F: /* S25FL128/256S spansion */
        cmd_data[0] = 0xff;
        spi->wr(spi, cmd_data, 1, NULL, 0);
        cmd_data[1] = 0xf0;
        spi->wr(spi, &cmd_data[1], 1, NULL, 0);
        break;
    case 0xef:/*winbond*/
    case 0xc8:/*GD*/
        cmd_data[0] = 0x66;
        spi->wr(spi, cmd_data, 1, NULL, 0);
        cmd_data[1] = 0x99;
        spi->wr(spi, &cmd_data[1], 1, NULL, 0);
        fh_udelay(100);
        break;
    case 0xc2:
    case 0x20: /* Micron, actually */
    default:
        break;
    }
        /* unlock SPI */
    if (spi->unlock)
        spi->unlock(spi);

    return SFUD_SUCCESS;

}
```

图片 41: rtt flash reset

按照 datasheet 进行 reset 命令配置，如 GD25Q128 描述为

Enable Reset	66H								
Reset	99H								

图片 42: gd25q128 reset cmd

即如代码所示先发送 66H，之后发送 99H，不同 flash 会有差异，请确认后进行配置。至此，rtt 下新增一颗 norflash 完成，需要重新编译 bin 文件。

15.2 SPI Nand Flash 器件的移植

15.2.1 SPI-Nand Uboot 下的移植

相关驱动路径

SPI Nand 驱动的路径：
`bsp\uboot\drivers\mtd\spi-nand`
路径下与移植相关的文件

```
spi-nand-id.c:SPI Nand 器件 ID 表，移植 SPI Nand 器件主要修改的文件
```

如何新增一颗 SPI Nand Flash

首先进入 spi-nand-id.c 文件中，然后查看 id 表各成员，定义在
`uboot\drivers\mtd\spi-nand\spi-nand.h`中

```
struct spi_nand_flash {
    char        *name;
    struct spi_nand_id_info id_info;
    u8      dev_id[SPINAND_MAX_ID_LEN];
    u32     page_size;
    u32     page_spare_size;
    u32     pages_per_blk;
    u32     blks_per_chip;
    u32     options;
    u8      ecc_mask;
    u8      ecc_uncorr;
    struct nand_ecclayout *ecc_layout;
};
```

图片 43: Nandflashid 参数

 版权所有 @ 上海富瀚微电子股份有限公司

之后可以看到表中实际填写时的参数，我们以 XT26G01C 为例添加一款 NandFlash

```
{
    .name = "XT26G01C",
    .id_info = {
        .id_addr = 0,
        .id_len = 2,
    },
    .dev_id = {0x0b, 0x11},
    .page_size = 2048,
    .page_spare_size = 128,
    .pages_per_blk = 64,
    .blks_per_chip = 1024,
    .options = 0,
    .ecc_mask = 15,
    .ecc_uncorr = 15,
    .ecc_layout = &xt_ecc_layout_128,
},
```

图片 44: Nandflash 添加示例

- id_info、dev_id: 其中 id 通过 datasheet 中查询 9FH 命令找到为 0x0B11

Address	Value	Description
Byte 0	0BH	Manufacture ID (XTX)
Byte 1	11H	Device ID (SPI NAND 3.3V 1Gbit)

图片 45: Nandflash id

- page_size、page_spare_size、page_per_blk、blks_per_chip: 容量大小通过查询 array organization 找到

Table1. Array Organization

Each device has	Each block has	Each page has	Unit
128M + 8M	128K + 8K	2K + 128	bytes
1024 x 64	64	-	Pages
1024	-	-	Blocks

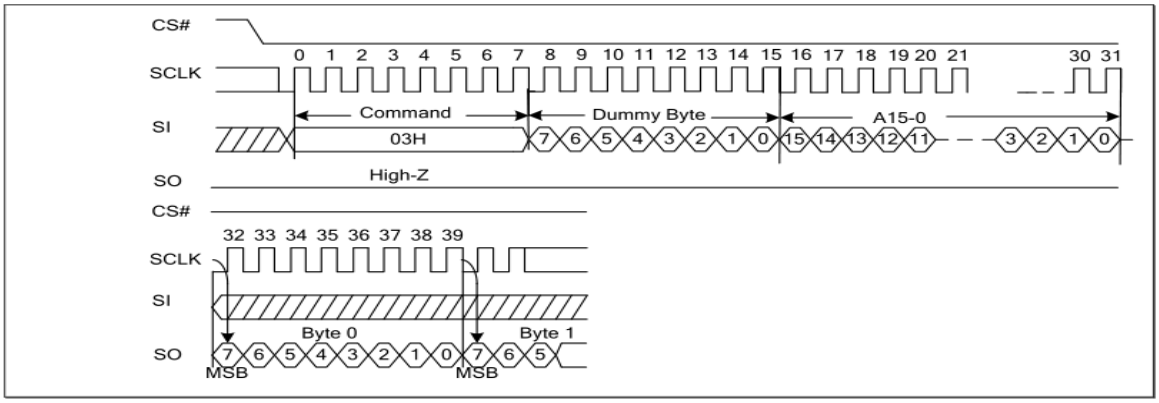
图片 46: Nandflash array

- option: 添加特殊选项，一般设置为 0 即可；特殊选项包括

 版权所有 @ 上海富瀚微电子股份有限公司

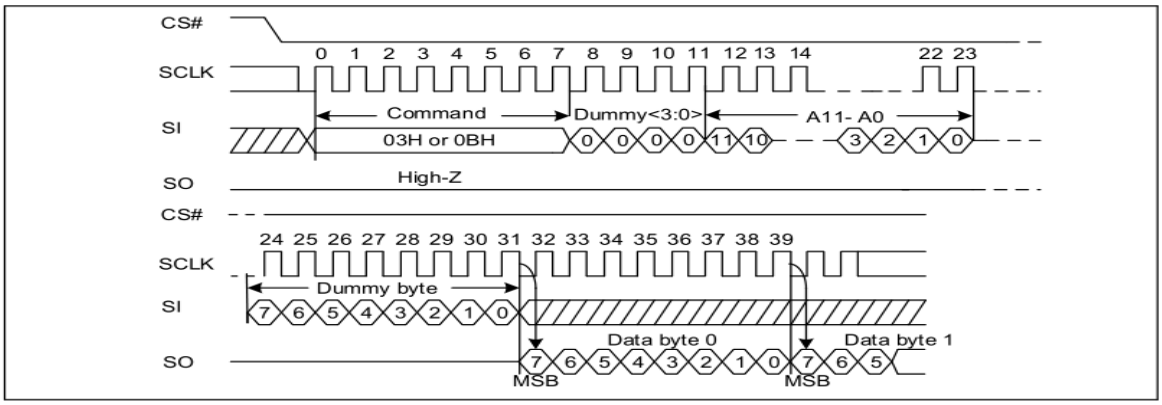
FIRST_DUMMY_BYTE：即某些器件进行读写时与标准协议 dummybyte 放 address 之后不一致，是需要放在 address 之前，这时需要设置此选项，驱动内会进行特殊处理

比如在 GD5FxGQ4xC 这款 flash 的 datasheet 描述中，找到 03H 命令也就是 ReadFromCache 命令，可以看到波形图如下：



图片 47: 前置 dummybyte

即 dummybyte 在前。而在新款的 GD5F1GQ5UExxG 中，波形如下：



图片 48: 后置 dummybyte

现在一般标准协议都是后置 dummybyte, 如遇到特殊器件，则通过添加此 option 位进行区分。

SPINAND_NEED_PLANE_SELECT：某些厂商会将一颗 Nand 分成 2 个 plane,2 个 plane 里的 block 是交叉编号，此时需要此选项来区分，驱动内会进行特殊处理

- ecc_mask、ecc_uncorr: 此两项表示读取寄存器 ecc 状态时的掩码和判断是否发生 ecc 错误的标志，通过查看寄存器说明来配置

 版权所有 @ 上海富瀚微电子股份有限公司

Register	Address	Data Bits							
		7	6	5	4	3	2	1	0
Block Lock	A0H	BRWD	Reserved	BP2	BP1	BP0	INV	CMP	Reserved
Feature	B0H	OTP_PRT	OTP_EN	Reserved	ECC_EN	Reserved	Reserved	Reserved	QE
Status	C0H	ECCS3	ECCS2	ECCS1	ECCS0	P_FAIL	E_FAIL	WEL	OIP
Drive Strength	D0H	Reserved	DS_IO[1]	DS_IO[0]	Reserved	Reserved	Reserved	Reserved	Reserved

图片 49: Nandflash reg

这是基本的寄存器布局，可以看到与 ecc 相关的 bit 主要在 c0 与 b0 寄存器内，其中 b0H 的 ECC_EN bit 表示是否开启 Flash 内部的 ecc 功能；c0H 的 ECCS0-ECCS3 表示的是内部进行 ecc 校正后的结果，会放置在这四个 bit，我们通过这 4 个 bit 来获取内部 ecc 状态。首先 ecc_mask 表示的就是 ECCS0-ECCS3 的掩码，因为驱动内会主动右移四位，所以如果有 4 个 bit 来显示 ecc 状态，则 ecc_mask 填 0xf 即 15 便可；如果有 3 个 bit 来显示 ecc 状态，则 ecc_mask 填 0x7 即 7 便可；如果有 2 个 bit 来显示来显示 ecc 状态，则 ecc_mask 填 0x3 即 3 便可。这需要根据不同型号的 flash 提供的 datasheet 来确认，比如 GD5F1GQ5UExxG 中的寄存器如下显示

Status	C0H	Reserved	Reserved	ECCS1	ECCS0	P_FAIL	E_FAIL	WEL	OIP

图片 50: GDNand reg

只有两个 bit 来显示 ecc 状态，所以 ecc_mask 填 0x3 即 3 便可。
而 ecc_uncorr 所需要填的值为内部寄存器显示 ecc 错误时即出现内部 ecc 都无法纠正的时候，寄存器的状态。也需要通过 datasheet 查询，如下所示

 版权所有 @ 上海富瀚微电子股份有限公司

ECCS3 ECCS2 ECCS1 ECCS0	ECC Status	ECCS provides ECC Status as follows: 0000b = No bit errors were detected during the previous read algorithm. 0001b = bit errors were detected and corrected, error bit number = 1. 0010b = bit errors were detected and corrected, error bit number = 2. 0011b = bit errors were detected and corrected, error bit number = 3. 0100b = bit errors were detected and corrected, error bit number = 4. 0101b = bit errors were detected and corrected, error bit number = 5. 0110b = bit errors were detected and corrected, error bit number = 6. 0111b = bit errors were detected and corrected, error bit number = 7. 1000b = bit errors were detected and corrected, error bit number = 8. 1111b = Bit errors greater than ECC capability(8 bits) and not corrected. Bit errors cannot be detected and corrected if their number exceeds the tolerance. Therefore, block data should be refreshed when ECC status is equal to 1111b.

图片 51: Nandflash ecc statue

根据描述我们能知道当发生 flash 内部无法纠正的 ecc 错误即超过 8bit 的错误时，这 4 个 bit 的值为 1111b，所以 ecc_uncorr 填入的就是 1111b 即十进制的 15。

- ecc_layout: 此参数用来显示 flash 内部 spare 区域的分布，同样在 data sheet 内搜索 ECC Protection 能获取到 spare 区域分布

Min Byte Address	Max Byte Address	ECC Protected	Number Of Bytes	Area	Description
000H	1FFH	Yes	512	Main 0	User data 0
200H	3FFH	Yes	512	Main 1	User data 1
400H	5FFH	Yes	512	Main 2	User data 2
600H	7FFH	Yes	512	Main 3	User data 3
800H	80FH	Yes	16	Spare 0	User meta data 0
810H	81FH	Yes	16	Spare 1	User meta data 1
820H	82FH	Yes	16	Spare 2	User meta data 2
830H	83FH	Yes	16	Spare 3	User meta data 3
840H	873H	Yes	52	Spare 4	Internal ECC parity data
874H	87FH	NO	12	Spare 5	User meta data 4

图片 52: Nandflash ECC layout

按照此分布，我们建立一个 struct nand_ecclayout xt_ecc_layou_128 的结构体对象

 版权所有 @ 上海富瀚微电子股份有限公司

```
static struct nand_ecclayout xt_ecc_layout_128 = {
    .eccbytes = 52,
    .eccpos = {
        64, 65, 66, 67, 68, 69, 70, 72,
        72, 73, 74, 75, 76, 77, 78, 79,
        80, 81, 82, 83, 84, 85, 86, 87,
        88, 89, 90, 91, 92, 93, 94, 95,
        96, 97, 98, 99, 100, 101, 102, 103,
        104, 105, 106, 107, 108, 109, 110, 111,
        112, 113, 114, 115,
    },
    .oobavail = 74,
    .oobfree = { {2, 62}, {116, 12} }
};
```

图片 53: Nandflash ECC layout

其中 eccbyte 是 ecc 码的个数，按 datasheet 所示填入 52；eccpos 填入具体存放 ecc 码的位置；oobabail 存放 oob 区域除去 ecc 码和坏块标识位（一般两个 byte）之后的空闲空间，此款是 128-52-2=74byte;oobfree 填入空闲空间的具体位置，通过 {offset,length} 的格式按照实际 datasheet 所述填入即可。
至此，uboot 下一款 Nandflash 的添加便完成，重新编译 uboot 进行测试与确认。

15.2.2 SPI Nand Flash rtthread 下的移植

相关驱动路径

SPI Nand 驱动的路径：
`rt-thread\drivers\nand\src\spi-nand-ids.c`
路径下与移植相关的文件

```
spi-nand-id.c:SPI Nand 器件 ID 表，移植 SPI Nand 器件主要修改的文件
```

如何新增一颗 SPI Nand Flash

首先进入 spi-nand-id.c 文件中，然后查看 id 表各成员，定义在
`drivers\nand\inc\spi-nand.h`中

```
struct spi_nand_flash {
	char		*name;
	struct spi_nand_id_info id_info;
	u8		dev_id[SPINAND_MAX_ID_LEN];
	u32		page_size;
	u32		page_spare_size;
	u32		pages_per_blk;
	u32		blks_per_chip;
	u32		options;
	u8		ecc_mask;
	u8		ecc_uncorr;
	struct nand_ecclayout *ecc_layout;
	u32		qe_addr;
	u32		qe_flag;
	u32		qe_mask;
	u32		multi_wire_command_length;
	u32		bbt_options;
	u32		bitflip_threshold;
};
```

图片 54: Nandflash 参数

与 uboot 中十分类似，但是有些许差异，以下以 XT26G01C 为例讲解一下我们在 id 表中添加此款 flash 的参数配置

```
	{
		.name = "GD5F1GQ5UE",
		.id_info = {
			.id_addr = 0,
			.id_len = 2,
		},
		.dev_id = {0xC8, 0x51},
		.page_size = 2048,
		.page_spare_size = 128,
		.pages_per_blk = 64,
		.blks_per_chip = 1024,
		.options = 0,
		.ecc_mask = 3,
		.ecc_uncorr = 2,
		.ecc_layout = &fs35_ecc_layout_64,
		.qe_addr = 0xb0,
		.qe_flag = 0x1,
		.qe_mask = 1<<0,
		.multi_wire_command_length = 4,
		},
```

图片 55: Nandflash id

 版权所有 @ 上海富瀚微电子股份有限公司

其中大部分参数已经在前文描述过，便不再赘述，部分新参数介绍一下

- qe_addr:qe_addr 指示的是在哪个寄存器进行 qe 的配置
- qe_mask:qe_mask 指示的是哪个 bit 位来使能 qe

如下所示，是在 B0H 的第 0bit 设置 QE。

Register	Address	Data Bits							
		7	6	5	4	3	2	1	0
Block Lock	A0H	BRWD	Reserved	BP2	BP1	BP0	INV	CMP	Reserved
Feature	B0H	OTP_PRT	OTP_EN	Reserved	ECC_EN	Reserved	Reserved	Reserved	QE
Status	C0H	ECCS3	ECCS2	ECCS1	ECCS0	P_FAIL	E_FAIL	WEL	OIP
Drive Strength	D0H	Reserved	DS_IO[1]	DS_IO[0]	Reserved	Reserved	Reserved	Reserved	Reserved

图片 56: Nandflash reg

- qe_flag：表示是否将此 bit 位置 1
- multi_wire_command_length: 指示多线操作时指令长度

至此，rtt 下的 nandflash 添加便完成，需要重新编译生成 bin 文件。

 版权所有 @ 上海富瀚微电子股份有限公司

16 RTOS 调试指南

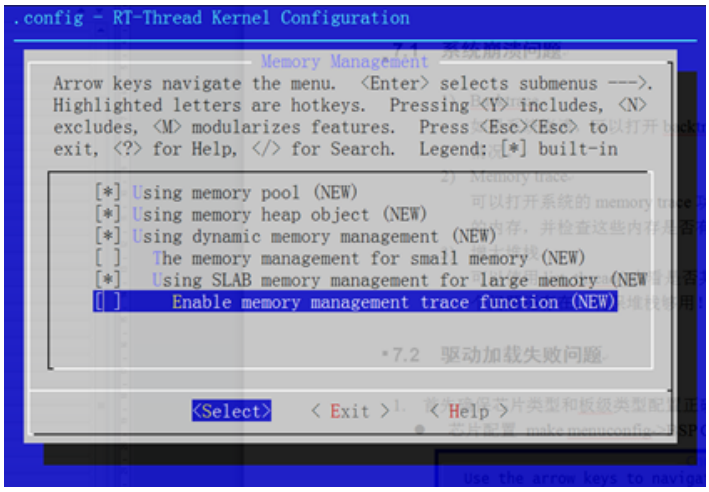
RTOS 的开发调试相对基于 Linux 的开发要难一些，主要源于在 RTOS 上，所有进程共用一个地址空间，进程之间没有隔离，此时，对于非法指针的使用，非常容易导致系统工作异常。

本节简要介绍一下在 RTOS 上面进行程序开发的常用调试手段。

16.1 内存跟踪

我们在进行程序开发时，经常由于非法指针的使用，导致线程 A 异常访问了（主要是写入了）其它线程（B）的数据，从而导致线程 B 出现工作异常，为跟踪系统内存的非法使用，可以通过内存跟踪方法，及时发现系统中的异常，从而缩小问题范围，便于定位并解决问题。

内存跟踪是系统可配置选项，默认不开启该功能，需要使用时，首先需要在 menuconfig 中开启该功能，开启方法见



图片 57: 开启内存跟踪

默认情况下，当系统出现异常导致程序崩溃时，会对系统已分配内存进行跟踪，并打印出现异常的内存单元。

我们也可以主动对内存单元进行检查，以便及时发现系统的内存异常；此时，我们需要在可疑代码处调用如下接口：

```
void ptck(char *file, int line);
```

其中，

1. 参数 file 表示检查点的文件名称
2. line 表示检查点所在的代码行数

开启了内存跟踪后，如果检查到内存的踩踏，则会打印类似如下的字串：

```
rt_mm_check: bad memory node(0xa0100210), magic1=0x339dbfac magic2=0
    ↪ x339dbfac,alloc_len=64, caller_fn=func1<--func2<--func3
```

其中，

1. bad memory node(xxxx) 标识了异常的内存节点
2. magic1/2 分别标识内存节点的头部与尾部，如果出现异常，通常这 2 个数值会被修改，可以据此判断是整个内存块被破坏，或者是仅头部/尾部被破坏。
3. alloc_len 标识该内存块的长度
4. caller_fn 标识该内存块的调用关系

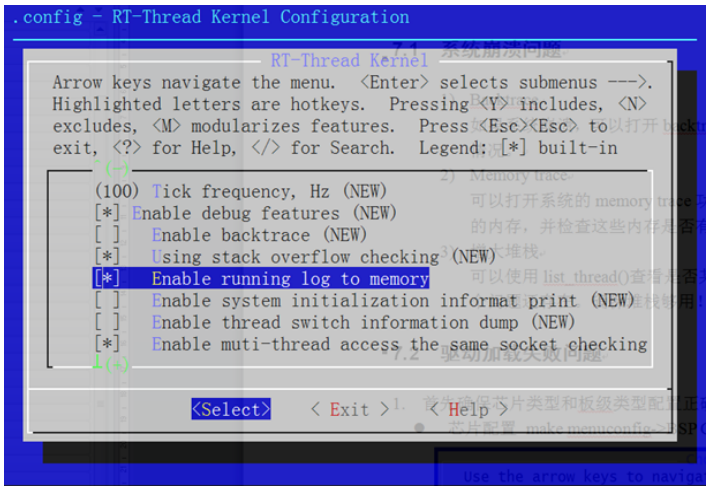
开发人员可以根据打印信息来判定故障原因。

注意，这里的打印信息不能明确定位故障点，该信息仅用于协助开发人员定位问题，出现内存踩踏的真实原因还需要开发人员具体分析和定位。

16.2 运行日志

运行日志，是指记录一段时间内的代码运行路径，包含线程切换信息，中断等；开发人员根据运行日志可以知道某一段时间内有哪些线程在运行，运行的先后顺序关系是什么，从而可以找到可能是哪些线程/中断的执行影响了本线程的执行逻辑。

运行日志功能也是可配置选项，默认关闭，可以通过 menuconfig 进行开启。开启方法见



图片 58: 开启运行日志

运行日志功能开启后，还需要为运行日志分配内存，以保存运行时的日志数据：

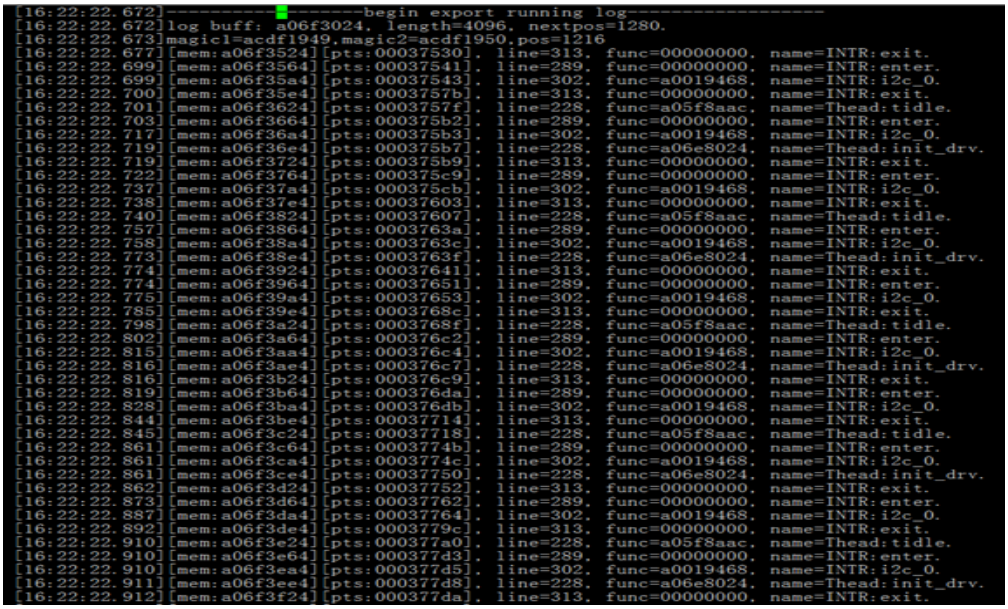
```
void runlog_init(int size);
```

其中，size 表示需要分配的内存大小，即使用多大的内存空间来保存运行日志。

大的内存空间可以保存更多的运行日志信息，可以有效防止重要信息的丢失，但对内存需求高，并且由于数据量比较大，对后续问题的查找带来挑战。较小的内存空间则对系统需求比较低，但是，有可能错过异常数据。一般使用 64KB~1MB 内存大小可以满足绝大多数应用程序的需求。

当开启运行日志功能后，当由于出错导致程序崩溃时，会在串口输出运行日志，当日志比较多时，打印需要一些时间，请耐心等待打印完成。

打印信息如下面图形显示。



图片 59: 开启运行日志

运行日志同样也不能精确定位问题，只能缩小问题的查找范围；在确定问题的影响因素后，也可以进一步自行增加运行日志，以达到进一步缩问题查找范围的目的。

自行增加 runlog 接口如下：

```
void runlog_add(int line, char *file, char *name1, char *name2);
```

参数定义如下：

1. line，行号，标识日志运行的所在行
2. file，文件名称，标识该日志所在的文件
3. name1/name2，自定义标识

也可以在必要时，强行输出运行日志：

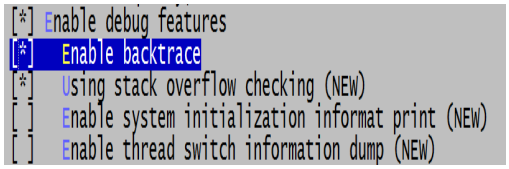
 版权所有 @ 上海富瀚微电子股份有限公司

```
void runlog_export();
```

16.3 栈回溯

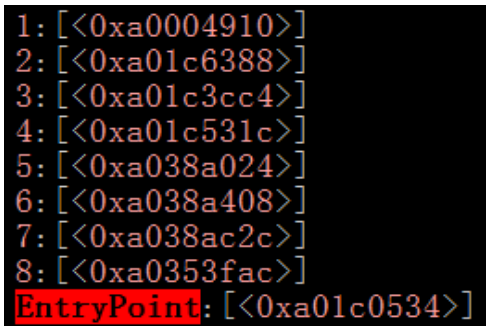
栈回溯功能用于在系统出错时，查看栈的调用情况，从而确定问题所在。

栈回溯功能默认也是关闭的，需要用户自行开启。



图片 60: 开启栈回溯

当开启栈回溯功能后，系统出现崩溃时，会打印出异常线程的函数调用关系，如图所示：



图片 61: 开启栈回溯

此时，结合运行代码的符号表，可以方便查找出工作异常的函数；此时如果还能有准确的 CPU 寄存器输出，一般都可以精确定位出出错原因，进而可以分析出异常的根因。

另外，在开启栈回溯功能后，此时查看进程，可以看到各进程的当前栈回溯情况，即，可以看到所有进程的当前函数调用关系。

 版权所有 @ 上海富瀚微电子股份有限公司

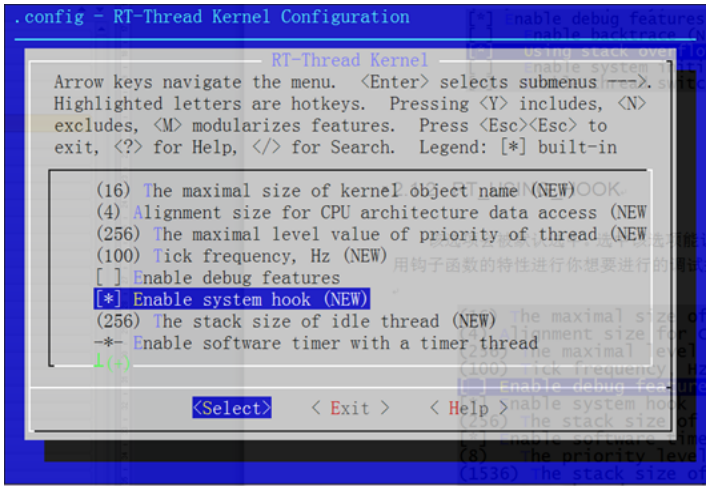
```
pri  status   max used  error  function
---  -------  ------   ---    --------
 70  suspend    35%     000    a0007704<-a0009644<-a0213e94<-a032972c<-a01c0534<-
 50  suspend    11%     000    a0007704<-a0002e04<-a01e1d30<-a01e9be4<-a01da838<-a02e59ec<-a01c0534<-
 50  suspend    37%     000    a0007704<-a0002e04<-a01e1d30<-a01e9be4<-a01da838<-a02e8fc8<-a02e0d74<-a(
 80  suspend    18%     000    a0007704<-a0002ad4<-a01c0278<-a03f88c4<-a01c0534<-
192  suspend    02%     000    a0007704<-a0009644<-a02ac0fc<-a01c0534<-
 50  suspend    38%     000    a0007704<-a0009644<-a02ed45c<-a01c0534<-
 60  suspend    69%     000    a0007704<-a0002ad4<-a01c0278<-a030b8a8<-a03f6110<-a03f6a4c<-a01c0534<-
 80  suspend    07%     000    a0007704<-a00024a8<-a01e234c<-a01e2e48<-a01e7ea0<-a01dabc8<-a03db740<-a(
 50  suspend    03%     000    a0007704<-a0009644<-a03edab8<-a01c0534<-
192  suspend    32%     000    a0007704<-a0009644<-a0213e94<-a030a928<-a01c0534<-
 50  suspend    24%     000    a0007704<-a0002e04<-a01e1d30<-a01e9be4<-a01da838<-a02ee37c<-a01c0534<-
 50  suspend    10%     000    a0007704<-a0009644<-a02ee0e0<-a01c0534<-
192  suspend    19%     000    a0007704<-a00031f8<-a01c68d0<-a01c3718<-a01c5190<-a0306884<-a0306940<-a(

 50  suspend    87%     000    a0007704<-a0009644<-a02ecfdc<-a01c0534<-^M
 60  suspend    22%     000    a0007704<-a0002e04<-a01c2cb4<-a030bad8<-a020da24<-a01c0534<-
 60  suspend    21%     000    a0007704<-a0002ad4<-a01c0330<-a030b8a8<-a02b4194<-a01c0534<-
 60  suspend    55%     000    a0007704<-a0002ad4<-a01c0330<-a030b8a8<-a02b374c<-a02b4a1c<-a01c0534<-
192  suspend    32%     000    a0007704<-a0009644<-a0213e94<-a0322c8c<-a01c0534<-
192  suspend    29%     000    a0007704<-a0009644<-a0213e94<-a03cbc74<-a03cb400<-a03cbadc<-a01c0534<-
192  suspend    26%     000    a0007704<-a0009644<-a0213e94<-a03cbc74<-a03cb3a8<-a03cbadc<-a01c0534<-
192  suspend    11%     000    a0007704<-a0009644<-a035d750<-a01c0534<-
 50  suspend    88%     000    a0007704<-a0009644<-a0213e94<-a032a34c<-a01c0534<-
```

图片 62: 开启栈回溯

16.4 系统钩子

系统钩子是 RTT OS 提供的系统级功能，用户通过注册一些回调函数，这些函数可以在系统特定的节点被调用，开发人员可以借此了解当前系统的工作情况，并辅助调试应用代码。

系统钩子功能开启方式如下：



图片 63: 开启系统钩子

一个实际的应用案例就是 top 功能，用户可以查看当前系统的负载情况，目前的 RTOS 系统的中的 top 功能即是通过钩子来实现的。

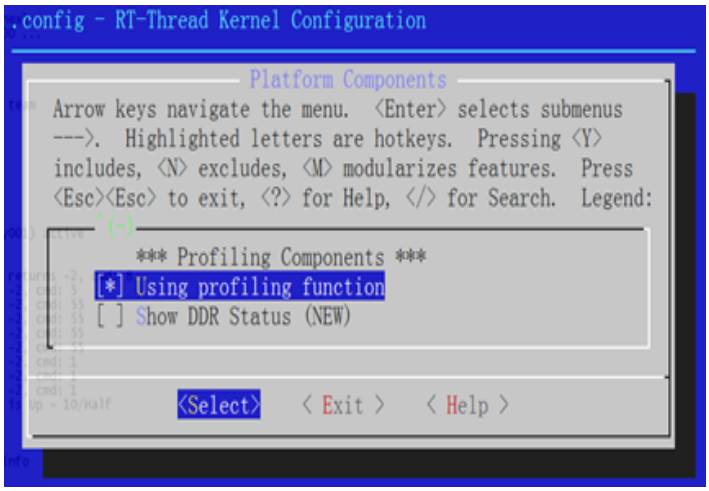
```
finsh />prof_init
--function:
prof_init        -- initialize thread profiling
finsh />prof_init()
        'x', -1609629576, 0xa00f0078
finsh />prof_view()
thread           pri      enter    tick(ms)    pct.
------           -------  -------  -------     ---
tshell           20       16       0           0
mmcsd_detect     64       0        0           0
etx              126      0        0           0
erx              126      0        0           0
tcpip            100      73       1           0
vpu_buff_thrd    8        8        0           0
ipf              9        8        0           0
vpu_task         9        8        0           0
tidle            255      97       7129        99
timer            8        0        0           0
        '#', 35, 0x00000023
finsh />
```

图片 64: 系统钩子应用实例

注：

1. top 功能指的 Linux 命令：top
2. 在 finsh 下面使用时，需要先调用 prof_init
3. finsh 下面，查看系统负载则使用 prof_view
4. SDK 中，msh 下提供 linux 同名命令 top

top 功能默认开启，即，用户可以在命令行下执行 top 查看系统负载；如果该功能不能正常使用，请确认该功能已经被正确开启。



图片 65: 系统钩子开启

根据系统负载，用户可以查看当前系统的负载，如果系统负载长时间维持在 99% 以上，开发人员需要对系统进行优化，避免部分模块因为不能得到及时调度而出现工作异常。

 版权所有 @ 上海富瀚微电子股份有限公司

16.5 栈溢出检查

栈溢出检查是 RTT 系统提供的功能，用于动态监测线程栈是否有溢出，便于及时发现系统中的异常。该功能默认关闭，开启方式如下。

```
[*] Enable debug features
[ ]   Enable backtrace (NEW)
[*]   Using stack overflow checking (NEW)
[ ]   Enable system initialization informat print (NEW)
[ ]   Enable thread switch information dump (NEW)
```

图片 66: 栈溢出监测开启

开启该功能后，每次进行线程切换时，都会检查待切入线程栈的溢出情况，如果有溢出，则报告错误。

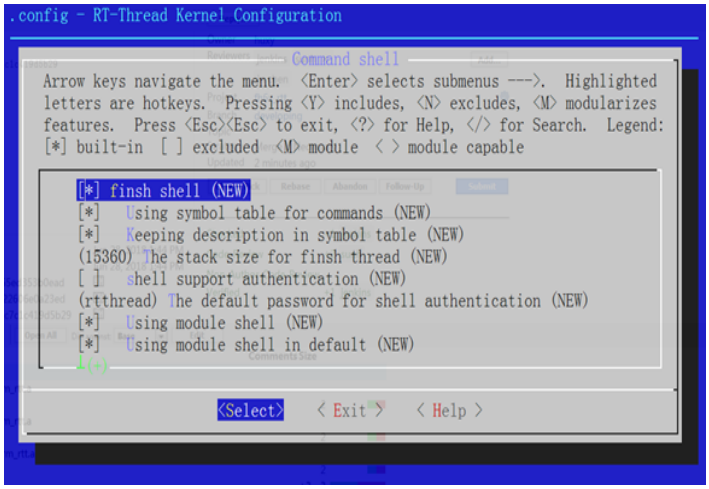
```
PHY: - Auto Negotiation is ON, Link is Up - 10/Half
thread:init_drv stack overflow
thread        pri  status      sp     stack size max used left tick  error
------------- ---  ------- ---------- ----------  ------  ---------- ---
tshell         20  suspend 0x000000c4 0x00003c00    01%   0x0000000a 000
mmcsd_detect   64  suspend 0x000000c0 0x00000400    64%   0x00000010 000
etx           126  suspend 0x000000bc 0x00000400    18%   0x00000010 000
erx           126  suspend 0x000000bc 0x00000400    42%   0x00000010 000
tcpip         100  suspend 0x00000118 0x00002000    06%   0x00000014 000
vpu_buff_thrd   8  suspend 0x00000230 0x00000c00    18%   0x0000000a 000
ipf             9  suspend 0x000000a0 0x00001000    03%   0x00000014 000
vpu_task        9  suspend 0x000000d0 0x00001000    05%   0x00000014 000
tidle         255  ready   0x0000005c 0x00000400    16%   0x00000020 000
timer           8  suspend 0x00000078 0x00000600    07%   0x0000000a 000
init_drv       80  ready   0x000100c8 0x00008000    04%   0x00000004 -02
```

图片 67: 栈溢出监测开启

16.6 自定义命令

在系统运行中，我们也可以通过自定义命令，来实时监测特定的数据结构/系统工作状态等，此时需要开启命令行支持。

 版权所有 @ 上海富瀚微电子股份有限公司



图片 68: 命令行支持

需要注意的是 RTT 系统支持 2 种命令行 finsh/msh，在 RTOS SDKv3.0 之后系统默认使用 msh，且不开启 finsh（finsh 为函数式编程，msh 为命令式编程，详细情况请参考 RTT 官方文档）

命令导出方式如下：

```
finsh_function_export(user_defined_finsh_function, function
    ↪ description);
msh_cmd_export(user_defined_msh_function, function description);
```

我们自定义显示内存和更改内存的示例如下：

```
finsh />disp_memory(0xa0060000,1)

Memory info:[a0060000,+1]
a0060000: e2422001                                  . B.
```

```
finsh />disp_memory(0xf0000000,4)

Memory info:[f0000000,+4]
f0000000: 46488302 00008302 00000000 07000071 ..HF.......q...
```

```
finsh />set_memory(0xf000000c,0x0f000071)
```

图片 69: 内存命令

 版权所有 @ 上海富瀚微电子股份有限公司

16.7 如何 debug 内存破坏问题，以及各种随机 exception 问题?

1) make menuconfig, 打开
   RT_ENABLE_RUNNING_LOG
   RT_USING_BACKTRACE //必开, 调试很重要
   RT_USING_MM_TRACE //必开, 调试很重要
   RT_USING_OVERFLOW_CHECK

   另外, 要是内存指针到处乱飞, 各个模块相互影响, 非常难判断
   到底哪个模块才是元凶, 建议各个模块使用各自独立的内存.
   接下来的一节讲如何划分内存 pool.

2) 内存池的划分
   在文件 slab_config.h 中,
   数组 g_mem_pool_config 控制分多少个 pool 及各个 pool 的大小。

   文件 slab_customer.c，里面是各个 pool 内存的分配函数，
   各个模块需要把内存分配 -释放函数替换为该文件里面对应的函数。

   举例来说，如何为 lwip 协议栈单独分配一个 pool 呢，可以这样改，
   static rt_uint32_t g_mem_pool_config[] = {
   0, //pool 0 memory size in bytes, 使用剩余内存, 配置为 0 即可
   0x100000, //pool 1 memory size in bytes,lwip 使用该 pool, 大小为 1MB
   };

   static rt_uint32_t g_mem_pool_map[FHMEM_TYPE_MAX] = {
   0, //pool id of FHMEM_TYPE_KERNEL
   1, //pool id of FHMEM_TYPE_NET，for lwip
   0, //pool id of FHMEM_TYPE_JFFS2
   0, //pool id of FHMEM_TYPE_APP
   0, //pool id of FHMEM_TYPE_RESERV1
   0, //pool id of FHMEM_TYPE_RESERV2
   0, //pool id of FHMEM_TYPE_RESERV3
   };

```
注意：一个pool不能小于64KB，否则会死机。
另外，文件net/lwip-2.1.2/src/arch/sys_arch.c中，

rt_free替换成lwip_free
rt_malloc替换成lwip_malloc
```

```
void *mem_malloc(mem_size_t size)
{
    return rt_malloc(size);
}
void mem_free(void *mem)
{
    rt_free(mem);
}

3) 如何解决double free问题？
   如果有内存重复释放的问题，通常有如下打印，
   rt_kprintf("#FHMM:invalid pointer %p!\n", ptr);
   这时候，就需要把某个内存池的分配释放历史记录下来，
   文件slab_config.h修改如下，

   #define RT_USING_MEM_FOOT //打开这个宏
   #ifdef RT_USING_MEM_FOOT
   #define MEM_FOOT_MM_SIZE (2*1024*1024) /*用于跟踪的内存大小*/
   static rt_uint32_t g_mem_pool_foot[] = {
          0, //pool 0不跟踪
          1, //表示跟踪pool 1
   };
   #endif

   等double free出现的时候，会自动把出问题的地址处的malloc/free
      ↪ 历史打印出来，
   并且有backtrace.

4) 如何解决指针乱飞，以及各种越界访问问题？
   文件slab_config.h中，打开宏RT_USING_MEM_ASAN_CHECK，
   并且把ASAN_CHECK_POOL_ID定义为你出问题的那个pool id.

   在需要ASAN检查的目录增加以下编译选项，注意不要整个工程加，
   LOCAL_CFLAGS += -fsanitize=kernel-address

   build/config.mk末尾加上
      LDFLAGS +=-Wl,--wrap,memset
      LDFLAGS +=-Wl,--wrap,memmove
```

```
LDFLAGS +=-Wl,--wrap,memcpy
LDFLAGS +=-Wl,--wrap,strcat
LDFLAGS +=-Wl,--wrap,strcpy
LDFLAGS +=-Wl,--wrap,strncat
LDFLAGS +=-Wl,--wrap,strncpy
```

这样，如果有内存越界访问的情况，会第一时间被抓住，并伴随着报错以及 backtrace.

 版权所有 @ 上海富瀚微电子股份有限公司

17 附录开发调试工具

工具位于:

```
SDK/docs_tools/software/pc
```

17.1 视频开发调试 CoolView 工具

将 rar 文件解压，双击 Release/CoolViewPro.exe 即可运行。

17.1.1 连接开发板

1. 点击图标按钮 ⚙，或者按键盘 F10
2. 在打开的对话框中，可以选择 ``**串口**" 或 ``**网络**"，一般选择 ``**网络**"
3. ``**本机 IP 地址**"：从下拉列表中选择实际的电脑 IP
4. ``**远程 IP 地址**"：修改为开发板的 IP 地址
5. 修改 ``**远程端口号**"，和开发板上应用程序运行 debug interface 的端口号保持一致
6. 如果开发板上应用程序运行 debug interface 使用的是 TCP 协议，则需要勾选 ``**启用 TCP**"
7. 点击 ``**保存**" 按钮，连接成功后，窗口右下角的状态栏会显示绿色的连接成功的文字。

17.1.2 修改 ISP 配置

连接成功后，在左侧 ``**详细列表**" 中选择功能模块，在中间标签页中进行具体配置。

17.2 LOGO 图片转换 PIC2ARGB 工具

将常用的图片如 JPG、PNG 转换为 ARGB 格式。使用方法：

1. 双击 PIC2ARGB.exe
2. 将源图片拖拽到窗口上部的图片预览区域

3. 程序下方选项中，`data`选择`ARGB15555`
4. 使用鼠标将程序窗口上部的图片预览区域拖拽到程序窗口外面任意地方，会生成转换好的文件

选项说明：

- file
  - Hex: 生成的文件为二进制格式
  - Array：生成的文件为一个头文件，将图片数据保存为 C 语言数组

 版权所有 @ 上海富瀚微电子股份有限公司

18 附录 RTTv3 兼容接口

18.1 Posix 接口支持

RTTv3 支持的 Posix 接口列表如下：

序号	接口名称	功能	备注
1	clock_getres	获取时钟精度	仅支持 CLOCK_REALTIME/CLOCK_MONOTONIC
2	clock_gettime	获取当前时间	仅支持 CLOCK_REALTIME/CLOCK_MONOTONIC
3	clock_settime	设置当前时间	仅支持 CLOCK_REALTIME
4	difftime	计算 2 个时间的差值	全功能支持
5	asctime	时间转换为字符串	全功能支持
6	asctime_r	时间转换为字符串	全功能支持
7	ctime	UNIX 时间转换为字符串	全功能支持
8	ctime_r	同上，可重入版本	全功能支持
9	localtime	UNIX 时间转换为当地时间	全功能支持
10	localtime_r	同上，可重入版本	全功能支持
11	tzset	设置系统当前时区	全功能支持
12	gmtime	把 UNIX 时间转换为 GMT 时间	全功能支持
13	gmtime_r	同上，可重入版本	全功能支持
14	mktime	把时间转换为 UNIX 时间	全功能支持
15	strftime	格式化时间	全功能支持
16	strptime	格式化时间	全功能支持
17	time	获取系统当前的 UNIX 时间	全功能支持
18	gettimeofday	获取当前的系统时间	全功能支持
19	settimeofday	设置当前的系统时间	全功能支持
20	tempnam	生成临时文件名称	全功能支持
21	vsscanf	解析字符串	全功能支持
22	fgets	从文件读取字符串	全功能支持
23	freopen	打开文件	全功能支持
24	getc_unlocked	非阻塞读入字符	全功能支持
25	getchar	读入字符	全功能支持
26	getchar_unlocked	非阻塞读入字符	全功能支持
27	gets	读入字符串	全功能支持
28	perror	打印错误字符串	全功能支持
29	asprintf	格式化字符串	全功能支持
30	clearerr	清除文件错误	全功能支持
31	fclose	关闭文件	全功能支持
32	fopen	打开文件	全功能支持
33	fdopen	打开文件	全功能支持
34	fread	读取文件	全功能支持
35	fwrite	写文件	全功能支持
36	fgetc	从文件读取字符	全功能支持
37	fputc	写字符到文件	全功能支持
38	feof	检测文件结尾	全功能支持
39	ferror	获取文件 I/O 错误	全功能支持
40	fflush	刷新文件缓存	全功能支持
41	fgetpos	获取文件当前位置	全功能支持
42	fileno	获取文件句柄	全功能支持
43	fprintf	格式化输出到文件	全功能支持

 版权所有 @ 上海富瀚微电子股份有限公司

序号	接口名称	功能	备注
44	fputs	写字符串到文件	全功能支持
45	fseek	移动文件指针	全功能支持
46	fseeko	移动文件指针	全功能支持
47	fsetpos	移动文件指针	全功能支持
48	ftell	获取文件位置	全功能支持
49	ftello	获取文件位置	全功能支持
50	printf	格式化字符串	全功能支持
51	putc	输出字符	全功能支持
52	putc_unlocked	输出字符	全功能支持
53	putchar	输出字符	全功能支持
54	putchar_unlocked	非阻塞输出字符	全功能支持
55	puts	输出字符串	全功能支持
56	remove	删除文件	全功能支持
57	rewind	复位文件指针	全功能支持
58	scanf	读取数据	全功能支持
59	setvbuf	配置流缓存	全功能支持
60	snprintf	格式化字符串	全功能支持
61	sprintf	格式化字符串	全功能支持
62	sscanf	从字符串读取数据	全功能支持
63	tmpnam	生成临时文件名称	全功能支持
64	ungetc	回退字符到流	全功能支持
65	vfscanf	从文件中格式化读取	全功能支持
66	vfprintf	格式化输出到文件	全功能支持
67	vprintf	格式化输出	全功能支持
68	vsnprintf	格式化输出	全功能支持
69	vsprintf	格式化输出	全功能支持
70	getpid	获取进程 ID	全功能支持
71	gethostname	获取主机名称	全功能支持
72	getcwd	获取当前的工作目录	全功能支持
73	read	读取文件	全功能支持
74	write	写文件	全功能支持
75	sync	刷新系统缓存	有限支持
76	gettid	获取线程 ID	全功能支持
77	mq_open	创建/打开消息队列	仅支持 O_RDWR/O_CREAT
78	mq_close	关闭消息队列	全功能支持
79	mq_unlink	删除消息队列	全功能支持
80	mq_send	向消息队列发送消息	全功能支持
81	mq_receive	从消息队列收取消息	全功能支持
82	mq_timedsend	向消息队列发送消息，支持超时	全功能支持
83	mq_timedreceive	从消息队列收取消息，支持超时	全功能支持
84	mq_getattr	获取消息队列的属性配置	全功能支持
85	mq_setattr	设置消息队列的属性配置	全功能支持
86	pthread_attr_getschedparam	获取 pthread 属性中的调度参数	仅支持优先级调整
87	pthread_attr_setschedparam	设置 pthread 属性的调度参数	仅支持优先级调整
88	pthread_attr_getscope	获取 pthread 属性域	仅支持 PTHREAD_SCOPE_SYSTEM
89	pthread_attr_setscope	设置 pthread 属性的域	仅支持 PTHREAD_SCOPE_SYSTEM
90	pthread_attr_init	pthread 属性初始化	全功能支持
91	pthread_attr_destroy	销毁 pthread 属性	全功能支持
92	pthread_attr_setdetachstate	设置 pthread 属性为分离状态	全功能支持
93	pthread_attr_getdetachstate	获取 pthread 属性为分离状态	全功能支持
94	pthread_attr_getschedpolicy	获取 pthread 属性中的调度策略	全功能支持，OS 仅支持基于优先级调度
95	pthread_attr_setschedpolicy	设置 pthread 属性中的调度策略	全功能支持
96	pthread_attr_getstacksize	获取 pthread 属性中的栈大小	全功能支持
97	pthread_attr_setstacksize	设置 pthread 属性中的栈大小	全功能支持

序号	接口名称	功能	备注
98	pthread_attr_getstackaddr	获取 pthread 属性中的栈地址	全功能支持
99	pthread_attr_setstackaddr	设置 pthread 属性中的栈地址	全功能支持
100	pthread_attr_setstack	设置 pthread 栈属性	全功能支持
101	pthread_attr_getstack	获取 pthread 栈属性	全功能支持
102	pthread_attr_getguardsize	获取 pthread 属性中的 guardsize	仅限接口，返回不支持
103	pthread_attr_setguardsize	设置 pthread 属性中的 guardsize	仅限接口，返回不支持
104	pthread_create	创建 pthread 线程	全功能支持
105	pthread_exit	pthread 线程退出	全功能支持
106	pthread_join	等待 pthread 线程终止	全功能支持
107	pthread_kill	杀死 pthread 线程	全功能支持
108	pthread_cancel	取消 pthread 线程	全功能支持
109	pthread_detach	分离 pthread 线程	全功能支持
110	pthread_equal	判断是否是相同线程	全功能支持
111	pthread_getschedparam	获取线程的调度参数	全功能支持
112	pthread_atfork	pthread 线程创建钩子	仅限接口支持，返回不支持
113	pthread_once	单次线程	全功能支持
114	pthread_self	获取 pthread 线程句柄	全功能支持
115	pthread_setcanceltype	设置 pthread 线程的取消类型	全功能支持
116	pthread_setcancelstate	设置 pthread 线程的取消状态	全功能支持
117	pthread_setschedparam	设置 pthread 线程的调度参数	全功能支持
118	pthread_testcancel	测试 pthread 线程是否被取消	全功能支持
119	pthread_cleanup_push	注册 pthread 清理函数	全功能支持
120	pthread_cleanup_pop	注销 pthread 清理函数	全功能支持
121	pthread_mutex_init	pthread 互斥锁初始化	全功能支持
122	pthread_mutex_destroy	pthread 互斥锁销毁	全功能支持
123	pthread_mutex_lock	pthread 互斥锁锁定	全功能支持
124	pthread_mutex_unlock	pthread 互斥锁解锁	全功能支持
125	pthread_mutex_trylock	pthread 互斥锁尝试锁定	全功能支持
126	pthread_mutexattr_init	pthread 互斥锁属性初始化	全功能支持
127	pthread_mutexattr_destroy	pthread 互斥锁属性销毁	全功能支持
128	pthread_mutexattr_gettype	pthread 互斥锁获取类型	全功能支持
129	pthread_mutexattr_settype	pthread 互斥锁设置类型	全功能支持
130	pthread_mutexattr_getpshared	pthread 互斥锁属性获取共享特征	全功能支持
131	pthread_mutexattr_setpshared	pthrad 互斥锁属性设置共享特征	全功能支持
132	pthread_condattr_init	pthread 条件属性初始化	全功能支持
133	pthread_condattr_destroy	pthread 条件属性销毁	全功能支持
134	pthread_condattr_getclock	获取 pthread 条件属性的时钟配置	仅限接口支持
135	pthread_condattr_setclock	配置 pthread 条件属性中的时钟特征	仅限接口支持
136	pthread_cond_init	pthread 条件变量初始化	仅支持进程私有属性
137	pthread_cond_destroy	销毁 pthread 条件变量	全功能支持
138	pthread_cond_signal	发送 pthread 条件变量信号	全功能支持
139	pthread_cond_wait	pthread 条件等待	全功能支持
140	pthread_cond_timedwait	pthread 条件超时等待	全功能支持
141	pthread_cond_broadcast	pthread 条件广播	全功能支持
142	pthread_rwlockattr_init	pthread 读写锁属性初始化	仅支持进程私有属性
143	pthread_rwlockattr_destroy	销毁 pthread 读写锁属性	全功能支持
144	pthread_rwlockattr_setpshared	配置 pthread 读写锁共享属性	仅支持进程私有属性
145	pthread_rwlockattr_getpshared	获取 pthread 读写锁共享属性	仅支持进程私有属性
146	pthread_rwlock_init	pthread 读写锁初始化	全功能支持
147	pthread_rwlock_destroy	销毁 pthread 读写锁	全功能支持
148	pthread_rwlock_rdlock	pthread 读写锁读锁定	全功能支持
149	pthread_rwlock_tryrdlock	pthread 读写锁尝试读锁定	全功能支持
150	pthread_rwlock_timedrdlock	pthread 读写锁超时锁定	全功能支持
151	pthread_rwlock_wrlock	pthread 读写锁写锁定	全功能支持

版权所有 @ 上海富瀚微电子股份有限公司

序号	接口名称	功能	备注
152	pthread_rwlock_trywrlock	pthread 读写锁尝试写锁定	全功能支持
153	pthread_rwlock_timedwrlock	pthread 读写锁超时写锁定	全功能支持
154	pthread_rwlock_unlock	pthread 读写锁解锁	全功能支持
155	pthread_spin_init	pthread 自旋锁初始化	全功能支持
156	pthread_spin_destroly	销毁 pthread 自旋锁	全功能支持
157	pthread_spin_lock	pthread 自旋锁锁定	全功能支持
158	pthread_spin_trylock	pthread 自旋锁尝试锁定	全功能支持
159	pthread_spin_unlock	pthread 自旋锁解锁	全功能支持
160	pthread_barrierattr_init	pthread 障碍属性初始化	仅支持进程私有属性
161	pthread_barrierattr_destroy	销毁 pthread 障碍属性	全功能支持
162	pthread_barrierattr_getpshared	获取 pthread 障碍属性中的共享属性	全功能支持
163	pthread_barrierattr_setpshared	设置 pthread 障碍属性的共享属性	仅支持进程私有属性
164	pthread_barrier_init	pthread 障碍初始化	全功能支持
165	pthread_barrier_destroy	销毁 pthread 障碍	全功能支持
166	pthread_barrier_wait	pthread 障碍等待	全功能支持
167	poll	等待文件事件	全功能支持
168	select	等待文件事件	全功能支持
169	sched_get_priority_max	获取调度策略最高优先级	全功能支持
170	sched_get_priority_min	获取调度策略最低优先级	全功能支持
171	sched_yield	让出 CPU	全功能支持
172	sched_setscheduler	设置调度策略	全功能支持
173	sem_init	信号量初始化	全功能支持
174	sem_destroy	信号量销毁	全功能支持
175	sem_open	打开信号量	全功能支持
176	sem_close	关闭信号量	全功能支持
177	sem_unlink	删除信号量	全功能支持
178	sem_wait	等待信号量	全功能支持
179	sem_timedwait	超时等待信号量	全功能支持
180	sem_trywait	尝试等待信号量	全功能支持
181	sem_post	释放信号量	全功能支持
182	sem_getvalue	获取信号量值	全功能支持
183	va_arg	可变参数	全功能支持
184	va_start	开始可变参数	全功能支持
185	va_copy	可变参数拷贝	全功能支持
186	va_end	可变参数结束	全功能支持
187	malloc	分配内存	全功能支持
188	free	释放内存	全功能支持
189	calloc	分配内存并清零	全功能支持
190	realloc	重新分配内存	全功能支持
191	div	除法	全功能支持
192	ldiv	长整数除法	全功能支持
193	getenv	获取环境变量	全功能支持
194	mblen	多字节字符串长度	全功能支持
195	mbstowcs	多字节字符转换为宽字符	全功能支持
196	wctomb	宽字符转多字节字符	全功能支持
197	abs	绝对值	全功能支持
198	atoi	字符串转整数	全功能支持
199	atol	字符串转长整数	全功能支持
200	atoll	字符串转 64 位长整数	全功能支持
201	bsearch	二分法搜索	全功能支持
202	labs	长整数绝对值	全功能支持
203	llabs	64 位数字绝对值	全功能支持
204	rand	随机数初始化	全功能支持
205	srand	随机数初始化	全功能支持

 版权所有 @ 上海富瀚微电子股份有限公司

序号	接口名称	功能	备注
206	random	生成随机数	全功能支持
207	srandom	生成随机数	全功能支持
208	srand48	随机数初始化	全功能支持
209	strtod	字符串转浮点数	全功能支持
210	strtof	字符串转浮点数	全功能支持
211	strtol	字符串转长整数	全功能支持
212	strtold	字符串转浮点数	全功能支持
213	strtoll	字符串转 64 位长整数	全功能支持
214	strtoul	字符串转无符号长整数	全功能支持
215	strtoull	字符串转 64 位无符号长整数	全功能支持
216	qsort	快速排序	全功能支持
217	mkstemp	生成临时文件	全功能支持, 系统资源有限, 可能导致失败
218	mkstemps	生成临时文件	全功能支持, 系统资源有限, 可能导致失败
219	wcstombs	宽字符转多字节字符	全功能支持
220	syslog	系统日志	应用层支持
221	openlog	打开系统日志	应用层支持
222	closelog	关闭系统日志	应用层支持
223	vsyslog	记录系统日志	应用层支持
224	setlogmask	设置系统日志过滤	全功能支持
225	strcat	连接字符串	全功能支持
226	strcoll	比较字符串	全功能支持
227	strerror	生成错误字符串	全功能支持
228	strncat	连接字符串	全功能支持
229	strnlen	字符串长度	全功能支持
230	strxfrm	字符串转换	全功能支持
231	strchr	字符串查找	全功能支持
232	strcmp	字符串比较	全功能支持
233	strcpy	字符串拷贝	全功能支持
234	strcspn	字符串前缀长度	全功能支持
235	strdup	字符串克隆	全功能支持
236	strlen	字符串长度	全功能支持
237	strcasecmp	字符串比较	全功能支持
238	strcasestr	字符串搜索	全功能支持
239	strncmp	字符串比较	全功能支持
240	strncpy	字符串拷贝	全功能支持
241	strrchr	字符串搜索	全功能支持
242	strspn	字符串前缀长度	全功能支持
243	strstr	字符串搜索	全功能支持
244	strtok	字符串搜索	全功能支持
246	strtok_r	字符串搜索, 可重入	全功能支持
246	strerror_r	生成错误字符串, 可重入版本	全功能支持
247	memchr	内存搜索	全功能支持
248	memcmp	内存比较	全功能支持
249	memcpy	内存拷贝	全功能支持
250	memset	内存填充	全功能支持
251	memmove	内存移动	全功能支持
252	tolower	转小写	全功能支持
253	toupper	转大写	全功能支持
254	isalnum	字母 or 数字判断	全功能支持
255	isascii	ASCII 字符判断	全功能支持
256	isblank	空格判断	全功能支持
257	iscntrl	控制字符判断	全功能支持
258	isdigit	数字判断	全功能支持
259	isgraph	制表符判断	全功能支持

序号	接口名称	功能	备注
260	islower	小写判断	全功能支持
261	isprint	可打印字符判断	全功能支持
262	ispunct	非空格/字母/数字判断	全功能支持
263	isspace	空格判断	全功能支持
264	isupper	大写判断	全功能支持
265	isxdigit	16 进制字符判断	全功能支持
266	toascii	转 ASCII 字符	全功能支持
267	bzero	清零	全功能支持
268	offsetof	结构体偏移	全功能支持
269	creat	创建文件	全功能支持
270	opendir	打开目录	全功能支持
271	closedir	关闭目录	全功能支持
272	readdir	读取目录	全功能支持
273	telldir	获取目录位置	全功能支持
274	seekdir	移动目录指针	全功能支持
275	rewinddir	重置目录指针	全功能支持
276	getrlimit	获取系统资源限制	有限支持
277	getrusage	获取资源使用情况	有限支持
278	readv	读取数据	仅支持 socket
279	writev	写数据	仅支持 socket
280	socket	创建 socket	支持 IPv4/IPv6，有限支持 RAW
281	bind	绑定 socket	有限支持 RAW
282	accept	接受 socket 连接	全功能支持
283	connect	连接远程 socket	全功能支持
284	getpeername	获取对端名称	全功能支持
285	getsockname	获取 socket 名称	全功能支持
286	getsockopt	获取 socket 选项	仅支持部分 socket 属性
287	recv	从 socket 收取数据	全功能支持
288	recvfrom	从 socket 收取数据	全功能支持
289	send	发送数据	全功能支持
290	sendto	发送数据到指定目标	全功能支持
291	setsockopt	设置 socket 属性	仅支持部分 socket 属性
292	inet_aton	socket 地址转换	全功能支持
293	inet_addr	socket 地址转换	全功能支持
294	inet_ntoa	socket 地址转换	全功能支持
295	inet_ntop	socket 地址转换	全功能支持
296	inet_pton	socket 地址转换	全功能支持
297	strpbrk	字符串搜索	全功能支持
298	access	文件存取属性判断	全功能支持
299	lseek	文件读写指针的移动	全功能支持
300	lseek64	文件读写指针的移动	全功能支持
301	open	打开文件	全功能支持
302	close	关闭文件	全功能支持
303	fcntl	操纵文件句柄属性	仅支持 FLAG
304	ftruncate	截断文件	全功能支持
305	rename	重命名文件	全功能支持
306	unlink	删除文件	全功能支持
307	stat	获取文件状态	全功能支持
308	fstat	获取文件状态	全功能支持
309	ioctl	操纵流设备	全功能支持
310	mkdir	创建文件夹	全功能支持
311	rmdir	删除文件夹	全功能支持
312	chdir	改变工作路径	全功能支持

版权所有 @ 上海富瀚微电子股份有限公司

18.2 不支持的 POSIX 接口

以下 POSIX 接口目前不被支持：

 版权所有 @ 上海富瀚微电子股份有限公司

序号	接口名称	功能	备注
1	clock	获取 CPU 时间	
2	times	设置 CPU 时间	
3	posix2time		
4	time2posix		
5	timegm	local 时间转 GM	
6	timelocal	GM 时间转 local	
7	getdelim	从文件读取内容	
8	getline	从文件读取行	
9	tmpfile	生成临时文件	
10	flockfile	锁定文件	
11	ftrylockfile	尝试锁定文件	
12	funlockfile	解锁文件	
13	dlclose	关闭动态加载文件	
14	dlopen	打开动态加载文件	
15	dlsym	获取动态加载文件中的符号	
16	dlerror	获取动态加载错误	
17	alarm	设置警告通知	
18	_exit	退出	
19	execve	执行程序	
20	fchown	改变文件属主	
21	fork	创建进程	
22	nice	调整进程优先级	
23	pipe	管道	
24	readlink	读取链接	
25	pread	读取文件	
26	pwrite	写入文件	
27	mq_notify	通知消息队列	
28	pthread_attr_getinheritsched	pthread 线程的继承属性	pthread 线程不支持继承属性
29	pthread_attr_setinheritsched	pthread 线程的继承属性	
30	pthread_getspecific	pthread 线程的特别数据	
31	pthread_key_create	pthead 健	
32	pthread_key_delete	pthead 健	
33	pthread_setspecific	pthread 线程的特别数据	
34	pthread_mutex_timedlock	pthread 互斥锁超时锁定	
35	pthread_mutex_getprioceiling	pthread 互斥锁最高优先级	
36	pthread_mutex_setproiceiling	pthread 互斥锁最高优先级	
37	pthread_mutexattr_setprioceiling	pthread 互斥锁最高优先级	
38	pthread_mutexattr_getprioceiling	pthread 互斥锁最高优先级	
39	pthread_mutexattr_getprotocol	pthread 互斥锁协议	
40	pthread_mutexattr_setprotocol	pthread 互斥锁协议	
41	pthread_condattr_getpshared	pthread 条件变量共享属性	条件变量仅支持私有属性
42	pthread_condattr_setpshared	pthread 条件变量共享属性	
43	cond_resched	让出 CPU	
44	signal_pending	发送信号	RTOS 不支持信号
45	kill	杀死线程	
46	pthread_sigmask	pthread 信号掩码	
47	raise	生成异常	
48	signal	发送信号	RTOS 不支持信号
49	sigaction	信号处理	RTOS 不支持信号
50	sigaddset	信号处理	RTOS 不支持信号
51	sigdelset	信号处理	RTOS 不支持信号
52	sigemptyset	信号处理	RTOS 不支持信号
53	sigismember	信号处理	RTOS 不支持信号
54	sigpending	信号处理	RTOS 不支持信号

 版权所有 @ 上海富瀚微电子股份有限公司

序号	接口名称	功能	备注
55	sigprocmask	信号处理	RTOS 不支持信号
56	sigqueue	信号处理	RTOS 不支持信号
57	sigsuspend	信号处理	RTOS 不支持信号
58	sigtimedwait	信号处理	RTOS 不支持信号
59	sigwait	信号处理	RTOS 不支持信号
60	sigwaitinfo	信号处理	RTOS 不支持信号
61	abort	异常中止	
62	atexit	进程退出钩子	
63	exit	进程退出	
64	system	系统调用	
65	memalign	内存对齐分配	
66	posix_memalign	posix 内存对齐分配	
67	waitpid	等待进程退出	
68	assert	断言	
69	fdopendir	打开文件夹	
70	readdir_r	读取文件夹	暂不支持重入
71	fstatfs	文件系统统计	
72	mmap	内存映射	
73	munmap	内存结束映射	
74	setmntent	访问文件系统文件	
75	getmntent	访问文件系统文件	
76	addmntent	访问文件系统文件	
77	endmntent	访问文件系统文件	
78	if_indextoname	网络接口序号转名称	
79	if_nametoindex	网络接口名称转序号	
80	hstrerror	生成网络错误字符串	
81	timer_create	创建定时器	不支持 POSIX 定时器
82	timer_delete	销毁定时器	
83	timer_gettime	获取定时器时间	
84	timer_settime	设置定时器超时时间	
85	timer_getoverrun	获取定时器超时运行时间	

18.3 Linux 接口支持

其它 Linux 接口：

序号	接口名称	功能	备注
1	mount	安装文件系统	
2	umount	卸载文件系统	
3	msleep	睡眠等待	
4	prctl	设置进程属性	仅支持修改进程名称
5	stime	设置系统时间	
6	reboot	重启	有限支持
7	statfs	文件系统统计	全功能支持
8	sysinfo	系统信息	有限支持

 版权所有 @ 上海富瀚微电子股份有限公司

19 附录 RTTv3 组件 License 列表

RTTv3 中使用了部分外部组件，其中一部分为可选组件，另外一部分则与系统紧密相关，现将各组件的 License 列举如下：

组件名称	License	说明
RTOS	Apache License	系统内核，必选
FAL	GPLv2	Flash 驱动代码，必选
LZOP	GPLv2	分段加载解压代码，可选 GZIP/LZMA，应用可选 (三选一)
ZLIB	AS IS/Free	分段加载解压代码，可选 LZOP/LZMA，应用可选 (三选一)
LZMA	GPLv2.1	分段加载解压代码，可选 LZOP/GZIP，应用可选 (三选一)
bonjour	Apache License v2.0	组播域名服务/设备发现，应用可选
cJSON	AS IS /Free	字符串解析，应用可选
iperf3	BSD	网络性能测试，应用可选
iperf2	Free	网络性能测试，应用可选
libcurl	AS IS /Free	网络资源下载，应用可选
mbedtls	Apache License v2	嵌入式网络完全，应用可选
miniupnpc	AS IS /Free	端口映射，应用可选
mxml	Apache License v2.0	XML 语法解析，应用可选
srt	Mozilla Public License V2.0	开源视频传输协议，应用可选
udt	AS IS / Free	基于 UDP 的数据传输协议，应用可选
zbar	LGPLv2.1	二维码扫描，应用可选
optparse	MIT / AS IS	命令参数解析，可选
vi	GPLv2	文本编辑，可选
LwIP	AS IS	TCP/IP 协议栈, 可选

 版权所有 @ 上海富瀚微电子股份有限公司